22603A



Klein, Texas

Date: 2/25-26/2026 Event Name: Texas V5RC High School Region 3 Championship

Innovate Award Submission Information Form

Instructions: Fill out all information, printing clearly. This form should be included immediately after the Engineering Notebook's cover page. In the case of physical notebooks, this form can be printed out and placed in the notebook. For digital notebooks, this form can be scanned in and included. Teams may only submit one aspect of their design to be considered for this award at each event. Submission of multiple aspects will nullify the team's consideration for this award.

Full Team Number: 22603A

Brief description of the novel aspect of the team's design:

Our team noticed that defense is a major aspect of this game and one of the most important parts of defense is descoring and being able to do so before our opponents can stop us. To help give ourselves more time, we have created a system to be able to verbally queue our driver without our opponents knowing. Many different colors were added to our verbal commands include:

Green- We made our descore arm partially green, and the claw on our controller scuffs is also green. This allows us to call out to our driver "green" when we want to descore, and if our opponents hear us, they will not immediately process that we are trying to descore, which gives us an additional 1-2 seconds of time to pull our strategy off.

Red- We added red tape to our match loader and pistons. In addition, our 3d printed paddle implemented on our controller has a red arm, which when hit, activates our match loader to flip down. This allows for our co-driver to give short and fast verbal commands to our driver to load up on blocks.

Blue- We made our goal liner out of blue polycarbonate. This directly corresponds to our "blue" command. In addition to calling out the color blue, we also implemented direction to our scoring command. This gives the driver precise instructions on where to score efficiently. Having this order gives our robot a competitive advantage against other teams, since they will not know what we are saying.

Identify the page numbers and/or the section(s) where documentation of the development of this aspect can be found:

11/9/2025 - 1st Competition Recap.....68
11/11/2025 - Descore Brainstorming.....69
11/13/2025 - Descore Selection.....70
11/15/2025 - Descore Design Plan.....72
11/16/2025 - Descore: Testing.....73
12/13/2025 - OnAlaska Competition.....88
10/20/2025 - Matchloader Design Plan.....50
12/2/2025 - Co-Driving Analysis.....79

Explain why your submission is unique from other approaches to the problem it solves or task it performs:

Our submission is unique because it allows our driver to be effectively drive coached without compromising our strategy. Other teams have to rely on the driver to see the opportunity for descore, score, and load themselves, taking focus away from their current task to look around and spot it, or they would have to rely on the drive coaches to be quiet and subtle enough when saying commands that the opponents do not risk overhearing. However, given how hectic matches can be, the driver might not hear them under these conditions, and they won't get the instructions. Our method of communication and visual queues effectively eliminate both of these potential issues.

Table of Contents

Innovate Award Submission Information Form..... 2
8/13/2025 - Initial Meeting..... 1
8/14/2025 - Game Analysis..... 2
8/15/2025 - V1 Design Cycle: Identify Requirements..... 5
8/15/2025 - V1 Design Cycle: Gantt Chart..... 6
8/16/2025 - Drive Brainstorming: Drive Type..... 7
8/17/2025 - Drive Selection: Drive Type..... 9
8/16/2025 - Drive Brainstorming: Wheel Size/Gear Ratio..... 11
8/17/2025 - Drive Selection: Wheel Size/Gear Ratio..... 12
8/18/2025 - Intake Brainstorming: Intake Type..... 14
8/19/2025 - Intake Selection: Intake Type..... 15
8/18/2025 - Intake Brainstorming: Block Holder..... 17
8/19/2025 - Intake Selection: Block Holder..... 18
8/22/2025 - Matchloader Brainstorming..... 20
8/25/2025 - Matchloader Selection..... 22
8/29/2025 - Drive Design Plan..... 24
9/1/2025 - September Plans..... 26
9/2/2025 - Intake/Block Holder Design Plan..... 27
9/5/2025 - Matchloader Design Plan..... 29
9/18/2025 - Robot Construction: Frame..... 30
9/18/2025 - Robot Construction: Drive..... 32
9/18/2025 - Robot Construction: Intake..... 33
9/18/2025 - Robot Construction: Matchloader..... 35
9/20/2025 - V1 Driver Control Programming..... 36
9/22/2025 - Drive: Testing..... 37
9/24/2025 - Intake: Testing..... 39
9/27/2025 - Matchloader: Testing..... 40
10/1/2025 - October Plans..... 41
10/2/2025 - V2 Design Cycle: Identify Requirements..... 42
10/5/2025 - V2 Intake Brainstorming..... 43
10/10/2025 - V2 Intake Selection..... 44
10/13/2025 - Matchloader Brainstorming..... 46
10/15/2025 - Matchloader Selection..... 47
10/17/2025 - Intake Design Plan..... 49
10/20/2025 - Matchloader Design Plan..... 51
10/29/2025 - V2 Robot Construction..... 53
10/30/2025 - V2 Driver Control Programming..... 59

11/1/2025 - November Plans.....60
11/2/2025 - Intake: Testing.....61
11/3/2025 - Intake: Tuning.....62
11/4/2025 - Matchloader: Testing.....64
11/5/2025 - Matchloader: Tuning.....66
11/9/2025 - 1st Competition Recap.....69
11/11/2025 - Descore Brainstorming.....70
11/13/2025 - Descore Selection.....71
11/15/2025 - Descore Design Plan.....73
11/16/2025 - Descore: Testing.....74
11/17/2025 - Descore: Tuning.....76
11/18/2025 - Autonomous Strategies.....78
12/1/2025 - December Plans.....79
12/2/2025 - Co-Driving Analysis.....80
12/4/2025 - Skills Practice - Driver.....82
12/8/2025 - Skills Practice - Autonomous.....84
12/13/2025 - OnAlaska Competition.....89
12/15/2025 - Drivetrain Brainstorm: Rpm.....91
12/17/2025 - Drive Selection: RPM.....92
12/18/2025 - Drive RPM: Design plan.....94
12/19/2025 - Drive RPM: Testing.....95
12/20/2025 - Drivetrain: Tuning.....97
12/21/2025 - Program: break down.....98
12/22/2025 - Program: Testing.....101
12/23/2025 - Programing: Tuning.....103
1/07/2026 - January Plans.....105
1/8/2026 - Bridgeland Skills.....106
1/12/2026 - Mid goal score: Brainstorm.....107
1/13/2026 - Mid goal score: Selection.....108
1/14/2026 - Mid goal score: Design plan.....110
1/15/2026 - Mid goal score: Testing.....112
1/16/2026 - Mid goal: Tuning.....114
1/17/2026 - Bridgeland 2v2, Skills.....116
1/20/2026 - Poly sleds: brainstorming.....118
1/21/2026 - Poly Sleds: Selection.....120
1/22/2026 - Poly Sleds: Testing.....122
1/22/2026 - Poly Sleds: Tuning.....125
1/24/2026 - Autonomous Skills: Testing.....126

1/27/2026 - Autonomous 2v2: Improvements..... 129
1/28/2026 - Win Point Break Down..... 131
1/30/2026 - Mechanically InKleined..... 132
2/1/2026 - February Plans..... 134
2/2/2026 - Descore Improvements: Brainstorming..... 135
2/3/2026 - Descore Improvements: Selection..... 136
2/4/2026 - Descore Improvements: Testing..... 138
2/5/2026 - VEX UIL Texas State Championship 6A..... 140
2/6/2026 - Code: Overhaul and Explanation..... 142
2/9/2026 - Mid Goal Score v2: Brainstorming..... 157
2/10/2026 - Mid Goal Scoring V2: Selection..... 158
2/11/2026 - Mid Goal Scoring V2: Design Plan..... 160
2/16/2026 - Mid Goal Scoring V2: Testing..... 161
2/17/2026 - Mid goal: Tuning..... 163
2/18/2026 - PID: Tuning..... 164
Notebook Appendix..... 167

8/13/2025 - Initial Meeting

Meeting Goal: Establish teams, competition plans, estimated budget, and assign roles.

Participants: David, Alex, Hamsa, Andrew, Austin, Jenea

Estimated competition outline:

1. Bridgeland Skills - 11/8/2025
    a. Cost: $10.00
2. Onalaska - 12/13/2025
    a. Cost: $105.00

Estimated budget:

1. Competition costs: $115.00
2. Parts costs: $50

Team List & Roles:

Mechanical team → David, Alex, Austin
Programing team → Hamsa, Andrew
Research/Notebook team → Austin, Jenea

Gantt Chart:

	Project start date:	8/13/25		Project name:	Push Back Robot		Day:	8/13	8/14
#	Activity	Assigned to	Start	End	Status	% Done	Progress Bar		
1	Set up Gantt Chart	Austin	8/13/2025	8/13/2025	Completed	100%			
2	Game Analysis	All	8/14/2025	8/14/2025	Not started	0%			

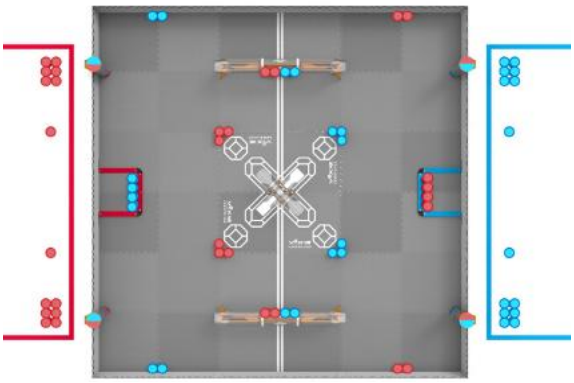
We have started our gantt chart based on the tasks we know we must complete in the future. Once we have our system outline for the V1 robot, we will update the timeline with this. Our final deadline for the V1 robot can be **no later than 11/8/2025,** which is the first competition day.

8/14/2025 - Game Analysis

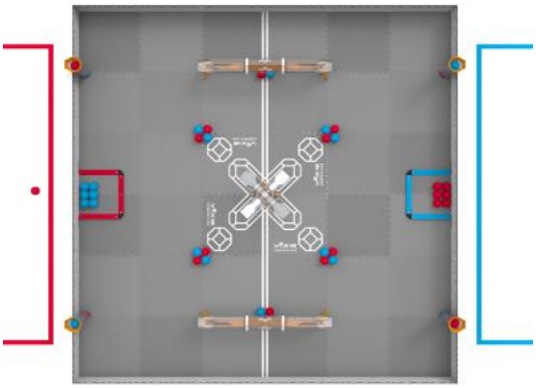
Meeting Goal: Figure out how to play Push Back.

Participants: David, Alex, Hamsa, Andrew, Austin, Jenea

Push Back Head-to-Head Match Field:



Push Back Skills Match Field:



Element 1: Blocks

Blocks are the primary element that robots interact with this year. There is no possession limit on blocks. There are red blocks that count for the red alliance and blue blocks that count for the blue alliance.

Important notes:

1. They are small enough to easily be able to store between 6 and 20. Our goal will be to hold at least 6-7 blocks at a time so we can completely fill the center goals in one go.
2. They are made of plastic, so we will need a grippy intake material to counteract this.



Citation: VEX Game Manual, Game Design Committee, Sept 4, V2.0, accessed 9/24/2025, https://content.vexrobotics.com/docs/25-26/v5rc-push-back/docs/Push-Back-2.0.pdf

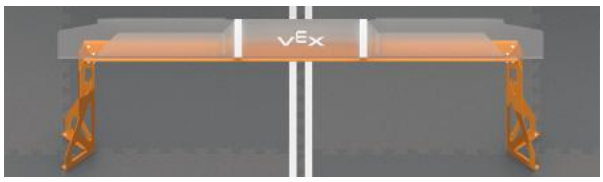
Material: Plastic
Qty: 44 red, 44 blue
Dimensions: 3.25in wide, 18 sides

Element 2: Long Scoring Tubes

These are long tubes located on each side of the field with a narrow passage between them and the wall. They have a slot at the top for descoring and can hold 15 blocks.

Important notes:

1. With the shape of the poles it appears likely an aligner will be needed for fast unloading.
2. Descore will be a major part for 2v2 play and will have to be incorporated in our design.



Citation: VEX Game Manual, Game Design Committee, Sept 4, V2.0, accessed 9/24/2025, https://content.vexrobotics.com/docs/25-26/v5rc-push-back/docs/Push-Back-2.0.pdf

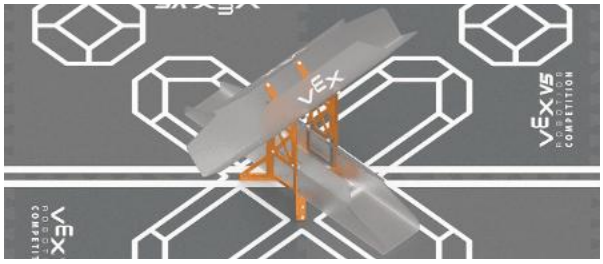
Material: Mix of metal and plastic
Qty: 2 long goals
Dimensions: 48.8” wide, 12.67 tall

Element 3: Middle Scoring Tubes

This goal has two parts to it: an upper and lower goal. The top goal has a control bonus of 8, while the bottom one has a control bonus of 6. They both hold 7 blocks each. There is a white tape design around the structuring, but this serves no purpose and isn't required.

Important notes:

1. The height of the bottom center goal seems to line up with the match loader so it can serve a double use.
2. The goal seems to be a harder scoring spot unless accounted for in the design with a center ball dispense.



Citation: VEX Game Manual, Game Design Committee, Sept 4, V2.0, accessed 9/24/2025, https://content.vexrobotics.com/docs/25-26/v5rc-push-back/docs/Push-Back-2.0.pdf

Material: Metal structure, plastic tubes
Qty: 1 center goal
Dimensions: 22.6” wide each, top height: 8in, bottom height: 0.5in

Element 4: Match Load tubes

There are two tubes at opposite sides of the drive station per alliance. They hold 6 blocks each. Along with these, each alliance gets 12 matchloads that can be introduced through either of these tubes.

Important notes:

1. There is a lip at the bottom of the match loader which still has to be accounted for in the design process.

Citation: VEX Game Manual, Game Design Committee, Sept 4, V2.0, accessed 9/24/2025, https://content.vexrobotics.com/docs/25-26/v5rc-push-back/docs/Push-Back-2.0.pdf

Material: Plastic with rubber funnel 2
Qty: 2 tubes per alliance
Dimensions: Gap: 3.94”, 21.34” tall, 3.94” diameter, .79” bottom lip

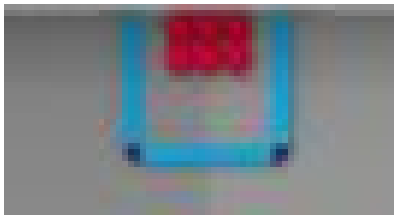


Element 5: Parking barrier

In between the match loading tubes on both sides of the field lies the red or blue parking zones. The red parking zone marks the starting spot for the skills competition. Depending on if it is skills or 2v2 the parking zone will hold 4-6 blacks each.

Important notes:

1. The amount in the barrier changes between skills and 2v2.
2. Stacked motors can not make it over the parking barrier.



Citation: VEX Game Manual, Game Design Committee, Sept 4, V2.0, accessed 9/24/2025, https://content.vexrobotics.com/docs/25-26/v5rc-push-back/docs/Push-Back-2.0.pdf

Material: Plastic
Qty: 1 per alliance
Dimensions: 19” by 16.75”, 1” tall

Scoring summary point list:

Autonomous: 10 Points

Each game Peice: 3 Points

Long Goal Control: 10 Points

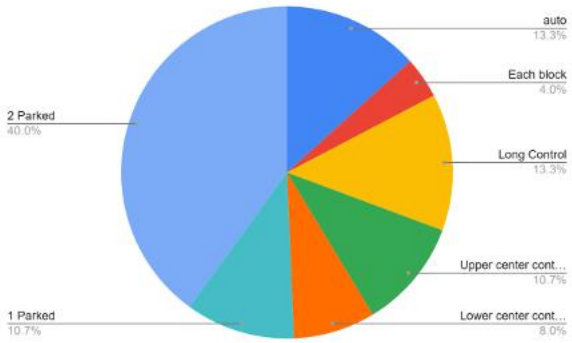
Upper Control: 8 Points

Lower Control: 6 Points

1 Parked: 8 Points

2 Parked: 30 Points

Scoring pie chart:



Scoring notes:

On paper double park appears to be an easy win but in the time it takes to double park the whole game can be descored. Our team believes that double park would be good for qualifications but not for tournament play. Our focus will be long goal control as it makes up for 13.3% of points in the game and if you have a teammate parked and one protecting a long goal you will have 24% of the points plus all other blocks scored and auto depending on the route we program.

Important Rules:

Robot limit rules:

1. Plastic limit rule **(<R25>)**
2. Expansion limit rule **(<SG2>)**
3. Motor limit rule **(<R12>)**

Don't do this rules:

1. Don't break robots rule
2. Don’t throw elements out of the field rule **(<SG4>)**
3. Don’t pin rule **(<GG17>)**
4. Don’t holecount rule **(<G4>)**

8/15/2025 - V1 Design Cycle: Identify Requirements

Meeting Goal: Figure out what is required from our robot.

Participants: David, Alex, Hamsa, Andrew, Austin, Jene

Drive:

Functions:

- Move around the field
  - Fast (target: 450 RPM)
  - Doesn't get pushed around
  - Good acceleration
  - Moves in multiple directions (target: forwards/backwards/turns)
- Parks
  - Single/double park
  - Parks within 5s from anywhere on the field

Component Summary:

- Drive type (brainstormed/selected)
- Wheel type (design choice in design plan)
- Wheel size (brainstormed/selected)
- Gear ratio (brainstormed/selected)

Intake:

Functions:

- Picks up/scores blocks
  - Fast (target: 1200 RPM)
  - Intakes 2 blocks at a time
  - Holds 7 blocks

Component Summary:

*Will highly depend on general concept requirements, but may include:

- Bottom stage (picking up blocks)
- Elevator (lifting blocks to given desired height)
- Outtake (scoring blocks)
- Material (design choice in design plan)

Matchloader:

Functions:

- Removes blocks from matchload tubes
  - Dispenses all 6 at once
  - Feeds directly into intake

Component Summary:

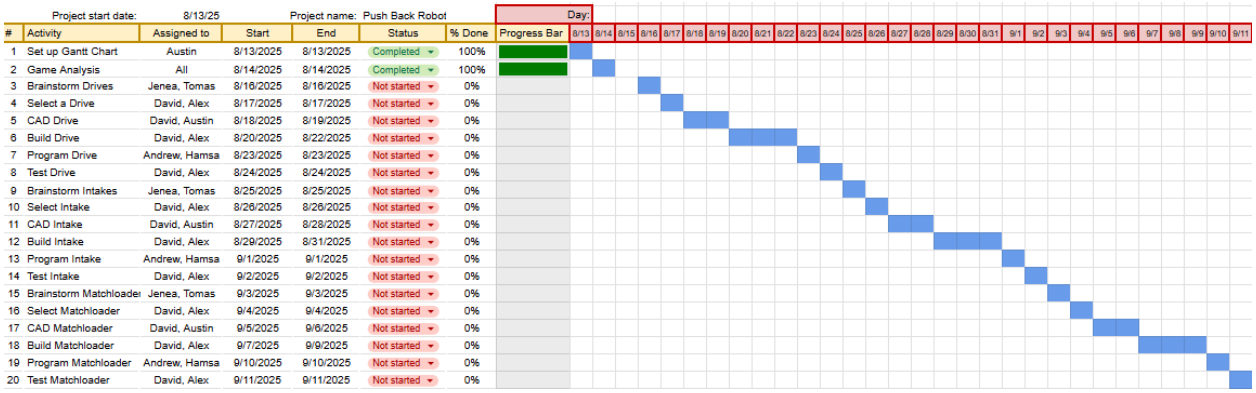
- Primary system only

All drawings unless cited are made by the 22603A Team

8/15/2025 - V1 Design Cycle: Gantt Chart

Meeting Goal: Figure out what our timeline for the V1 robot looks like.

Participants: David, Alex, Hamsa, Andrew, Austin, Jenea



Project start date: 8/13/25 Project name: Push Back Robot Day:

#	Activity	Assigned to	Start	End	Status	% Done	Progress Bar
1	Set up Gantt Chart	Austin	8/13/2025	8/13/2025	Completed	100%	
2	Game Analysis	All	8/14/2025	8/14/2025	Completed	100%	
3	Brainstorm Drives	Jenea, Tomas	8/16/2025	8/16/2025	Not started	0%	
4	Select a Drive	David, Alex	8/17/2025	8/17/2025	Not started	0%	
5	CAD Drive	David, Austin	8/18/2025	8/19/2025	Not started	0%	
6	Build Drive	David, Alex	8/20/2025	8/22/2025	Not started	0%	
7	Program Drive	Andrew, Hamsa	8/23/2025	8/23/2025	Not started	0%	
8	Test Drive	David, Alex	8/24/2025	8/24/2025	Not started	0%	
9	Brainstorm Intakes	Jenea, Tomas	8/25/2025	8/25/2025	Not started	0%	
10	Select Intake	David, Alex	8/26/2025	8/26/2025	Not started	0%	
11	CAD Intake	David, Austin	8/27/2025	8/28/2025	Not started	0%	
12	Build Intake	David, Alex	8/29/2025	8/31/2025	Not started	0%	
13	Program Intake	Andrew, Hamsa	9/1/2025	9/1/2025	Not started	0%	
14	Test Intake	David, Alex	9/2/2025	9/2/2025	Not started	0%	
15	Brainstorm Matchloader	Jenea, Tomas	9/3/2025	9/3/2025	Not started	0%	
16	Select Matchloader	David, Alex	9/4/2025	9/4/2025	Not started	0%	
17	CAD Matchloader	David, Austin	9/5/2025	9/6/2025	Not started	0%	
18	Build Matchloader	David, Alex	9/7/2025	9/9/2025	Not started	0%	
19	Program Matchloader	Andrew, Hamsa	9/10/2025	9/10/2025	Not started	0%	
20	Test Matchloader	David, Alex	9/11/2025	9/11/2025	Not started	0%	

Above is our gantt chart for the V1 robot. **The expected end date is 9/11/2025,** which gives us ~2 months to make any required adjustments and program our skills route for the robot. Tasks have been divided between team members according to their roles and time commitment to the team. We will update this chart when applicable.

8/16/2025 - Drive Brainstorming: Drive Type

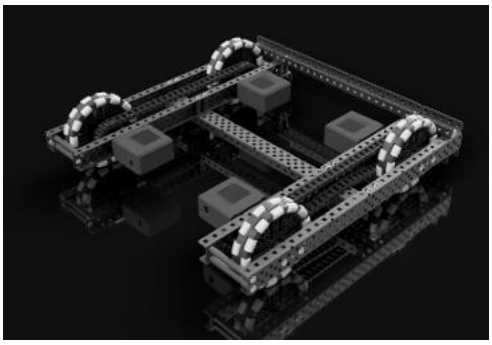
Meeting Goal: What options are available to us for moving our robot around on the field?

Participants: Austin, Jenea

Drive Types:

Tank Drive

The Tank Drive is considered the most basic drivetrain style; it has a good balance between configuration variation and overall simplicity. The wheels that are used are sandwiched between two C-channels and typically have a cut piece of C-channel to connect them. This is mirrored and the drive is connected with two cross beams, typically one on the edge and another located in the center. The overall build is square in formation. Due to being very simple in build complexity, it can be altered and customized fairly easily–such as the position for cross beams and wheels.



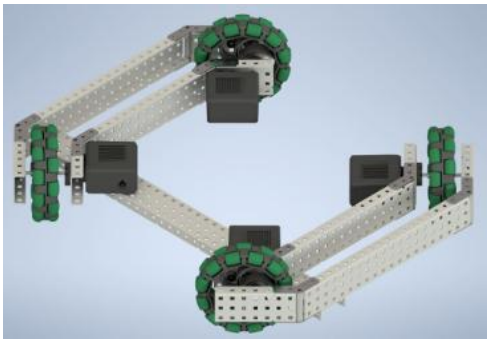
Citation: VEX Classic, Xenon27, Sept 2022, https://www.vexforum.com/t/creating-a-basic-drive/106873

Pros: Non-angled frame makes it simple to build, also allows for lots of customization through compatibility with many different gear ratios/wheel sizes/motor configurations

Cons: Cannot move sideways so left-to-right adjustment takes more time due to requiring moving backwards, turning, and then moving forwards

X-Drive

The X-Drive is known as the fastest among the 3 basic drive trains with a square formation. Unlike the tank drive, the motors are set up at forty-five degree angles. To achieve this, forty-five degree gussets are used to connect a long piece of C-channel to two pieces of smaller C-channel. Once again, a sandwich is created with the wheels and C-channels and a cross beam is utilized to keep the structure together. This unique configuration of the wheels makes the drive have more range in movement and better maneuverability. Because of the build needing the wheels to be at forty-five degrees, configuration customizability is less achievable.



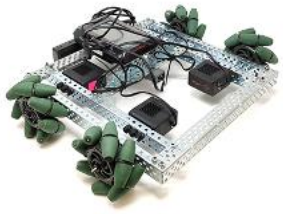
Citation: “Holonomic Drive - BLRS Wiki.” Purduesigbots.com, 2023, wiki.purduesigbots.com/hardware/vex-drivetrains/holonomic-drive.

Pros: Same speed forwards and sideways. Simple. Maneuverability.
Cons: Less space for mechanisms. Not much room

for configuration without losing performance.

Mecanum Drive

The Mecanum Drive is a combination of the frame setup of the tank drive and the maneuverability of the X-drive. This is achieved by using four mecanum wheels which consist of forty-five degree rollers and have little variation in size. Because of their unique design, each wheel must be individually driven. As well as this, the wheels are located directly outside of the frame rather than between pieces of C-channel, unlike the other two drives.



Citation: “CS-STEM Network.” Www.cs2n.org, www.cs2n.org/u/mp/badge_pages/104.

Pros: Moves sideways and forwards/backwards, harder to push

Cons: Little customizability/variation opportunity with motor count/wheel size/etc

8/17/2025 - Drive Selection: Drive Type

Meeting Goal: Of the brainstormed drive types above, what is the best to use?

Participants: Austin, Alex, David

Parameters:

Manuverability

- We want to be able to move in different directions to be able to quickly and efficiently move around the field.
- **Weighting:** This is our second highest weighted category at 0.26 because it is vital that we can get places easily, especially due to the fact that every competition category has a significant time limit.
- **Concept Scoring:**
  - ***Tank Drive: 3/5*** → *Can only move forwards/backwards and rotate, no sideways motions*
  - ***X-drive: 5/5*** → *Can move in any direction including side to side*
  - ***Mecanum Drive: 5/5*** → *Can move in any direction including side to side*

Complexity (building)

- We want a fairly simple drivetrain because this will take less time to build and leave more time for programming and driving. It also reduces the risk of building something wrong and simplifies repairs that happen at competitions.
- **Weighting:** This is our 3rd highest priority at 0.24 because we want a drive that we can easily work on and put together, but only as long as it does not negatively affect our performance. As long as we have enough time, we can figure out how to build any type of drive.
- **Concept Scoring:**
  - ***Tank Drive: 5/5*** → *Only requires 4 support rails with cross-bracing on top, plus we have a lot of familiarity with building them*
  - ***X-drive: 2/5*** → *Pods must be at 45 degree angles, which makes cross-bracing more difficult. There are also 4 pods instead of 2.*
  - ***Mecanum Drive: 3/5*** → *Can use a similar frame setup to tank, but each wheel has to be separated with its own motor, which makes it more complicated.*

Complexity (programming)

- We want to be able to program the drive fairly easily because it will leave more time for other tasks like driver practice and reduce the risk of issues arising. It will also give us more access to help if we select a commonly used drive type.
- **Weighting:** This is our lowest priority, at 0.22, because we have 2 dedicated programmers and programming drives take the lowest amount of time to figure out.
- **Concept Scoring:**
  - ***Tank Drive: 5/5*** → *We can use direct control power methods since the motors are all in line and working together. Also, all libraries have controls for tank and there is a lot of documentation if we need help.*

- ***X-drive: 2/5*** → *Since each pod is on its own motor, and the pods must be independently controlled to allow diagonal and side to side movements, this is significantly more complicated. There are also not many resources for help.*
- ***Mecanum Drive: 3/5*** → *Like the X-drive, wheels must be independently controlled, but the overall setup is a lot closer to tank drive setups.*

Customizability

- We want to be able to control how many motors are on the drive easily, as well as what our gear ratio is/how many wheels we have.
- **Weighting:** This has the highest weight of all the parameters at 0.28 because it affects the performance of the drive the most. We want the flexibility to choose a gear ratio with good speed, and we want flexibility on the number of motors we can use on the drive and where we have to put them.
- **Concept Scoring:**
  - ***Tank Drive: 5/5*** → *There are a ton of concepts that will work with this drive. Pretty much any gear ratio as long as the gears and motors fit within spacing with each other will work. There is an abundance of mounting holes as well.*
  - ***X-drive: 2/5*** → *Each pod has to have its own motor, so we are limited to either using 4 or 8 motors. Each gear ratio only has the length of a pod to fit within, since all pods have to have their own gearing.*
  - ***Mecanum Drive: 3/5*** → *Since all wheels have to be independent, we are similarly limited like the x-drive, but there's more space within the drive sides since the wheels are all in line.*

Decision Matrix:

	Weighting	Tank Drive		X Drive		Mecanum Drive	
Manuverability	0.26	3	0.78	5	1.30	5	1.30
Build Complexity	0.24	5	1.20	2	0.48	3	0.72
Programming Complexity	0.22	5	1.10	2	0.44	3	0.66
Customizability	0.28	5	1.40	2	0.56	3	0.84
Totals:	1.00		**4.48**		**2.78**		**3.52**

Conclusion:

We will be moving forward with the tank drive. This scored the highest weighted composite average, meaning it provides the most advantages for what we are looking for. The lowest score it received was a 3, which indicates that while we wish the maneuverability were higher, it is still satisfactory as is, so this is not a concern.

Next Steps:

Our next step is to brainstorm and select a wheel size/gear ratio combination.

8/16/2025 - Drive Brainstorming: Wheel Size/Gear Ratio

Meeting Goal: What options are available to us for moving our robot around on the field?

Participants: Austin, Jenea

Wheel Size/Gear Ratio Options:

3.25” at 450 RPM

Considered the most basic drivetrain style, It has a medium speed drivetrain which balances maneuverablitiy. This allows for steady control while moving game objects and helps with navigating the field.

Pros: Good all around, Very strong center of gravity

Cons: no apparent flaws, generally lower pushing force

2.75” at 600 RPM

Known as a very fast drive train. This drivetrain arganment optimizes quick and rapid movement. The rapid movement allows for repsositioning on the field, but this causes a loss of torque for speed.

Pros: High top speed
Cons: bad acceleration , bad pushing force

4” at 400 RPM

This drive train requires bigger gears which causes it to be a torque focused drivetrain. It provides stability and strong pushing capabilities. This drivetrain is ideal for controlling heavy loads or resisting force from opponents.

Pros: High pushing power, incredible parking cross

Cons: slightly lower speed

8/17/2025 - Drive Selection: Wheel Size/Gear Ratio

Meeting Goal: Of the brainstormed drive types above, what is the best to use?

Participants: Austin, Alex, David

Parameters:

Speed

- We want our bot to be able to move across the field as quickly as possible so that we can increase our chances of scoring high. As well as this, it is important during 2v2 competitions to beat the opposing side when it comes to both getting to the goals first and blocking them from scoring.
- **Weighting:** This has the highest weight of all the parameters at 0.28 because it strongly affects our performance in both Skills and 2v2 competitions
- **Concept Scoring:**
  - ***3.25" with 450 RPM: 5/5 →***
  - ***2.75" with 600 RPM: 5/5 →***
  - ***4" with 400 RPM: 4/5 →***

Acceleration

- For our bot's top speed to matter, we want it to reach that top speed fairly quickly. Having a quicker acceleration helps the bot get to its top speed and move across the field in a short amount of time.
- **Weighting:** 0.26
- **Concept Scoring:**
  - ***3.25" with 450 RPM: 5/5 →***
  - ***2.75" with 600 RPM: 2/5 →***
  - ***4" with 400 RPM: 5/5 →***

Pushing Force

- We want to be able to program the drive fairly easily because it will leave more time for other tasks like driver practice and reduce the risk of issues arising. It will also give us more access to help if we select a commonly used drive type.
- **Weighting:** 0.24
- **Concept Scoring:**
  - ***3.25" with 450 RPM: 4/5 →***
  - ***2.75" with 600 RPM: 2/5 →***
  - ***4" with 400 RPM: 4/5 →***

Complexity (building)

- We want a fairly simple drivetrain because this will take less time to build and leave more time for programming and driving. It also reduces the risk of building something wrong and simplifies repairs that happen at competitions.
- **Weighting:** 0.22
- **Concept Scoring:**
  - ***3.25" with 450 RPM: 5/5 →***

- ***2.75” with 600 RPM: 5/5 →***
- ***4” with 400 RPM: 5/5 →***

Decision Matrix:

	Weighting	3.25”/450 RPM		2.75”/600 RPM		4”/400 RPM	
Speed	0.28	5	1.40	5	1.40	4	1.12
Acceleration	0.26	5	1.30	2	0.52	5	1.30
Pushing Force	0.24	4	0.96	2	0.48	5	1.20
Build Complexity	0.22	5	1.10	5	1.10	5	1.10
Totals:	1.00		**4.76**		**3.56**		**4.72**

Conclusion:

We will be moving forward with the 3.25’ wheels with the 450 RPM. This scored the highest weighted composite average due to its overall rounded out design, meaning it provides the most advantages for what we are looking for. The lowest score it received was a 4 in pushing force, meaning that this parameter, while a little low, is still very satisfactory in its performance. We will move on to choosing a design for our intake which will be what collects the blocks on the field.

8/18/2025 - Intake Brainstorming: Intake Type

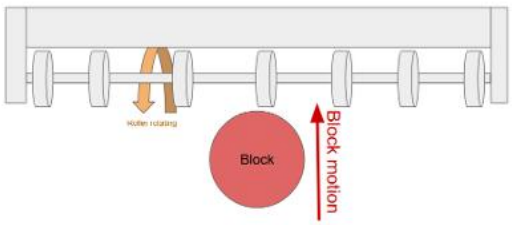
Meeting Goal: What options are available to us for picking up blocks?

Participants: Austin, Jenea

Intake Types:

Over top intake roller

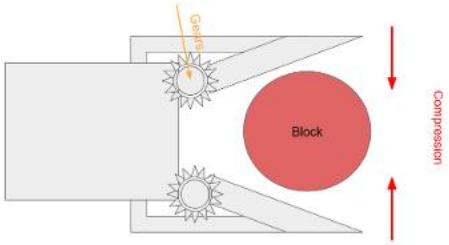
Rollers are positioned above the intake opening which pulls the block into the robot. The material options are flex wheels or rubber bands.



Pros: Fast load, simple, as wide as needed, higher load stability.
Cons: Jamming, axel bends, entanglement, compression needed.

Claw

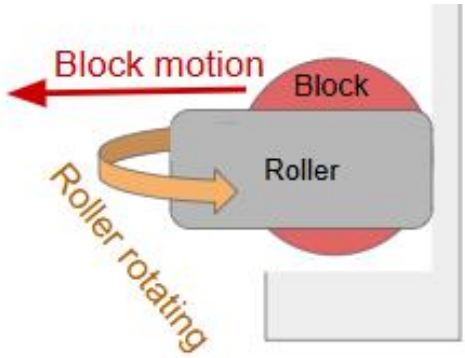
The claw is a gripping mechanism used to grab and hold objects of the game. This option does not allow for continuous cycling. But it allows for more control over elements.



Pros: easy pick up, simple, precision
Cons: slow load, inefficient, big, alignment

Side intake rollers

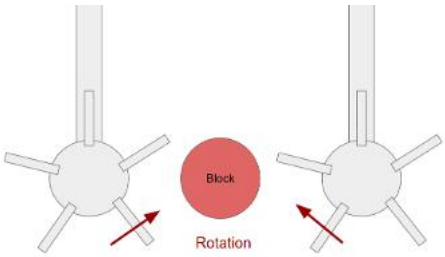
This is similar to over top intake rollers, but this is on the side and allows for the game objects to the center intake. This is good for bottom stage but requires a different top stage for propper lifting.



Pros: Reduced stalling, less motor stress, speed, alignment.
Cons: stress on structure, sizing, less capture area, lack of multi object capability.

Conveyor

This is a belt or chain driven system that carries game objects from the intake to the scoring area. The conveyor was popular in high stakes, but this year the game objects in the cente.



Pros: Simple, inexpensive
Cons: Not secure on the elements

8/19/2025 - Intake Selection: Intake Type

Meeting Goal: Of the brainstormed intake types above, what is the best to use?

Participants: Austin, David, Alex

Parameters:

Speed

- We want a fast intake to be able to pick blocks up quickly and score them quickly to score them before our opponents.
- **Weighting:** This is our highest weighted category at 0.28, because it affects our performance the most since we want to score as many blocks as possible to get as many points as possible.
- **Concept Scoring:**
    - ***Overtop Intake: 5/5*** → Super easy to change speed via gear ratio, and provides continuous cycling
    - ***Claw: 2/5*** → Difficult to change speed via gear ratio, and claw can only do 1 block at a time and must reset to a position before picking up another
    - ***Side Rollers: 3/5*** → Provides continuous cycling, but more difficult to increase speed via gear ratio because sides must be independently geared, or somehow linked across the robot.
    - ***Conveyor: 5/5*** → Super easy to change speed via gear ratio, and provides continuous cycling

Complexity (Building)

- We want an intake that is easy to build and tune because we are unfamiliar with how intakes will behave with the elements since this is the first one we are building this year.
- **Weighting:** This has the 3rd highest weighting at 0.24, because since this is our 1st robot we have plenty of time to figure out how to implement the concept.
- **Concept Scoring:**
    - ***Overtop Intake: 5/5*** → Consists only of rollers mounted in a line and geared or chained together, which is super easy to build and requires little structuring
    - ***Claw: 3/5*** → Requires more parts at various angles that must be connected certain ways to facilitate the correct motion for moving blocks
    - ***Side Rollers: 3/5*** → Fairly easy, but requires rollers on each side of the robot with nothing directly between so blocks can pass, which adds some complexity
    - ***Conveyor: 4/5*** → Spacing has to account for conveyor flaps, and a pre-roller would be required, but overall roller geometry should be very easy to build

Capacity

- We want to be able to intake multiple blocks at once. Having an intake that can do so will decrease the precision required to pick up blocks, making it faster for our driver and easier to pick up blocks in auton if the robot is slightly off as well.

- **Weighting:** This is our 2nd highest weighting at 0.25, because it affects the direct performance of our intake the 2nd most.
- **Concept Scoring:**
    - ***Overtop Intake: 5/5*** → The bottom roller can be made as wide as we want, and since contact is on the top of the block, we will maintain intaking capabilities all the way across the intake.
    - ***Claw: 1/5*** → Can only hold 1 block at once for a secure grip.
    - ***Side Rollers: 1/5*** → Since it requires contact on both sides of the block, only 1 block can be taken in at a time
    - ***Conveyor: 4/5*** → The conveyor flaps may create more complication/dead-spots, but the bottom pre-roller can be made to take in as many blocks as desired.

Grip

- We want the blocks to be secure in our intake so that they don't fall out. This will also help with our ability to grab them during matches.
- **Weighting:** This is our lowest weighted at 0.23 because many concepts have additional measures to help out any concept, like using flex wheels
- **Concept Scoring:**
    - ***Overtop Intake: 5/5*** → The bottom roller can be made as wide as we want, and since contact is on the top of the block, we will maintain intaking capabilities all the way across the intake.
    - ***Claw: 1/5*** → Can only hold 1 block at once for a secure grip.
    - ***Side Rollers: 1/5*** → Since it requires contact on both sides of the block, only 1 block can be taken in at a time
    - ***Conveyor: 4/5*** → The conveyor flaps may create more complication/dead-spots, but the bottom pre-roller can be made to take in as many blocks as desired.

Decision Matrix:

	Weighting	Over Top Intake		Claw		Side Intake Rollers		Conveyor	
Speed	0.27	5	1.35	2	0.54	3	0.81	5	1.35
Build Complexity	0.24	5	1.20	3	0.72	3	0.72	4	0.96
Capacity	0.25	5	1.25	1	0.25	1	0.25	4	1.00
Grip	0.23	3	0.69	5	1.15	4	0.92	4	0.92
Totals:	0.99		**4.49**		**2.66**		**2.70**		**4.23**

Conclusion:

We will move forward with the over top intake concept. This concept provides the best compromise between the parameters that we are interested in, and has an overall score average of 4.52/5, meaning it should solidly satisfy what we want out of our intake.

Next Steps:

Our next step in our block system is to brainstorm and select a method of holding blocks.

8/18/2025 - Intake Brainstorming: Block Holder

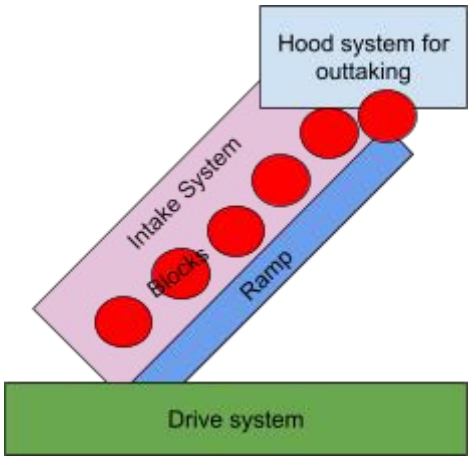
Meeting Goal: What options are available to us for picking up blocks?

Participants: Austin, Jenea

Block Holder Types:

Straight Ramp

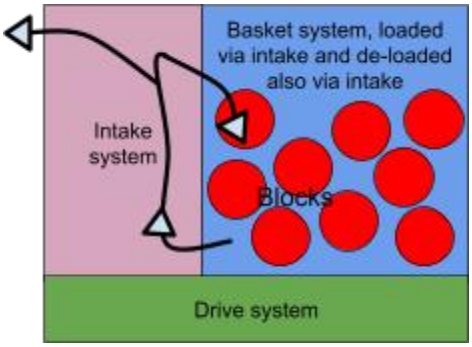
This is a flat inclined surface that guided the objects towards another mechanism. The materical options are flex wheels or rubber bands.



Pros: Fast load, simple, as wide as needed, higher load stability.
Cons: Jamming, axel bends, entanglement, compression needed.

Basket

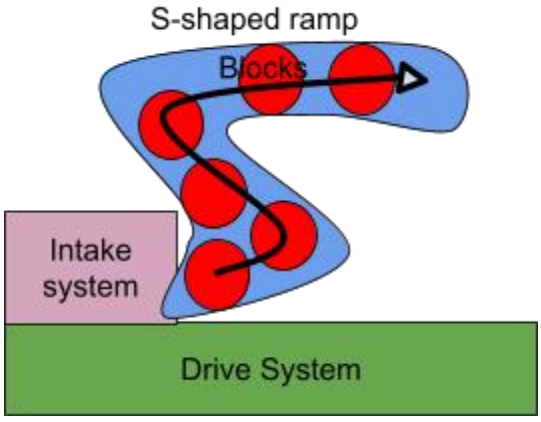
This contains multiple game objects before scoring. It doesnt allow for continuous cycling which makes it a slower option. But it allows for more control over the objects.



Pros: easy pick up, simple, precision
Cons: slow load, inefficient, big, alignment

S-Ramp

This is similar to the first option, it is a curved ramp the smoothly redirect the objects and reduced bounce and jamming. This makes for a good bottom stage but requires a different top stage for proper lifting.



Pros: Reduced stalling, less motor stress, speed, alignment.
Cons: stress on structure, sizing, less capture area, lack of multi object capability.

8/19/2025 - Intake Selection: Block Holder

Meeting Goal: Of the brainstormed intake types above, what is the best to use?

Participants: Austin, David, Alex

Parameters:

Speed

- We want a fast intake to be able to pick blocks up quickly and score them quickly to score them before our opponents.
- **Weighting:** This is our highest weighted category at 0.28, because it affects our performance the most since we want to score as many blocks as possible to get as many points as possible.
- **Concept Scoring:**
  - ***Straight Ramp:*** 5/5 → The straight ramp received a 5 due to the limited amount of obstacles between the intake and outtake.
  - ***Basket:*** 2/5 → Basket takes awhile to reach full capacity and to outtake.
  - ***S-Ramp:*** 4/5 → Flows through the shape at a fast speed close to that of the straight ramp.

Complexity (Building)

- We want an intake that is easy to build and easy to tune because we are unfamiliar with how intakes will behave with the elements since this is the first one we are building this year.
- **Weighting:** This has the 2nd highest weighting at 0.25, because since this is our 1st robot we have plenty of time to figure out how to implement the concept.
- **Concept Scoring**
  - ***Straight Ramp:*** 5/5 → Straight forward build with simple set up.
  - ***Basket:*** 4/5 → Easy to set up, and is very boxy which makes it easy to build.
  - ***S-Ramp:*** 3/5 → A bit more complex compared to the others.

Capacity

- We want to be able to hold multiple blocks and take in multiple blocks at a time so that we minimize how much time we spend intaking and so that we minimize the precision required to pick up blocks.
- **Weighting:** This is our 4th highest weighting at 0.23, because it affects the direct performance of our intake the 4th most. As capacity doesn't mean anything if the robot isn't fast and effective.
- **Concept Scoring**
  - ***Straight Ramp:*** 4/5 → With the width being customizable it allows the capacity to double if needed.
  - ***Basket:*** 5/5 → Has the biggest capacity amongst the designs and can be well suited to the 2v2 part of the game.
  - ***S-Ramp:*** 3/5 → The S-ramp has a decent capacity but lacks the ability to widen it like the straight ramp.

Efficiency

- We want to efficiently move the game pieces through the robot with little effort required to eliminate potential jams and breaks.
- **Weighting**: This is our 3rd rated category as this is our first robot and we can sacrifice efficiency for understanding the build and the speed.
- **Concept Scoring**
  - ***Straight Ramp:*** 2/5 → The straight ramp is frequent to jamming and axle bending. These reasons are why the score is so low.
  - ***Basket:*** 3/5 → As the basket takes a lot of gamepieces and time to till to make capacity which tanks its score.
  - ***S-Ramp:*** 5/5 → The S-ramp is great in skills as it allows for a quick outtake of blocks through the S shape. The design is known for limited jams which led to its score being higher.

Decision Matrix:

	Weighting	Straight Ramp		Basket		S-Ramp	
Speed	0.28	5	1.40	2	0.56	4	1.12
Build Complexity	0.25	5	1.25	5	1.25	3	0.75
Capacity	0.23	4	0.92	5	1.15	3	0.69
Efficiency	0.24	2	0.48	3	0.72	5	1.25
Totals:	1.00		**4.05**		**3.68**		**3.81**

Conclusion:

We will use the straight ramp design as it is a simple design and would work really well with the 2v2 format. The speed and capacity allows for efficient scoring and defending during a match which aligns with our team's goal.

8/22/2025 - Matchloader Brainstorming

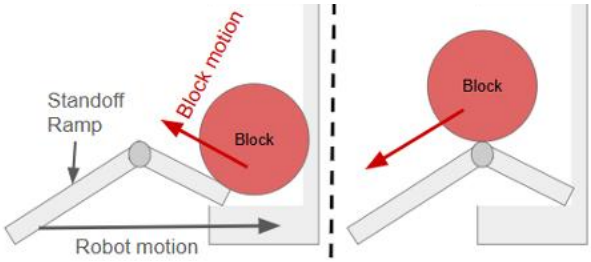
Meeting Goal: What options are available to us for getting blocks out of the tubes?

Participants: Austin, Jenea

Matchloader Types:

Standoff Ramps

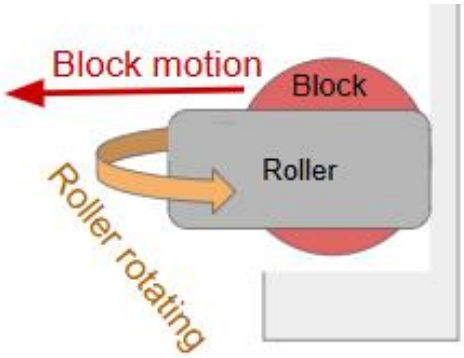
The ramp is supported by spacters to maintain a specific height and angle. The front standoffs are angled down and outward. This allows for the blocks to rise as the robot travels forward. When the block rach the end they come in contact with second set that is angles down and allows them to travel down into the take for the robot to collect.



Pros: fast, easy, adjustable
Cons: Rigid, gets stuck, unbalanced

Side Rollers

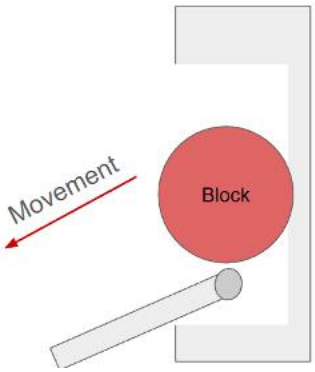
Additional rollers that stabilize and center the game objects while moving inside the robot. This consist of a set of power horizontal oriented roller that spin on the block. Some concerts are that it will not be able to block over the lip, and will require motors that atke away from the intakes abilities.



Pros: doubles as intake, easy, alinement
Cons: space, speed, catchability, requires motor

ZipTies

Plastic fatserd used for securing wires. This is similar to stand offs which uses a bar that enters the matchload tube. The bar is inserted into the tube and the blocks roll off the zip tie into the intake.



Pros: Easy, cheap, smooth, adjustable
Cons: flimsy, alignment,

8/25/2025 - Matchloader Selection

Meeting Goal: Of the brainstormed matchloader types above, what is the best to use?

Participants: Austin, David, Alex

Parameters:

Speed

- We want our matchloader to be able to dispense all of the blocks at once so we do not waste time. No repositioning should be required.
- **Weighting:** This is our highest weighted category at 0.28
- **Concept Scoring**
  - ***Standoff Ramps: 5/5*** →
  - ***Side Rollers: 4/5*** →
  - ***Zipties: 5/5*** →

Powering

- Since we intend to use 6 motors on the drive and 2 on the intake, we ideally want to be able to use only pneumatics to power this system.
- **Weighting**
- **Concept Scoring**
  - ***Standoff Ramps: 5/5*** →
  - ***Side Rollers: 1/5*** →
  - ***Zipties: 5/5*** →

Complexity

- The simpler a mechanism is, the more easily we will be able to implement and adjust it and the less time we will need to spend overall on it so we want to prioritize a simple mechanism.
- **Weighting**
- **Concept Scoring**
  - ***Standoff Ramps: 5/5*** →
  - ***Side Rollers: 2/5*** →
  - ***Zipties: 4/5*** →

Reliability

- We want our mechanism to be strong and reliable so it does not break in a match, since if it breaks we cannot access ~50% of our blocks for scoring.
- **Weighting**
- **Concept Scoring**
  - ***Standoff Ramps: 3/5*** →
  - ***Side Rollers: 5/5*** →
  - ***Zipties: 2/5*** →

Decision Matrix:

	Weighting	Standoff Ramps		Side Rollers		Zipties	
Speed	0.28	5	1.40	4	1.12	5	1.40
Powering	0.25	5	1.25	1	0.25	5	1.25
Complexity	0.23	5	1.15	2	0.46	4	0.92
Reliability	0.24	3	0.72	5	1.20	2	0.48
Totals:	1.00		**4.52**		**3.03**		**4.05**

Conclusion:

We will go with the **standoff ramps** because these had the highest composite average score, at a total of 4.52. This is ~0.5 higher than the next highest option (zipties) which we feel is enough to determine that this is the best option. Its lowest individual score is a 3 in reliability. This is a minor concern, but we will still move forward. We will just need to make sure that we test well.

8/29/2025 - Drive Design Plan

Meeting Goal: Create a CAD model of the drive concept.

Participants: Austin, David, Alex

Design Specifications:

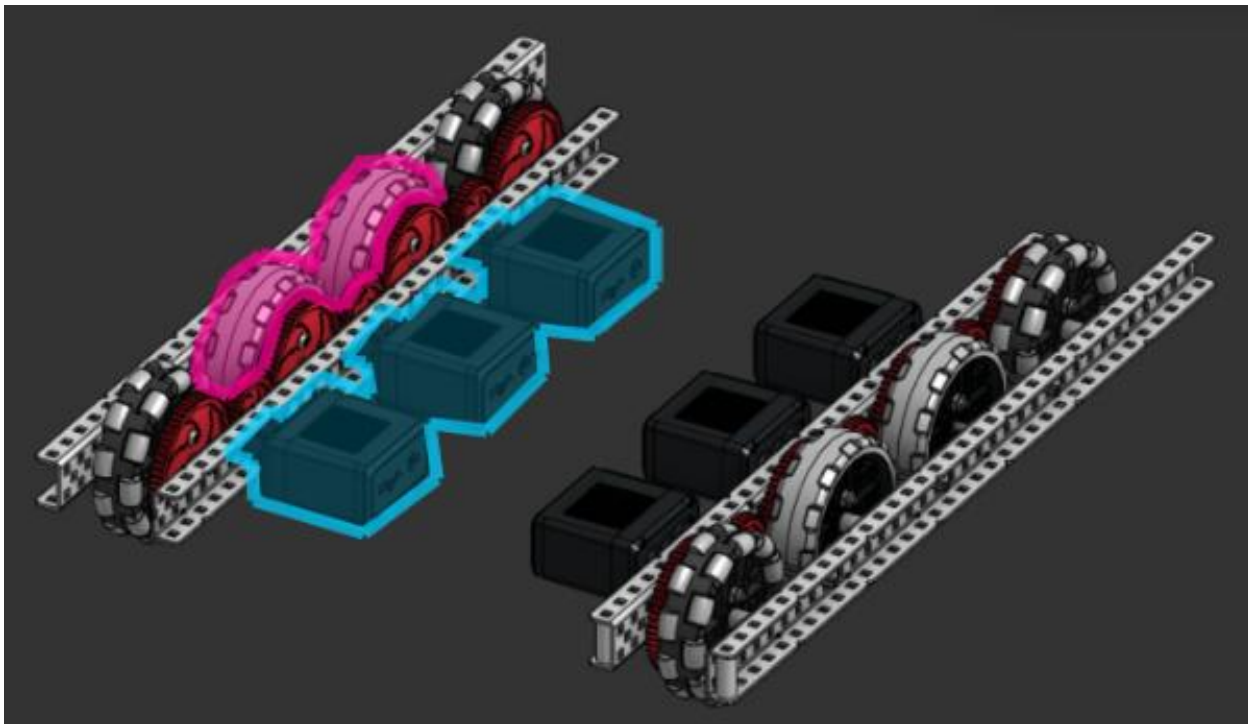
- **RPM:** 450
- **Wheel configuration:** 2 omni 2 traction
- **Wheel size:** 3.25 inch
- **Gear ratio:** 36:48
- **Motor configuration:** 6 11W, 3 per drive side

Pink:

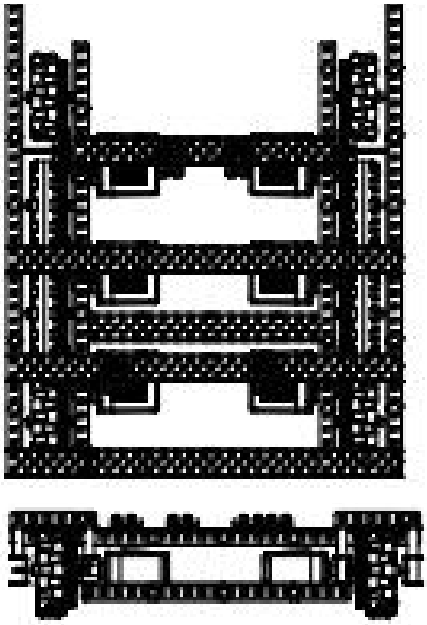
We chose to use double tractions as our middle wheels because this will help prevent us from sliding around when we are moving. This will make us harder to push and defend against.

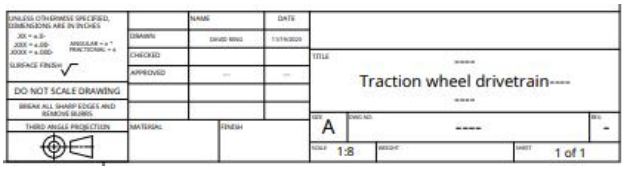
Blue:

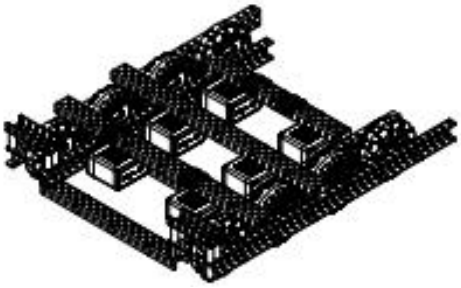
We chose to line all of our motors horizontally along the bottom of the drive to reduce our center of gravity to help prevent tipping.



Below, we included a drawing view of the drive that shows what our drive looks like. We have also included a bill of materials, as these are the parts we will need for building this model. We already have all of these parts, so we do not need to purchase any.









BOM type: Structured

Item	Name	Quantity
1	1 x 2 x 1 x 32 Aluminum C-Channel (276-2289)	2
2	#8-32 Nylock Nut (275-1027)	24
3	#8-32 x 2" Star Drive Screw (276-5004)	8
4	1/4" x 3/8" OD x #8 Nylon Spacer (276-6340)	8
5	3/8" x 3/8" OD x #8 Nylon Spacer (276-6340)	27
6	High Strength Shaft Adapter, #8 Round Bore, 1/2 Lon...	8
7	#8-32 x 1-1/4" Star Drive Screw (276-4997)	16
8	60T Gear (276-2169-003)	8
9	3.25" Anti-Static Wheel (260mm Travel) (276-7771)	4
10	3.25" Anti-Static Omni-Directional Wheel (260mm Tr...	4
11	#8-32 x 1/2" Star Drive Screw (276-5007)	12
12	Flat Bearing (276-1209)	6
13	V5 Smart Motor (6:1) (276-4840)	6
14	3.5" Standard Shaft (276-1149)	6
15	0.125" L x 0.25" OD Spacer	6
16	36T Gear (276-2169-002)	6
17	Shaft Collar (276-2010)	6
18	1" L x 0.375" OD Spacer	7
19	0.875" L x 0.375" OD Spacer	2
20	1 x 2 x 1 x 6 Aluminum C-Channel (276-2289)	2
21	1 x 2 x 1 x 35 Aluminum C-Channel (276-2289)	2
22	1/2" x 3/8" OD x #8 Nylon Spacer (276-6340)	4
23	1 x 2 x 1 x 19 Aluminum C-Channel (276-2289)	3
24	1 x 2 x 1 x 29 Aluminum C-Channel (276-2289)	2
25	Pillow Block Bearing (276-2016)	8
26	#8-32 x 1" Star Drive Screw (276-4996)	4

9/1/2025 - September Plans

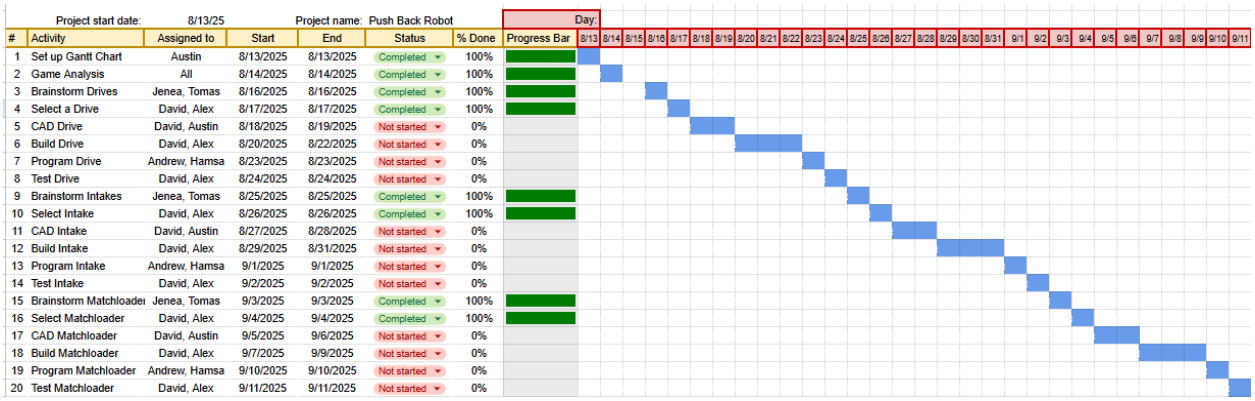
Meeting Goal: Figure out our goals for September.

Participants: Austin, David, Alex, Hamsa, Andrew, Jenea

The gantt chart has been updated with our progress. We are slightly behind schedule due to not being able to complete our initial brainstorming and selection as quickly as we originally thought. However, we do not believe it is significant enough to throw us off course for our first competition. **We still expect to be ready before 11/8/2025.**

Project start date: 8/13/25 Project name: Push Back Robot Day:

#	Activity	Assigned to	Start	End	Status	% Done	Progress Bar
1	Set up Gantt Chart	Austin	8/13/2025	8/13/2025	Completed	100%	
2	Game Analysis	All	8/14/2025	8/14/2025	Completed	100%	
3	Brainstorm Drives	Jenea, Tomas	8/16/2025	8/16/2025	Completed	100%	
4	Select a Drive	David, Alex	8/17/2025	8/17/2025	Completed	100%	
5	CAD Drive	David, Austin	8/18/2025	8/19/2025	Not started	0%	
6	Build Drive	David, Alex	8/20/2025	8/22/2025	Not started	0%	
7	Program Drive	Andrew, Hamsa	8/23/2025	8/23/2025	Not started	0%	
8	Test Drive	David, Alex	8/24/2025	8/24/2025	Not started	0%	
9	Brainstorm Intakes	Jenea, Tomas	8/25/2025	8/25/2025	Completed	100%	
10	Select Intake	David, Alex	8/26/2025	8/26/2025	Completed	100%	
11	CAD Intake	David, Austin	8/27/2025	8/28/2025	Not started	0%	
12	Build Intake	David, Alex	8/29/2025	8/31/2025	Not started	0%	
13	Program Intake	Andrew, Hamsa	9/1/2025	9/1/2025	Not started	0%	
14	Test Intake	David, Alex	9/2/2025	9/2/2025	Not started	0%	
15	Brainstorm Matchloader	Jenea, Tomas	9/3/2025	9/3/2025	Completed	100%	
16	Select Matchloader	David, Alex	9/4/2025	9/4/2025	Completed	100%	
17	CAD Matchloader	David, Austin	9/5/2025	9/6/2025	Not started	0%	
18	Build Matchloader	David, Alex	9/7/2025	9/9/2025	Not started	0%	
19	Program Matchloader	Andrew, Hamsa	9/10/2025	9/10/2025	Not started	0%	
20	Test Matchloader	David, Alex	9/11/2025	9/11/2025	Not started	0%	



We also completed some task out of order compared to what we originally intended, because due to the high interconnectivity of systems we wanted to pick all of our concepts before designing them so we can minimize adjustment time.

Team Decisions:

→ Omni wheel tank drive
→ Over top intake roller
→ Pivoting arm for matchloading

Team Build Accomplishments:

N/A.

Team Programming Accomplishments:

N/A.

Team Goals:

→ Get a fully constructed robot by end of the month
→ Meet 75% of our robot goals for the first version
→ Begin considering skills routes

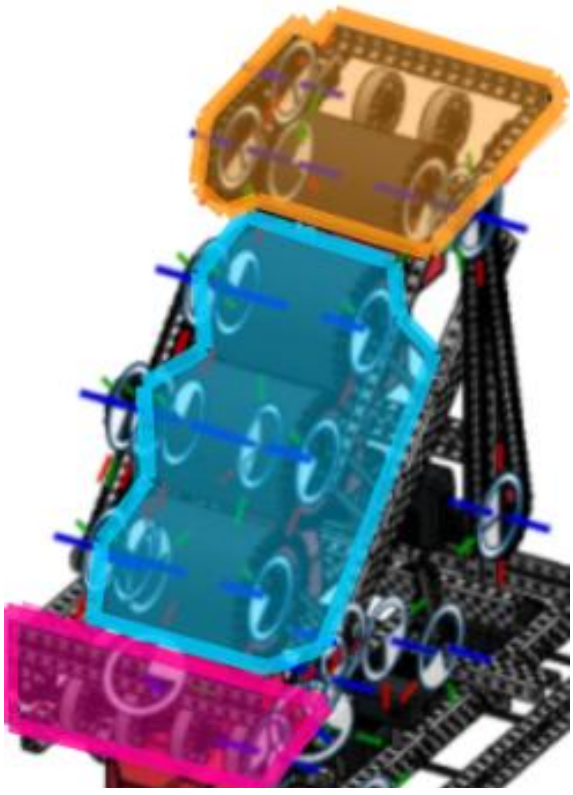
9/2/2025 - Intake/Block Holder Design Plan

Meeting Goal: Create a CAD model of the intake concept.

Participants: Austin, David, Alex

Design Specifications:

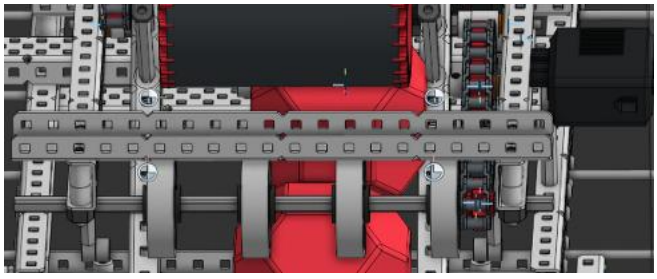
- **Pre-roller:**
  - **Number of flex wheels:** 4
  - **Flex wheel size:** 1.625 inch
  - **Gear ratio:** 600 direct
- **Elevator:**
  - **Number of band rollers:** 3
  - **Sprocket size:** 60T
  - **Gear ratio:** 600 direct
- **Hood:**
  - **Number of rollers:** 2
  - **Sprocket size:** 60T
  - **Flex wheel size:** 1.625 inch
- **Motor configuration:** 1 11w motor connected to middle rollers, 1 11W motor for hood
- **Rpm:** 600



Notes:

→ The elevator rollers have mesh around them to help grip the ball better and more consistently than just rubber bands

→ We used flex wheels on the bottom roller for the improved grip and width/gap variation, so we had to make our pre-roller stage a floating intake. This means that it pivots up slightly to allow the blocks to be taken in better. The motor is attached at the pivot point and chained back up to the roller so that we did not have to use a chain tensioner.



A bill of materials has been included below for parts estimate/plan when building.

BOM type: Structured

Item	Name	Quantity
› 1	drivetrain	1
2	3/8" x 3/8" OD x #8 Nylon Spacer (276-6340)	5
3	0.062" L x 0.375" OD Spacer	5
4	1/2" x 3/8" OD x #8 Nylon Spacer (276-6340)	25
5	1 x 2 x 1 x 30 Aluminum C-Channel (276-2289)	2
6	Aligner	2
7	1 x 2 x 1 x 20 Aluminum C-Channel (276-2289)	6
› 8	#8-32 Standoff with Shaft Collars	2
9	0.5" L x 0.375" OD Spacer	1
10	0.125" L x 0.375" OD Spacer	1
11	0.031" L x 0.375" OD Spacer	2
12	0.375OD x 1/64 L washer	1
13	Shaft Collar (276-2010)	9
14	3" Long #8-32 Standoff (276-2013)	1
15	Red Block	6
› 16	36T Sprocket Rollers	6
› 17	Sprocket Rollers	2
18	Part 1	1
19	6T High Strength Sprocket (276-3876)	2
20	18T High Strength Sprocket (276-3878)	3
41	1 x 1 x 14 Aluminum Angle (276-6484)	1
› 42	2" Flex Wheel with Adapters	2
43	mesh4	1
44	mesh2	1
45	mesh1	1
46	mesh3	1
47	V5 Smart Motor (5.5W) (276-4842)	2
48	V5 Smart Motor (6:1) (276-4840)	1
49	30 Link High Strength Chain (276-2172)	1
50	57 Link High Strength Chain (276-2172)	1
51	37 Link High Strength Chain (276-2172)	1
52	66 Link High Strength Chain (276-2172)	1
53	54 Link High Strength Chain (276-2172)	1
54	12T High Strength Sprocket (276-3877)	5
55	24T High Strength Sprocket (276-3879)	1
56	1/8" x 3/8" OD x #8 Nylon Spacer (276-6340)	1
57	V5 Robot Brain (276-4810)	1
58	1 x 1 x 13 Aluminum Angle (276-6484)	2
59	11.6" High Strength Shaft (276-7465)	1
60	276-9044-000 PF-015	2
61	276-9044-000 PF-015	2
62	276-9044-000 PF-015	2
63	276-9044-000 PF-015	2
64	276-9044-000 PF-015	2
65	276-9044-000 PF-015	2
66	276-9044-000 PF-015	2
67	276-9044-000 PF-015	2
68	276-9044-000 PF-015	2
69	276-9044-000 PF-015	2
70	276-9044-000 PF-015	2
71	276-9044-000 PF-015	2
72	276-9044-000 PF-015	2
73	276-9044-000 PF-015	2
74	276-9044-000 PF-015	2
75	Pillow Block Bearing (276-2016)	2
76	V5 Battery Clip (276-6020)	2
77	V5 Robot Battery (276-4811)	1
78	Part 1	1
› 21	2" Flex Wheel with Adapters	4
22	1 x 2 x 1 x 15 Aluminum C-Channel (276-2289)	3
23	High Strength Pillow Block Bearing (276-8383)	2
24	Flat Bearing (276-1209)	9
25	10" High Strength Shaft (276-7465)	1
26	High Strength Gear Shaft Insert (276-3881)	2
27	#8-32 x 1/2" Star Drive Screw (276-5007)	1
28	WASHER-TEFLON	6
29	#8-32 x 2" Star Drive Screw (276-5004)	1
30	#8-32 Keps Nut (275-1026)	3
31	1/2" x 1/2" OD x #8 Nylon Spacer (275-1066)	5
32	1/4" x 1/2" OD x #8 Nylon Spacer (275-1066)	5
33	#8-32 Nylock Nut (275-1027)	8
34	#8-32 x 1-1/2" Star Drive Screw (276-4998)	5

9/5/2025 - Matchloader Design Plan

Meeting Goal: Create a CAD model of the matchloader concept.

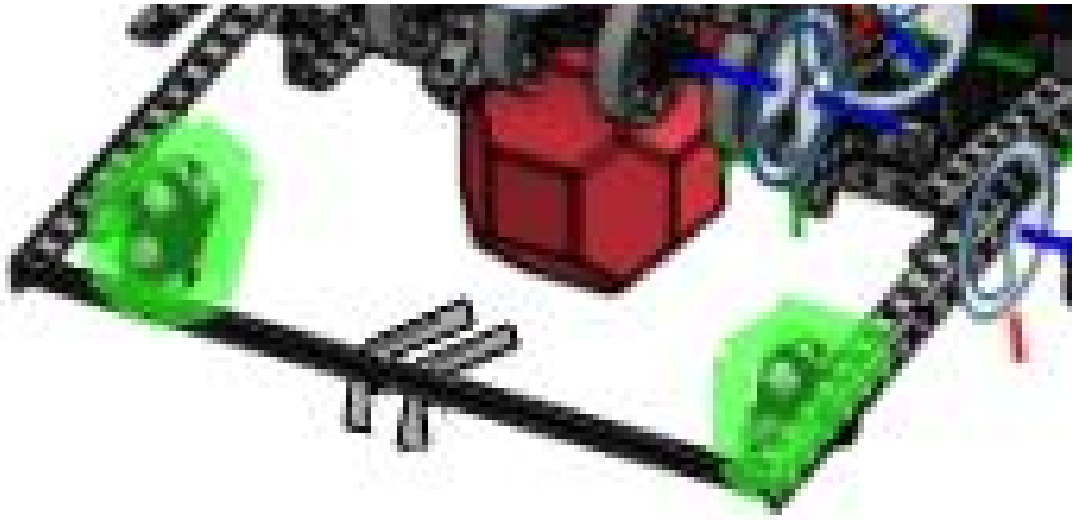
Participants: Austin, David, Alex

Design Specifications:

- **Powering:** Piston
- **Arm type:** 2-bar

Green:

We are using 2in omni wheels as the hard-stops for the mechanism. This will allow us to still move with it down because the wheels will roll instead of digging into the mat.



Bill of Materials:

35	#8-32 x 3/8" Star Drive Screw (276-4991)	2
36	Shaft Collar (276-2010)	1
37	1.5" Long #8-32 Standoff (276-2013)	1
38	1 x 2 x 1 x 25 Aluminum C-Channel (276-2289)	2
39	1 x 2 x 1 x 35 Aluminum C-Channel (276-2289)	4
40	1 x 1 x 10 Aluminum Angle (276-6484)	2

9/18/2025 - Robot Construction: Frame

Meeting Goal: Get a fully constructed robot ready for programming/testing.

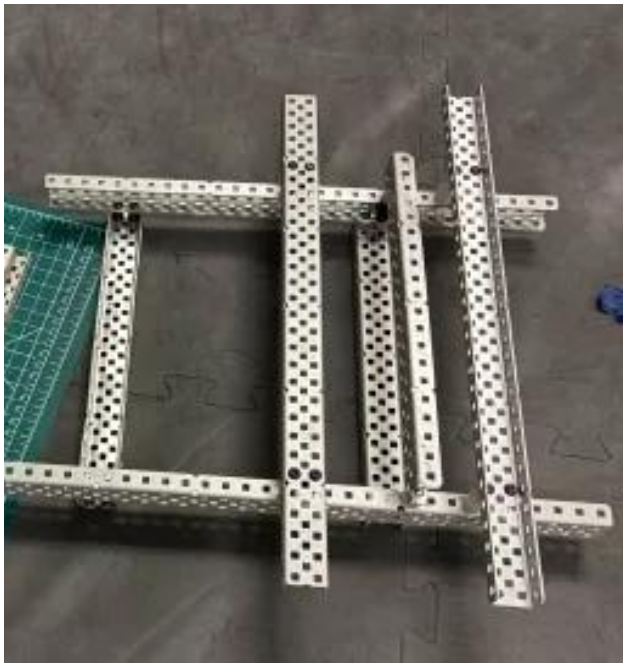
Participants: Austin, David, Alex

Step 1: Drive Frame

Our first step was to assemble the frame for the drive. There are 2 full cross braces and 3 inner cross braces to provide torsional resistance across the base. These will also be used as mounting points for the intake structuring. For now, we only attached 1 of each pod channel to have something to work with. This also gives us our intake placement for the front stage.

Parts Overview:

1. C-channel (various lengths, 7 total)
2. Assorted screws
3. Assorted spacers
4. Nylock nuts

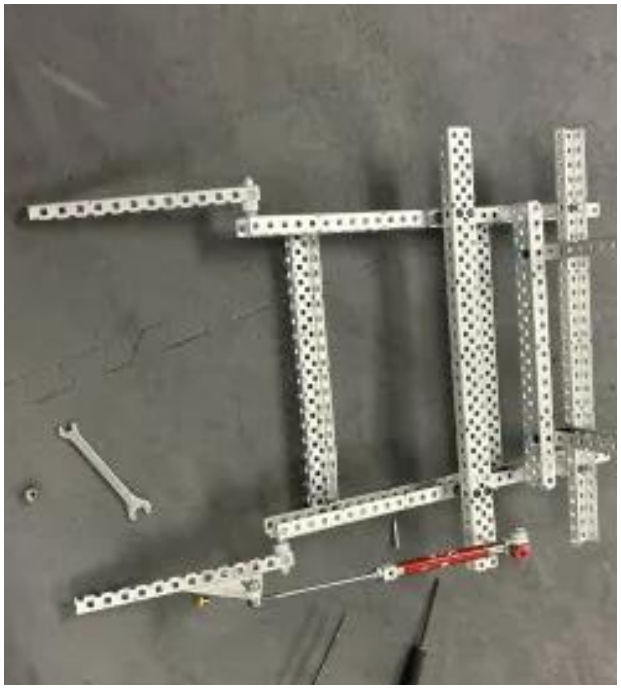


Step 2: Matchloader frame

With the drive frame together, we can attach the matchloader frame. The piston was mounted to one of the back full cross braces, and the 2-bar was mounted with the pivot above the very front of the drive. They are connected by a custom piece of plastic.

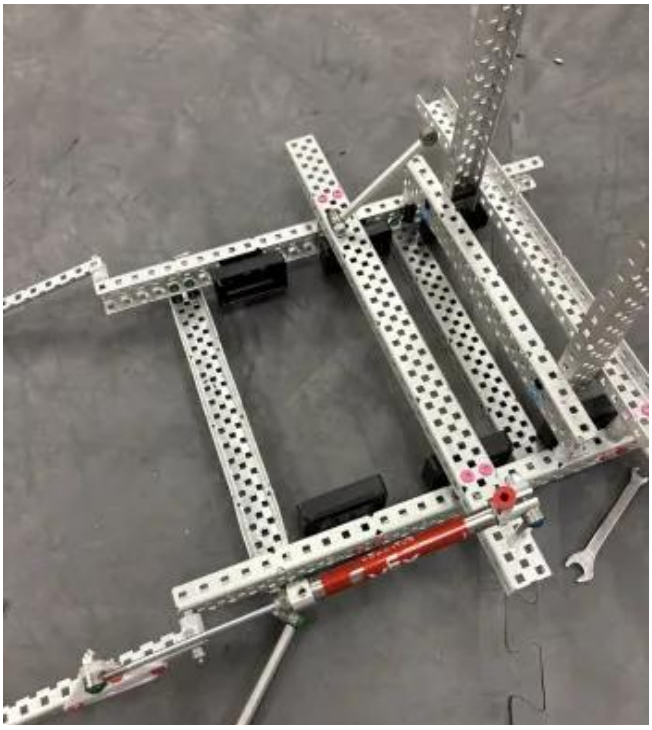
Parts Overview:

1. L-channel (2 total for matchloader)
2. Standoff/screws for piston mount
3. 3inch double-acting piston



Step 3: Motors/Towers

To finish off part of the drive, we added the motor caps. These are removed from the motors and screwed in independently to allow for hot swapping, which will let us remove our motors easily and replace them if they break or overheat during a match. They will be secured with rubber bands later on. In addition, we also added the vertical intake towers in this step. These were mounted on the back full crossbrace and go up to the height desired to make the angle/length for our elevator ramp.



Parts Overview:

1. Motor caps (6 total)
2. Assorted screws
3. Assorted spacers
4. Standoff bracing
5. 2 long c-channels

Step 4: Finish Intake Structuring

The next step was finishing the bottom structure base for our intake. This includes the angled c-channel that goes from the bottom of the drive to the top of the towers, as well as the structuring for the front stage. This was made using a U-frame, where the side lengths of the U are the connection/extension points for pivoting and the bottom length is where the roller will be connected.
It also has high strength bearings connected on the bottom side for a high strength axle that we will put the flex wheels on later.



Parts Overview:

1. Long c-channel (2 for intake ramp)
2. Medium c-channel (3 for preroller frame)
3. Assorted screws
4. Assorted spacers
5. Nylock nuts

9/18/2025 - Robot Construction: Drive

Meeting Goal: Get a fully constructed robot ready for programming/testing.

Participants: Austin, David, Alex

Step 1: Wheel Assemblies

With all of the primary structuring done, we can now start to add the working components. This drive uses two traction wheels and two omni wheels, all 3.25in, per side. They are assembled as shown in the picture to the right. Only 1 wheel assembly is shown, but we made 4 total traction assemblies and 4 total omni assemblies.





Parts Overview:

1. Screws
2. Green inserts
3. 48T HS gears (8 total)
4. Traction wheels (4 total)
5. Omni wheels (4 total)
6. Nylock nuts

Step 2: Gear Powering Assembly

To power the wheels we built above, we are using 36T gears connected to the motor shafts. We had to use the thin spacers so they would fit between the wheels without rubbing. We need 3 of these per side.



Parts Overview:

1. Axle
2. LS 36T gear
3. 3/4"x1/2" Spacers
4. 1/2"x1/2” Spacers
5. Shaft collar

Step 3: Motors/Pneumatic Tank Mounting

We also added the battery and pneumatic tank mounts. These are all slotted above the motors, between our cross-braces. We added them now because at any other point during building, they would be very hard to put on. Since our drive is finished, we also rubber banded the motors with blue carts into the motor caps.



Parts Overview:

1. Motors and crates
2. Pneumatic tanks
3. Outer drive rails
4. Assorted screws

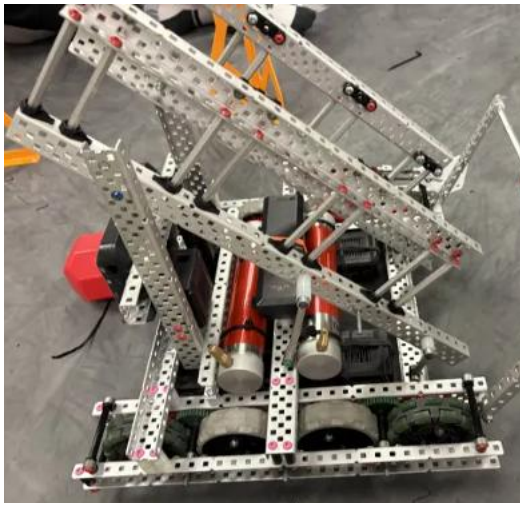
9/18/2025 - Robot Construction: Intake

Meeting Goal: Get a fully constructed robot ready for programming/testing.

Participants: Austin, David, Alex

Step 1: Adding Top Intake Rails

Since we need the rollers to be mounted above the ramp, we attached a set of parallel c-channels above the original ones via standoffs. These give us mounting positions for the rollers. We also went ahead and added the bearings to these rails, since we know where they will be positioned from our CAD model. We screwed them in for maximum security.

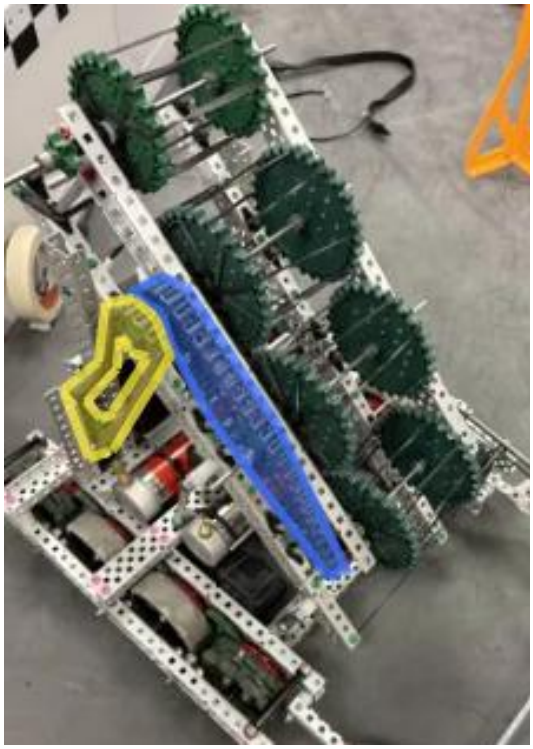


Parts Overview:

1. 2 Long C-Channels
2. 16 4” Standoff Supports
3. 16 Post Standoff Retainers
4. 8 Bearing Flats
5. Assorted Screws
6. Nylock Nuts

Step 2: Adding the Rollers/Chain

In this step, our goal was to finish the elevator stage of the intake entirely. To accomplish this, we put together the rollers in the top rails. We used 84T sprockets to have maximum flexing space in the bands for the blocks. They are spaced roughly 4 inches apart to have enough margin that we do not have to worry about precision/jamming with the 3.25inch blocks.



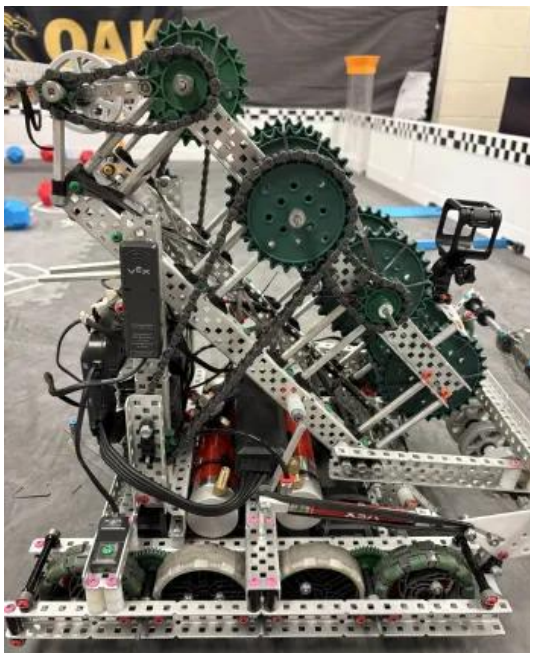
In addition to the rollers, we attached the motors and set up our chaining system. All of the rollers are chained together via sprockets on the left side (blue) and another chain goes from the top roller down to the motor (yellow). The 11W motor is positioned in the back of the robot so that it can be placed low down to reduce the height of our center of gravity.

Parts Overview:

1. 8 84T Sprockets
2. 6 18T Sprockets
3. Chains
4. 4 Axels
5. Lock Collars
6. 1/2" Spacers

7. 8 Bearing Flats
8. Assorted Screws
9. Nylock Nuts
10. Rubber Bands

Step 3: Adding the Hood/Front roller



The very last part of the robot to build is the hood, though we do still need to also add the front roller into the frame. The hood consists of two rollers: the first is a band roller that transitions from the intake, and the second is a flex-wheel roller that adds compression for pushing the blocks down/out into the long goals.

To stop the hood from flopping downwards into the bot, we added a hard stop by attaching a standoff to our elevator mechanism at the end using lock collars and screws. To avoid the hood getting stuck in an upwards position, we also attached a rubber band at the top corner of it and to the hard stop using screws, this way, the tension in the band will force the hood back down. We had to adjust its tension, however, since it would not lift up to outtake blocks.

Parts Overview:

1. 3 C-Channels cut in half
2. 6T Sprocket
3. 12p Chain
4. Flex Wheels
5. 2 2” Standoffs
6. 3/8" Spacers
7. Axel
8. Lock Collars
9. Assorted Screws
10. Rubber Band

9/18/2025 - Robot Construction: Matchloader

Meeting Goal: Get a fully constructed robot ready for programming/testing.

Participants: Austin, David, Alex

Step 1: Matchloader Ramp



To finish off our matchloader, we added in the ramp portion of it that will go into the match load tubes and help move the blocks into the intake mechanism. Our ramp consists of 2 standoffs that act as a beam near the front of the matchloader. This will push the blocks upwards once the bot moves into the tubes. The matchloader also has two standoffs angled back to guide the blocks in falling down into our intake. This was possible by using high strength shaft collars.

Parts Overview:

1. Two 2”, one 3”, and one 4” Standoffs
2. Assorted screws
3. 3 High strength shaft collars

9/20/2025 - V1 Driver Control Programming

Meeting Goal: Get a fully constructed robot ready for programming/testing.

Participants: Hamsa, Andrew

With the robot fully constructed, we are ready to move on to programming.
Step 1: Define all motors and pneumatics
Step 2: Set up main intake motor methodology and control
Step 3: Set up hood methodology and control

```
 void usercontrol(void) {
  // User control code here, inside the loop
  while (1) {

    if (Controller1.ButtonL1.pressing()) {
      lowintake.spin(forward, 100, percent);
      highintake.spin(forward, 100, percent);
    } else if (Controller1.ButtonL2.pressing()) {
      lowintake.spin(reverse, 100, percent);
      highintake.spin(reverse, 100, percent);
    } else {
      highintake.stop();
      lowintake.stop();
    }

    // hood
    if (Controller1.ButtonR1.pressing()) {
      hood.spin(forward, 100, percent);
    } else if (Controller1.ButtonR2.pressing()) {
      hood.spin(reverse, 100, percent);
    } else {
      hood.stop();
    }

    bool currentButtonState = Controller1.ButtonDown.pressing();

    if (currentButtonState && !previousButtonState) {
      pistonState = !pistonState;
      matchloader.set(pistonState);
    }

    previousButtonState = currentButtonState;
    // ..........................................................................
    chassis.control_arcade();
    wait(20, msec); // Sleep task for a short time to prevent wasted resources.

 }
}
```

9/22/2025 - Drive: Testing

Meeting Goal: Does our drive perform the way we want it to?

Participants: David, Alex, Austin

Test 1 Procedure: Speed

1. Line robot up with 72 inches of space in front
2. Start stopwatch and push joysticks forward
3. Stop stopwatch when robot hits 72 inch mark
4. Divide 72 inches by time in seconds and record
5. Repeat steps 1-4 5 times.

Diagram:

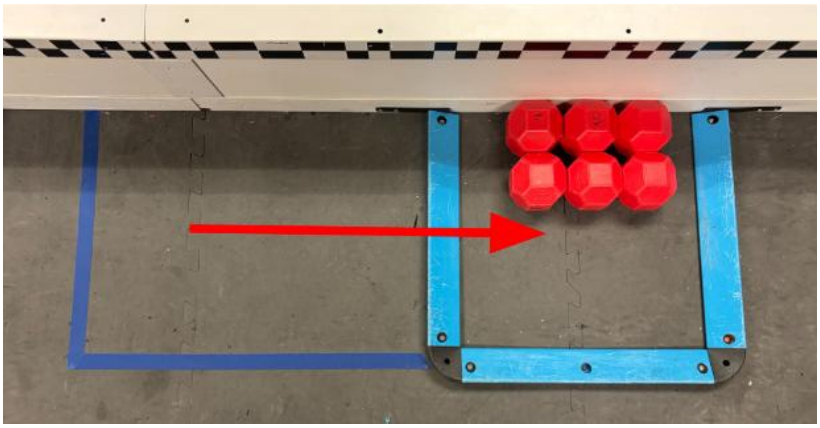


Trial 1	Trial 2	Trial 3	Trial 4	Trial 5	**Average**
69.902	80.900	73.470	77.419	72.727	**74.884 ms^-1**

Test 2 Procedure: Parking

1. Set our robot parallel with the parking zone
2. Start time and drive into the parking zone
3. Stop time when all blocks are cleared and the robot is parked.
4. Repeat 1-4 5 times

Diagram:



Trial 1	Trial 2	Trial 3	Trial 4	Trial 5	**Average**
3	5	4	3	4	**3.8 sec**

Conclusion:

Our drive reached all goals we previously outlined so it does not require reiteration. After our first competition, we will evaluate the performance of our drive further and determine if we need to adjust our parameters, goals, or drive in any way.

9/24/2025 - Intake: Testing

Meeting Goal: Does our intake perform the way we want it to?

Participants: David, Alex, Austin

Test 1 Procedure: Determine max capacity

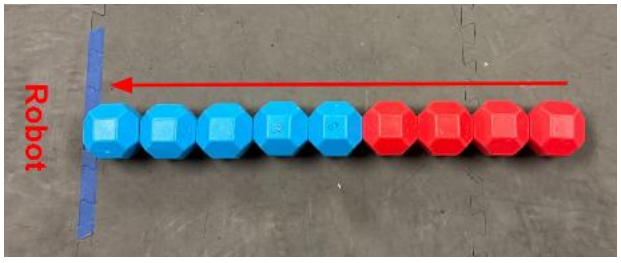
1. Start intake
2. Feed block at random into intake
3. Repeat step 2 until intake jams/can't hold any more blocks. Record value

Trial 1	Trial 2	Trial 3	Trial 4	Trial 5	**Max**
8	8	7	9	6	**9 blocks**

Test 2 Procedure:Intake to max capacity

1. Set up a series of 9 blocks in a line
2. Begin timer
3. Push the line of 9 blocks into the robot
4. Stop timer once robot has fully picked up all of the blocks

Diagram:



Trial 1	Trial 2	Trial 3	Trial 4	Trial 5	**Average**
4.29 sec	3.56 sec	3.91 sec	3.94 sec	6.32 sec	**4.04 sec**

Conclusion:

Our intake can hold 9 blocks at a time. This is higher than our goal of 6-7, so we are satisfied with it. In addition, our intaking speed is on average 4.04 seconds, which is 0.45 seconds per block. However, we noticed with the rollers that our intake stalls often before it is filled. Thus, we are going to reiterate on this system to try to fix it. This will be done in V2.

9/27/2025 - Matchloader: Testing

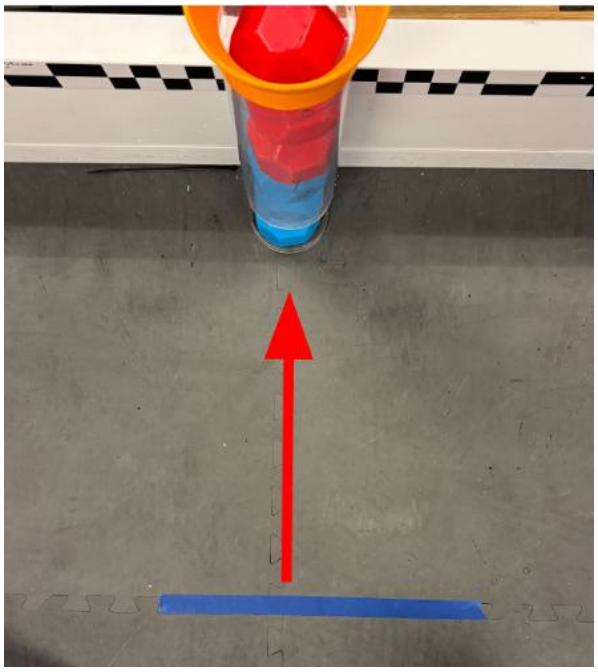
Meeting Goal: Does our matchloader perform the way we want it to?

Participants: David, Alex, Austin

Test 1 Procedure: Clearing the tube

1. Fill up matchload tube
2. Line up robot one tile from tube
3. Lower match loader
4. Begin timer
5. Drive into the tube
6. Stop timer once tube is cleared

Diagram:



Trial 1	Trial 2	Trial 3	Trial 4	Trial 5	**Average**
4.4	3.5	DNF	3.7	DNF	-

Conclusion:

We are going to have to reiterated on our matchloader because it is not consistent in getting the blocks out. We had two runs where the matchloader failed to get the blocks out. This is because the blocks jammed on the standoffs, between the top of the tube and the standoffs. In the next robot iteration, we will need to fix this.

10/1/2025 - October Plans

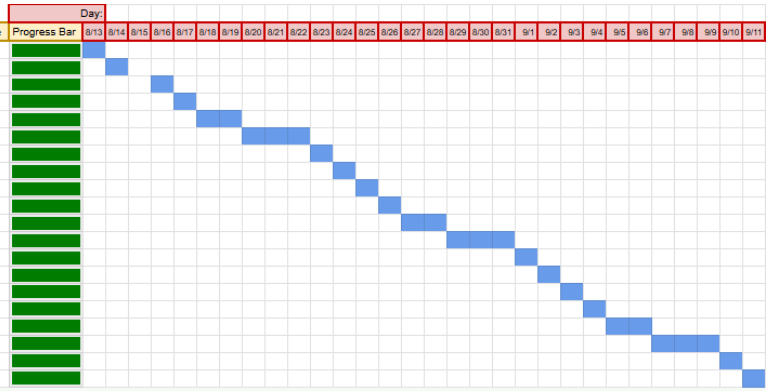
Meeting Goal: Figure out our goals for October

Participants: Austin, David, Alex, Hamsa, Andrew, Jenea

We completed the V1 robot, so our current gantt chart has all task completed. We will need to make another gantt chart for the V2 robot once we have identified what needs to be done.

Project start date: 8/13/25 Project name: Push Back Robot

#	Activity	Assigned to	Start	End	Status	% Done
1	Set up Gantt Chart	Austin	8/13/2025	8/13/2025	Completed	100%
2	Game Analysis	All	8/14/2025	8/14/2025	Completed	100%
3	Brainstorm Drives	Jenea, Tomas	8/16/2025	8/16/2025	Completed	100%
4	Select a Drive	David, Alex	8/17/2025	8/17/2025	Completed	100%
5	CAD Drive	David, Austin	8/18/2025	8/19/2025	Completed	100%
6	Build Drive	David, Alex	8/20/2025	8/22/2025	Completed	100%
7	Program Drive	Andrew, Hamsa	8/23/2025	8/23/2025	Completed	100%
8	Test Drive	David, Alex	8/24/2025	8/24/2025	Completed	100%
9	Brainstorm Intakes	Jenea, Tomas	8/25/2025	8/25/2025	Completed	100%
10	Select Intake	David, Alex	8/26/2025	8/26/2025	Completed	100%
11	CAD Intake	David, Austin	8/27/2025	8/28/2025	Completed	100%
12	Build Intake	David, Alex	8/29/2025	8/31/2025	Completed	100%
13	Program Intake	Andrew, Hamsa	9/1/2025	9/1/2025	Completed	100%
14	Test Intake	David, Alex	9/2/2025	9/2/2025	Completed	100%
15	Brainstorm Matchloader	Jenea, Tomas	9/3/2025	9/3/2025	Completed	100%
16	Select Matchloader	David, Alex	9/4/2025	9/4/2025	Completed	100%
17	CAD Matchloader	David, Austin	9/5/2025	9/6/2025	Completed	100%
18	Build Matchloader	David, Alex	9/7/2025	9/9/2025	Completed	100%
19	Program Matchloader	Andrew, Hamsa	9/10/2025	9/10/2025	Completed	100%
20	Test Matchloader	David, Alex	9/11/2025	9/11/2025	Completed	100%



We completed this gantt chart 16 days behind schedule. Part of this was due to unfamiliarity with what was required for us. Since we now have more experience, we believe we will be able to keep more on track for the next gantt chart.

Team Decisions:

→ Our drive is satisfactory

→ Our other systems are not, so we will move to a V2 robot

Team Build Accomplishments:

→ We successfully built a functional drive

→ We built an intake and became more familiar with how it will interact with blocks

→ We built a matchloader and got blocks out of the tubes, even though it had some issues

Team Programming Accomplishments:

→ We set up the configuration (ports/devices) for all of our systems

→ We got all of our systems running and able to be controlled in autonomous as well as by our controller in driver control

Team Goals:

→ Get a fully constructed V2 robot by end of the month

→ Meet rest of our goals for the robot in the next version: V2

→ Begin considering skills routes (did not accomplish last month)

10/2/2025 - V2 Design Cycle: Identify Requirements

Meeting Goal: What are the areas of focus for the 2nd robot?

Participants: Austin, David, Alex, Hamsa, Andrew, Jenea

Drive:

Functions:

- ☑ ~~Move around the field~~
  - ☑ ~~Fast (target: 450 RPM)~~
  - ☑ ~~Doesn't get pushed around~~
  - ☑ ~~Good acceleration~~
  - ☑ ~~Moves in multiple directions (target: forwards/backwards/turns)~~
- ☑ ~~Parks~~
  - ☑ ~~Single park~~
  - ☑ ~~Parks within 5s from anywhere on the field~~

We are satisfied with the drive and will not be changing it.

Intake:

Functions:

- ☑ ~~Picks up/scores blocks~~
  - ☐ Fast (target: 1200 RPM)
  - ☑ ~~Intakes 2 blocks at a time~~
  - ☐ Holds 7 blocks

We are not satisfied with our intake because it jams, which slows it down and reduces how many we can hold at once since we cannot intake more after. Our problems are with the components listed below.

Component Summary:

- Elevator (lifting blocks to given desired height)
  - Material (design choice in design plan)

Matchloader:

Functions:

- ☑ ~~Removes blocks from matchload tubes~~
  - ☐ Dispenses all 6 at once
  - ☑ ~~Feeds directly into intake~~

We are not satisfied with our matchloader because it does not always remove all of the blocks. We have identified the component below to be the problem with the system.

Component Summary:

- Ramp

10/5/2025 - V2 Intake Brainstorming

Meeting Goal: What options are available to us for fixing our intake problems?

Participants: Austin, David, Jenea

Switch to Conveyor Intake

This is a design choice where a roller intake transition into a conveyer belt for continuous object transportation. It is the simplest design because it changes the material of the elevator. It does not require a design change and does not interfere with other systems.

Pros: Simple, adjustable, cheap
Cons: Jamming, axel bends, entanglement, compression needed.

Add a Motor

This option could cause a lot of problems in implementation because we are already at our 88W limit. We would need to either chain the hood and elevator together and add another limiter mechanism, or take motors off the drive.

Pros: Adds more power, reduces intake overheat risk
Cons: Complicated, heavy, requires additional system, increases drive overheat risk

Increase Space Between Rollers and Ramp

A spacing adjustment to prevent jamming and allow smoother transferring. This is an easy change but doesn't effectively solve the issue. It also could prevent the rollers from getting adequate contact with the blocks. Although this would solve the jamming issue it would be slow and also might not be able to fully outtake and score.

Pros: Reduced stalling, less motor stress, speed, alignment.
Cons: stress on structure, sizing

10/10/2025 - V2 Intake Selection

Meeting Goal: Of the brainstormed intake types above, what is the best to use?

Participants: David, Alex, Austin

Parameters:

Efficiency

- It's important that the solution actually solves the problem, so we want to take into account how effective the solution will be.
- **Weighting**
- **Concept Scoring**
    - ***Conveyor: 4/5***
    - ***Add Motor: 5/5***
    - ***Increase Ramp Space: 3/5***

Impact

- Since we are short on time, we do not want a long and complicated solution. A simple one is preferable to take less time and affect fewer other robot systems.
- **Weighting**
- **Concept Scoring**
    - ***Conveyor: 5/5***
    - ***Add Motor: 2/5***
    - ***Increase Ramp Space: 5/5***

Reliability

- We do not want to impact the reliability of our robot. We also want a solution that is not going to break or require other additional solutions.
- **Weighting**
- **Concept Scoring**
    - ***Conveyor: 4/5***
    - ***Add Motor: 3/5***
    - ***Increase Ramp Space: 4/5***

Adjustability

- Since we will have to tune this system, we want a system with a high degree of adjustability, meaning that we can easily change things around to make it work if it still doesn't work after testing.
- **Weighting**
- **Concept Scoring**
    - ***Conveyor: 4/5***
    - ***Add Motor: 4/5***
    - ***Increase Ramp Space: 4/5***

Decision Matrix:

	Weighting	Conveyor		Add motor		Increase ramp space	
Efficiency	0.27	4	1.08	5	1.35	3	0.81
Impact	0.25	5	1.25	2	0.50	5	1.25
Reliability	0.24	4	0.96	3	0.72	4	0.96
Adjustability	0.24	4	0.96	4	0.96	4	0.96
Totals:	1.00		**4.25**		**3.53**		**3.98**

Conclusion:

We will move forward with changing to a conveyor roller. This is the simplest and most time efficient solution while being the most likely to work. We will move forward with implementing this into our design. We will not have to redesign our drive, since only the intake elevator material used is being changed. We also should not have to majorly change our hood or lower stage.

10/13/2025 - Matchloader Brainstorming

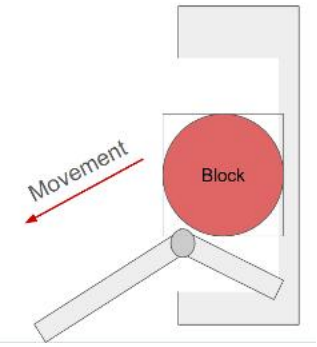
Meeting Goal: What options are available to us for getting blocks out of the tubes?

Participants: Austin, Jenea

Matchloader Types:

Folded Plastic

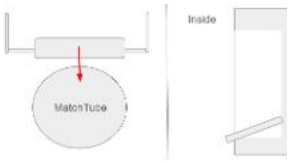
A plastic sheet bent to create lightweight structural shapes. This consist of a high strength shaft orr stand of to cross the between the gap and piece of plastic. The plastic is folded and stationary to make it over the lip so the game pieces rolls off.



Pros: speed, effective
Cons: inconsistency, uses limited poly, alinement, complex

Pivoting Plastic

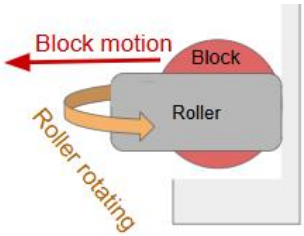
A plastic component that is minted on an axle to move out of way to redirect objects. This consist of a stand off to cross the between the gap and plastic on two high strength pillo blocks. While a rubber band is positioned at the top left corner of the plastic to allow a proper angle and then reset the the matchloader.



Pros: alinement, simple, speed unmatched, consistent
Cons: Uses limited poly, rubberband

Side Rollers

This is additional stablization which consist of a set of power horizontal oriented roller that spin on the block. Some concerts are that it will not be able to block over the lip, and will require motors that atke away from the intakes abilities.



Pros: Doubles as intake, simple, speed(both)
Cons: alinement, speed(both), jamming

10/15/2025 - Matchloader Selection

Meeting Goal: Of the brainstorming matchloader types above, what is the best to use?

Participants: Austin, David, Alex

Parameters:

Efficiency

- It's important that the solution actually solves the problem, so we want to take into account how effective the solution will be.
- **Weighting**
- **Concept Scoring**
  - ***Folded Plastic: 4/5***
  - ***Pivoting Ramp: 5/5***
  - ***Side Rollers: 3/5***

Impact

- Since we are short on time, we do not want a long and complicated solution. A simple one is preferable to take less time and affect fewer other robot systems.
- **Weighting**
- **Concept Scoring**
  - ***Folded Plastic: 5/5***
  - ***Pivoting Ramp: 5/5***
  - ***Side Rollers: 2/5***

Reliability

- We do not want to impact the reliability of our robot. We also want a solution that is not going to break or require other additional solutions.
- **Weighting**
- **Concept Scoring**
  - ***Folded Plastic: 4/5***
  - ***Pivoting Ramp: 4/5***
  - ***Side Rollers: 4/5***

Adjustability

- Since we will have to tune this system, we want a system with a high degree of adjustability, meaning that we can easily change things around to make it work if it still doesn't work after testing.
- **Weighting**
- **Concept Scoring**
  - ***Folded Plastic: 3/5***
  - ***Pivoting Ramp: 4/5***
  - ***Side Rollers: 3/5***

Decision Matrix:

	Weighting	Folded Plastic		Pivoting Ramp		Side Rollers	
Efficiency	0.27	4	1.08	5	1.35	3	0.81
Impact	0.25	5	1.25	5	1.25	2	0.50
Reliability	0.24	4	0.96	4	0.96	4	0.96
Adjustability	0.24	3	0.72	4	0.96	3	0.72
Totals:	1.00		**4.01**		**4.52**		**2.99**

Conclusion:

We ended up going with the pivoting plastic method due to its simplicity and speed. Though it would use one of our limited pieces of poly it would offer an easy unload of the match tube at unmatched speeds. The only downside other than the use of poly is it is reliant on a rubber band so if it comes loose mid match it could cause some problems but David is working to eliminate that.

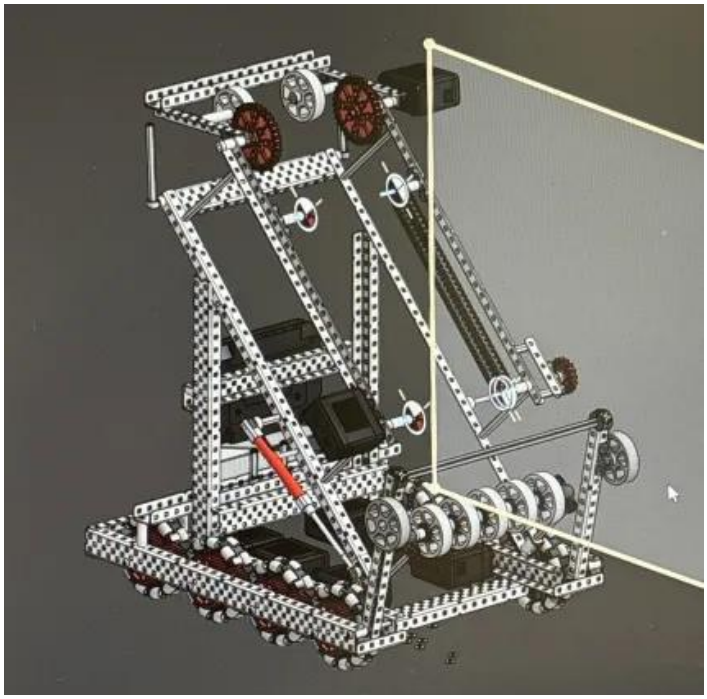
10/17/2025 - Intake Design Plan

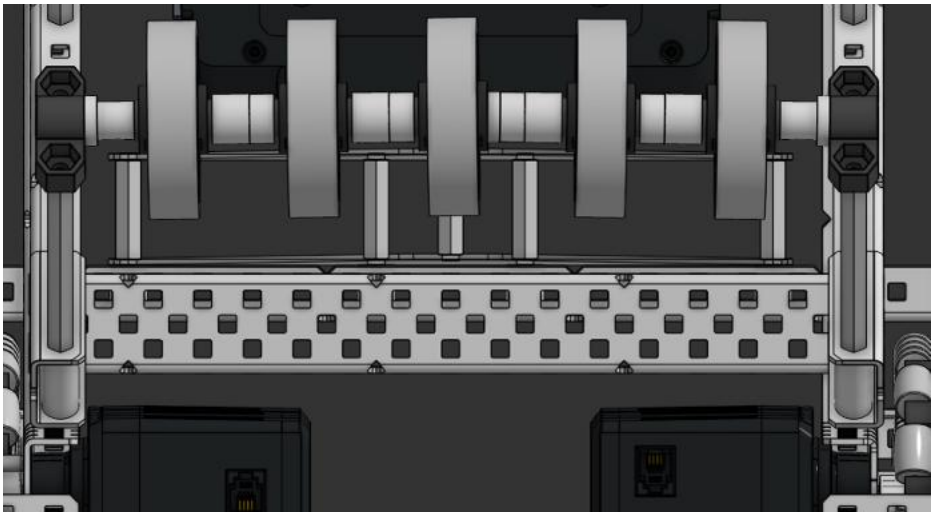
Meeting Goal: What is our intake going to look like?

Participants: Austin, David, Alex

Design Specifications:

- Number of flex wheels: 5
- Flex wheel size: 2 inch
- Gear ratio: 2:1
- Motor configuration: 1 11w motor, connected to middle rollers
- Rpm: 900





Item	Name	Quantity
21	#8-32 Nylock Nut (275-1027)	16
22	High Strength Shaft Adapter, #8 Round Bore, 1/2 L...	16
23	#8-32 x 1-1/4" Star Drive Screw (276-4997)	16
24	Flat Bearing (276-1209)	14
25	36T Gear (276-2169-002)	6
26	0.031" L x 0.375" OD Spacer	8
27	1 x 2 x 1 x 25 Aluminum C-Channel (276-2289)	1
28	Aligner	2
29	1" Long #8-32 Standoff (276-2013)	8
30	V5 Smart Motor (6:1) (276-4840)	8
31	3/8" x 3/8" OD x #8 Nylon Spacer (276-6340)	6
32	1 x 1 x 5 Aluminum Angle (276-6484)	7
33	1/8" x 3/8" OD x #8 Nylon Spacer (276-6340)	4
34	2" 30A Straight Flex Wheel (217-6353)	4
35	1.5" L x 0.375" OD Spacer	2
36	6T High Strength Sprocket (276-3876)	4
37	High Strength Pillow Block Bearing (276-8383)	2
› 38	2" Flex Wheel with Adapters	3
39	1/8" x 1/2" OD x #8 Nylon Spacer (275-1066)	6
40	3/8" x 1/2" OD x #8 Nylon Spacer (275-1066)	4
41	1/2" VersaHex Adapters (1/4" Square Bore, 1/4" Lo...	4
42	High Strength Gear Shaft Insert (276-3881)	4
43	16T Sprocket, 6P (276-8328)	1
44	80 Link 6P Chain (228-4983)	1
45	32T Sprocket, 6P (276-8330)	2
46	1 x 1 x 14 Aluminum Angle (276-6484)	2
47	High Strength Clamping Shaft Collar (276-6101)	2
48	0.5" Long #8-32 Standoff (276-2013)	4
49	1 x 1 x 13 Aluminum Angle (276-6484)	2
50	Pillow Block Bearing (276-2016)	3
51	#8-32 Keps Nut (275-1026)	2
52	2" 45A Straight Flex Wheel (217-6353)	2
53	11" High Strength Shaft (276-7465)	1
54	0.625" L x 0.375" OD Spacer	2
› 55	50mm Stroke Pneumatic Cylinder (276-8644) (1)	1
56	V5 Robot Brain (276-4810)	1
57	2.5" Long #8-32 Standoff (276-2013)	2

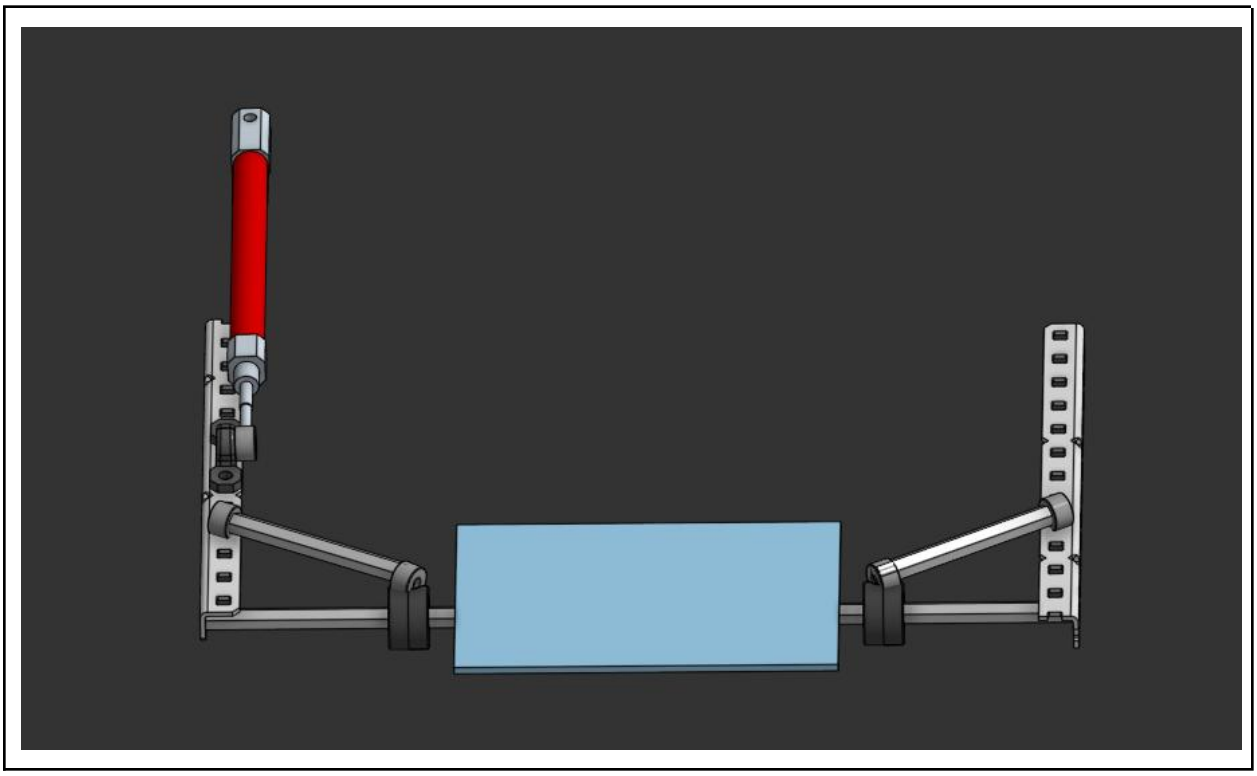
10/20/2025 - Matchloader Design Plan

Meeting Goal: Create a CAD model of the matchloader concept.

Participants: Austin, David, Alex

Design Specifications:

Be able to effectively get the game pieces out of the tube. Restrictions: has to lift up to fix in size regulation and uses part of our limited poly. We intend to use this poly to adjust to the match tube to effectively get the game pieces out.



Additional Information:

We plan to add red to our cat ear and to our controller. When the red cat ear is triggered, the matchloader flips down to load up the blocks. This allows for our co-driver to quickly call out and give commands to our driver. We use the phrase “red right” and “red left” to specifically give instructions to which matchload the driver should go to.

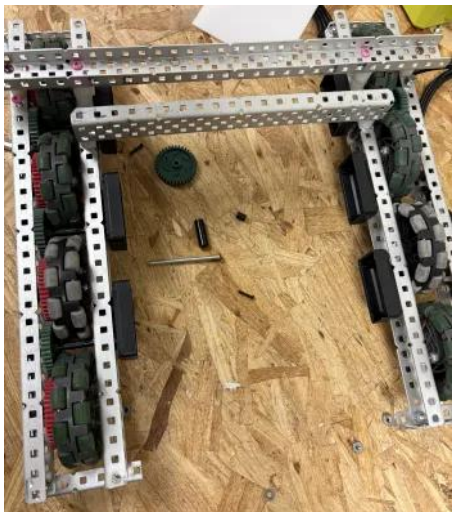
Item	Name	Quantity
1	6" Long #8-32 Standoff (276-2013)	1
2	5" Long #8-32 Standoff (276-2013)	1
3	1 x 1 x 12 Aluminum Angle (276-6484)	2
4	1" Long #8-32 Standoff (276-2013)	5
5	Shaft Collar (276-2010)	10
6	High Strength Clamping Shaft Collar (276-6101)	2
7	2.5" Long #8-32 Standoff (276-2013)	2
8	Pillow Block Bearing (276-2016)	1
9	75mm Stroke Pneumatic Cylinder Body (276-8646)	1
10	75mm Stroke Pneumatic Cylinder Rod (276-8646)	1
11	Shaft Collar (276-2010)	5
12	High Strength Pillow Block Bearing (276-8383)	2
13	Part 1	1

10/29/2025 - V2 Robot Construction

Meeting Goal: How will we construct our robot?

Participants: Austin, David, Alex

Step 1: Assemble Frame and Drive



Our first step in building the robot is to build the drive. This one was built nearly identically to the old one; however, we decided to get rid of the traction wheels since we are doing odometry pods, and traction wheels make horizontal movement more difficult.

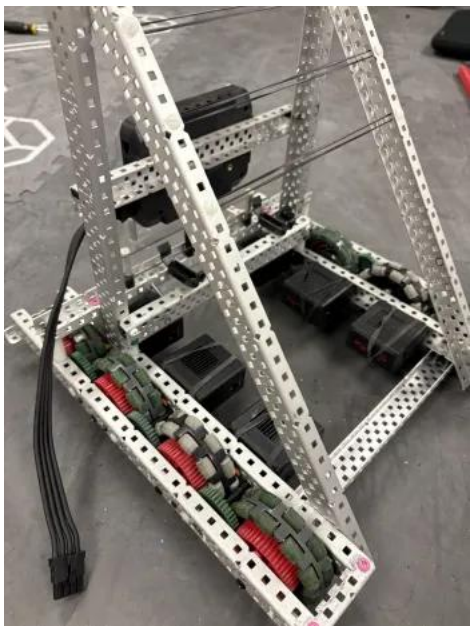
For our drive train, we used a 36:48 gear ratio in order to make our speed 450 RPM instead of the motor's 600 RPM.

Parts Overview:

1. 5 Long C-Channels
2. 1 medium C-Channel
3. 4 Small C-channels cut in half
4. 4 48T High Strength Gears
5. 3 36T Gears
6. Axels
7. Assorted Screws
8. Shaft Collars
9. Bearing Flats
10. 1/2" Diameter Spacers
11. 6 Motor Caps
12. 3.25" Omni Directional Wheels

Step 2: Intake Structuring

With the drive built, we moved on to starting the structuring for the intake ramp. Once again, we have an angled piece of c-channel connected to the drive, and a vertical set of towers in the back. On the brace across the back, we attached the brain as well, to keep our weight balanced and our center of gravity as low as possible.



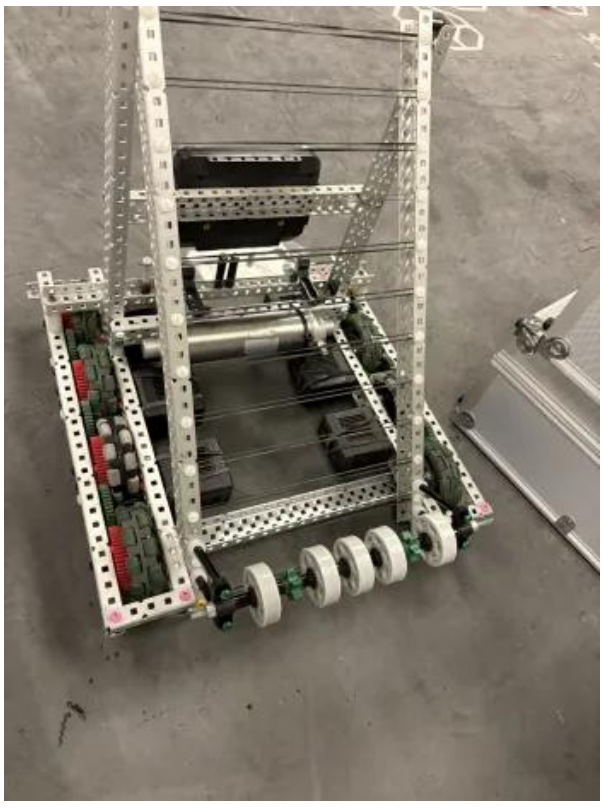
During this phase of construction, we also went ahead and attached polycarbonate pieces that were laser to serve as an aligner for when the bot is scoring the long goals.

Parts Overview:

1. 5 Long C-Channels
2. 1 Medium C-Channel
3. VEX Brain
4. 2 Battery Clips
5. Assorted Screws
6. 2 Polycarbonate Pieces
7. 1”x1/2” Spacers
8. 2 1” Standoffs

Step 3: Pre-roller/Ramp

Our pre-roller is still a pivoting intake, but we changed it to pivot vertically sliding up and down. Since it only moves ~1/4in, it doesn’t need a chain tensioner, and this method of assembly reduces a set of chains/sprockets required to power it, which reduces the overall system complexity and weight.



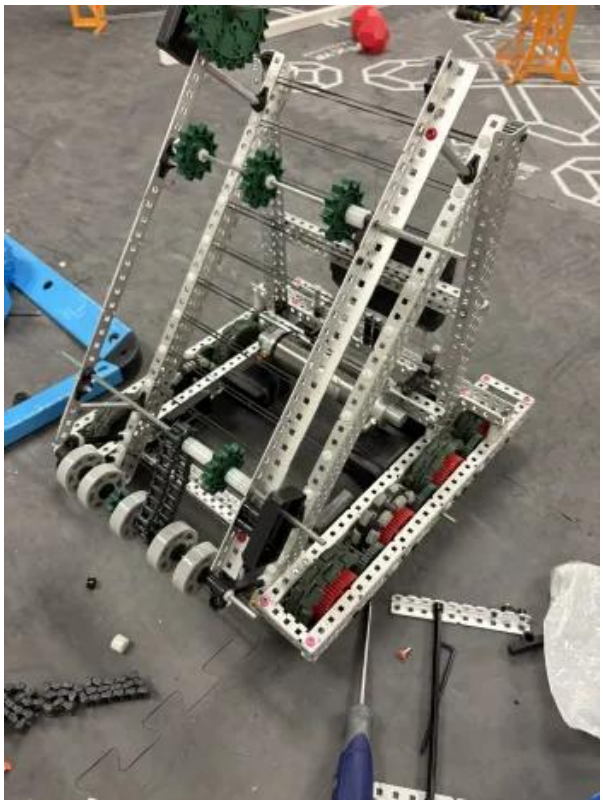
Our ramp consists of rubber bands with 1-1.5 inches of space between them, with the exception of where our middle goal scoring is.. This one has 2.5 inches of space. They are attached through the channel via nylon screws to save weight. We added in our pneumatics tank during this time using zip ties.

Parts Overview:

1. 1/2" Diameter Spacers
2. 1/4" Spacers
3. 2 6T Sprockets
4. 5 Flex Wheels

5. 2 Pillow Blocks
6. High Strength Axel
7. 4 2.5” Standoffs
8. Assorted Screws
9. 2 Shaft Collars
10. Rubber Bands
11. 22 Nylon Screws
12. 22 Nylon Nuts
13. Pneumatics Tank
14. 2 Zip Ties

Step 4: Top Rail/Conveyor Rollers



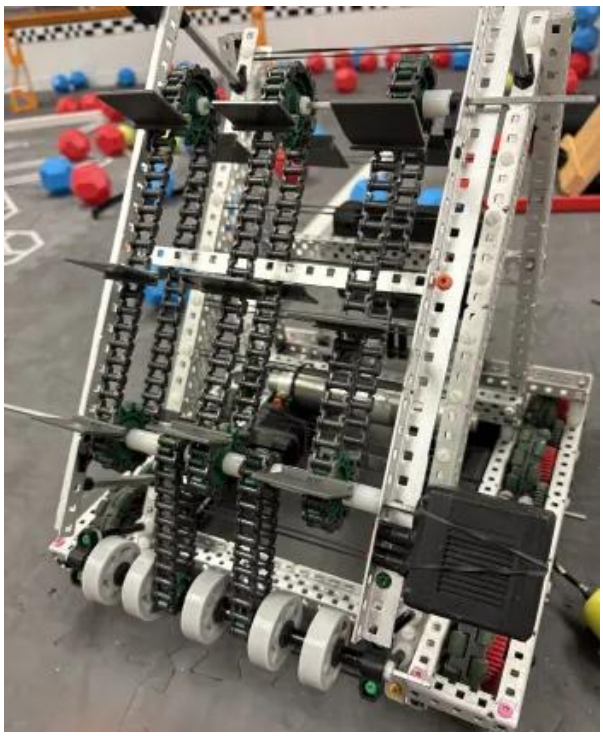
We decided to change to a full top rail instead of just bearings on standoffs for more support and connection points so there is less risk of the system bending in or out and increasing the friction. We are using a 3 wide c-channel that has been cut in half, since this is lighter than using a full c-channel and stronger than using an l-channel.

For the conveyor, we evenly spaced 3 12T sprockets along a top roller and a bottom roller. Links with paddles will be attached between these two in a later step. The pre-roller was also directly chained to the bottom conveyor roller, and a motor cap was added to add the motor in the future. We are connecting the motors directly on the rollers to eliminate chaining and sprockets and structuring that we previously incorporated because this saves weight on the robot. We also began construction for our hood which will be continued later on.

Parts Overview:

1. 3 wide c-channel cut in half
2. 4 4” Standoffs
3. 4 Post Standoff Retainers
4. Chain
5. 1/2"x1/2” Spacers
6. 6 12T Sprockets
7. 2 Motor Caps
8. 2 Axels
9. 4 Post Hex Nut Retainer with Bearing Flat
10. Assorted Screws
11. Nylon Nuts
12. 48T Sprocket

Step 5: Conveyor Links/Flaps



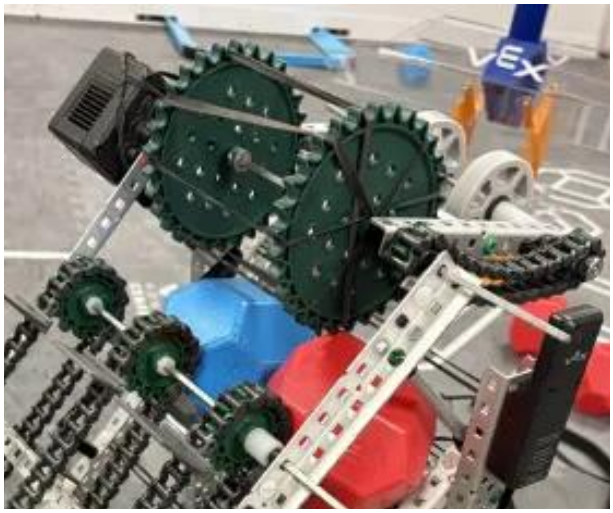
With the conveyor rollers on, we can now add the conveyor links and flaps. We used the smaller profile, lighter links for the sections between flaps. For the flaps, we used a combination of the medium and long size flaps. This allows us to better stagger the blocks for feeding them through the funnels and improves the speed of our intake overall.

We also added a cross brace in the middle of the top rail to help hold the spacing better and prevent bending in the system. Once again we used half-cut 3wide because it's lighter. To finish it off, we attached our motor for the elevator system.

Parts Overview:

1. 1 Half-cut 3 Wide C-Channel
2. Flaps
3. Chain
4. Motor

Step 6: Hood



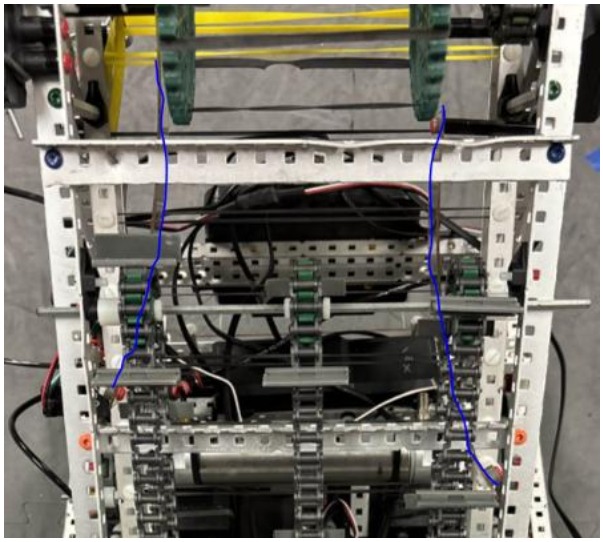
This robot's hood is virtually identical to the last robot's hood, because we found it worked well and were satisfied with it. We have a rubber band roller first, then a small set of flex wheels to help push into the goal. The biggest change between this version and the last version is that the motor is now mounted directly to the hood. Even though this raises our center of gravity, it makes for a lighter and less complex robot.

Parts Overview:

1. Motor
2. 2 48T Sprockets
3. Rubber Bands
4. 2 Half-Cut C-Channels
5. 2 6T Sprockets
6. 1/2" Diameter Spacers
7. 2 Flex Wheels
8. Chain
9. Axel
10. Shaft Collars
11. Assorted Screws

Step 7: Funnels

We are using plastic funnels because they have more flex than metal. They are staggered along the ramp to help facilitate a zipping action between the two sides. This allows us to quickly funnel blocks without them jamming.



Parts Overview:

1. 2 Polycarbonate Pieces
2. Assorted Screws
3. 2 1.5” Standoffs

Step 8: Matchloader

Our matchloader uses gears instead of 2in omni wheels as the hardstops because they are lighter and cheaper to get. The mechanism is now also plastic. It was attached to the robot the same way that the last iteration’s matchloader was attached, since the only thing that changed was the ramping material.

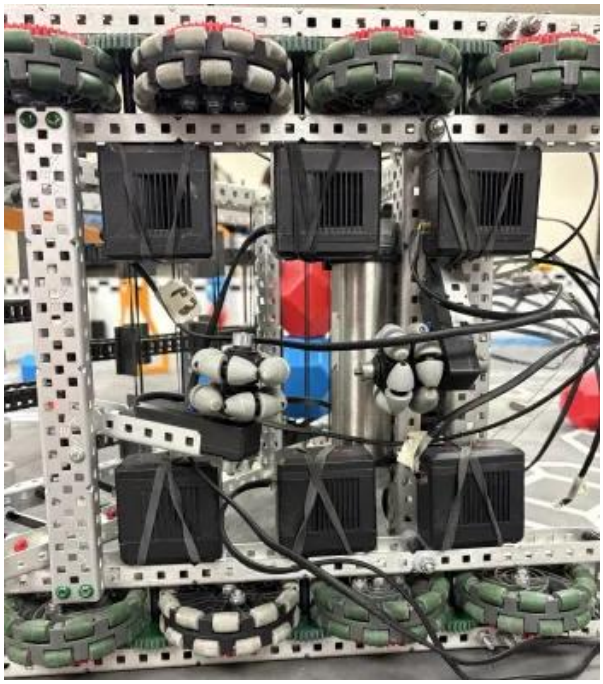


Parts Overview:

1. 4 Short Axels
2. 2 Half-Cut C-Channels
3. 1 Polycarbonate Piece
4. Nylon Screws
5. Nylon Nuts
6. 2 Pillow Blocks
7. 2 High Strength Gears
8. Assorted Screws
9. Shaft Collars
10. High Strength Shaft Collar
11. Rubber Band
12. 3 3” Standoffs
13. 2 2” Standoffs

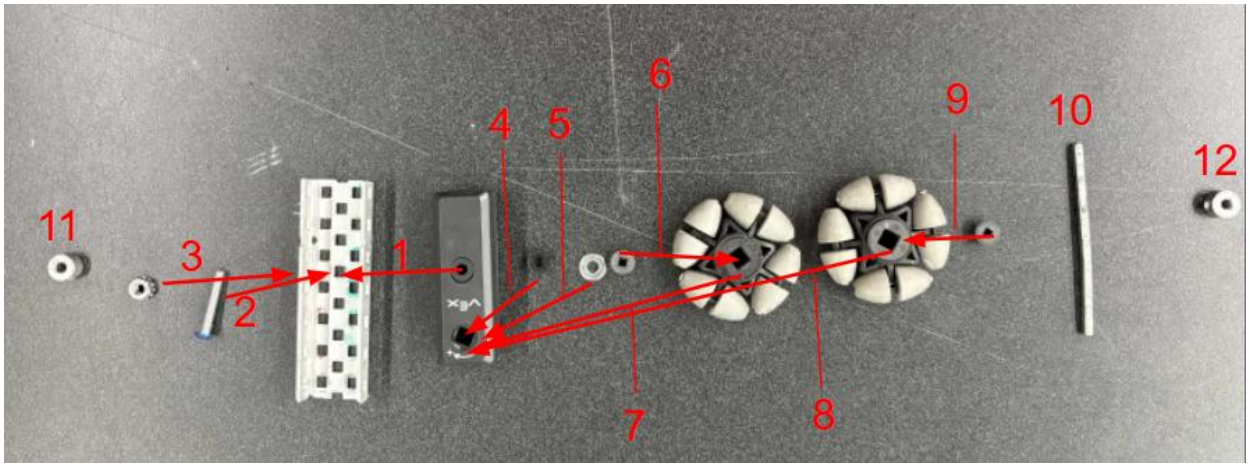
Step 9: Odom Pods

The last step was to attach the odometry pods. We are using double-wide omni wheels since the 2in wheels are square-like in shape and can cause problems with accurate tracking. Setting them like this ensures that they roll better and thus track more accurately when the robot is actually moving.



Parts Overview:

1. 4 Short Axels
2. 2 Double-Wide Omni Wheels
3. 2 Small C-Channels
4. Assorted Screws
5. Shaft Collars
6. Nylon Nuts
7. 1 Half-Cut C-Channel



10/30/2025 - V2 Driver Control Programming

Meeting Goal: Get a fully constructed robot ready for programming/testing.

Participants: Hamza, Andrew

```
void usercontrol(void) {
 // User control code here, inside the loop
 while (1) {
   if (Controller1.ButtonL1.pressing()) {
     highintake.spin(forward, 100, percent);
   } else if (Controller1.ButtonL2.pressing()) {
     highintake.spin(reverse, 100, percent);
   } else {
     highintake.stop();
   }

   // hood
   if (Controller1.ButtonR1.pressing()) {
     hood.spin(forward, 100, percent);
   } else if (Controller1.ButtonR2.pressing()) {
     hood.spin(reverse, 20, percent);
   } else {
     hood.stop();
   }

   bool currentButtonState = Controller1.ButtonDown.pressing();

   if (currentButtonState && !previousButtonState) {
     pistonState = !pistonState;
     matchloader.set(pistonState);
   }

   previousButtonState = currentButtonState;

   bool CurrentButtonState = Controller1.ButtonRight.pressing();

   if (CurrentButtonState && !PreviousButtonState) {
     PistonState = !PistonState;
     wing.set(PistonState);
   }

   PreviousButtonState = CurrentButtonState;

   // This is the main execution loop for the user control program.
   // Each time through the loop your program should update motor + servo
   // values based on feedback from the joysticks.
   // ........................................................................
   chassis.control_arcade();
   wait(20, msec); // Sleep the task for a short amount of time to
                   // prevent wasted resources.
 }
}
```

11/1/2025 - November Plans

Meeting Goal: Figure out our goals for November.

Participants: Austin, David, Alex, Hamza, Andrew, Jenea

We are still working on the V2 robot, so all our task revolve around getting that done.

Team Decisions:

→ We are competing on 11/8 and want to score in the top 5 for auton skills
→ We are competing on 11/8 and want to score in the top 5 for driver skills

Team Build Accomplishments:

→ We successfully built the V2 robot
→ We successfully implemented the solutions that were previously selected

Team Programming Accomplishments:

→ N/A.

Team Goals:

→ Get a fully tested and programmed robot for skills before 11/8.
→ Ensure the robot is functional for matchplay as well before 12/1 for competition on 12/13
→ Begin considering skills routes (did not accomplish last month)

11/2/2025 - Intake: Testing

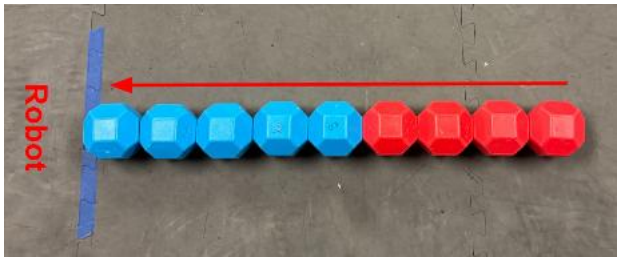
Meeting Goal: Does our intake perform the way we want it to?

Participants: Austin, David, Alex

Test 1 Procedure:Intake to max capacity

1. Set up a series of 9 blocks in a line
2. Begin timer
3. Push the line of 9 blocks into the robot
4. Stop timer once robot has fully picked up all of the blocks

Diagram:



Trial 1	Trial 2	Trial 3	Trial 4	Trial 5	**Average**
3.79	4.19	3.66	5.73	3.54	**4.182 sec**

Test 2 Procedure:

4. Start intake
5. Feed block at random into intake
6. Repeat step 2 until intake jams/can't hold any more blocks. Record value

Trial 1	Trial 2	Trial 3	Trial 4	Trial 5	**Max**
9	8	10	10	9	**10 blocks**

Conclusion:

Changing to floating intake increased our ability to hold blocks. Though it's only a minor step up from our V1 robot, every game piece makes a difference. Jamming was a massive problem with our V1 robot that we worked to try to fix with swapping to rubberbands rather than latex tubes which helped in addition with the swap to converter intake ramp as it allowed the blocks to spin more within the robot allowing for more to get in before stopping. This is a massive upgrade as it allows us to gather more during the autonomous period to step us up nicely for driver control.

11/3/2025 - Intake: Tuning

Meeting Goal: Tune our robot to increase performance.

Participants: Austin, David, Alex

Observations:

The intake for V2 didn't have any observable issues or major problems. It was able to collect a large amount of blocks in a timely speed, and it was able to stay very consistent. Our intake is also very sturdy in structure, so we are quite confident that it will be reliable during competitions. Minor fixes can be made to reduce jamming but overall quite happy with the results.

Problem Status:

Minor to none

Solutions:

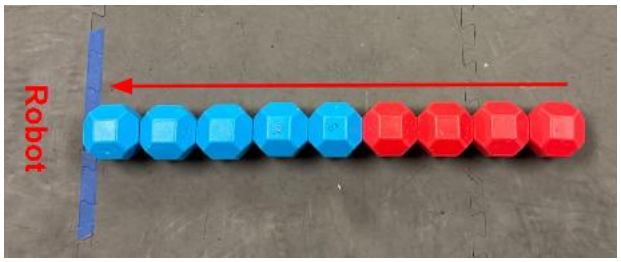
Cutting out 2 of the inserts in the 2 inch flex wheels to allow for easier intaking of more than one block and to gain a better grip of the block.

Tests Re-Ran:

Test 1 Procedure:Intake to max capacity

1. Set up a series of 9 blocks in a line
2. Begin timer
3. Push the line of 9 blocks into the robot
4. Stop timer once robot has fully picked up all of the blocks

Diagram:



Trial 1	Trial 2	Trial 3	Trial 4	Trial 5	**Average**
3.92	3.76	4.08	4.93	3.87	**4.112 sec**

Test 2 Procedure:

7. Start intake

8. Feed block at random into intake
9. Repeat step 2 until intake jams/can’t hold any more blocks. Record value

Trial 1	Trial 2	Trial 3	Trial 4	Trial 5	**Max**
9	10	10	9	10	**10 blocks**

Conclusions:

The solution was simple but improved consistency and slight improvement in speed. We are very happy with this design and will likely use it in future robot designs.

11/4/2025 - Matchloader: Testing

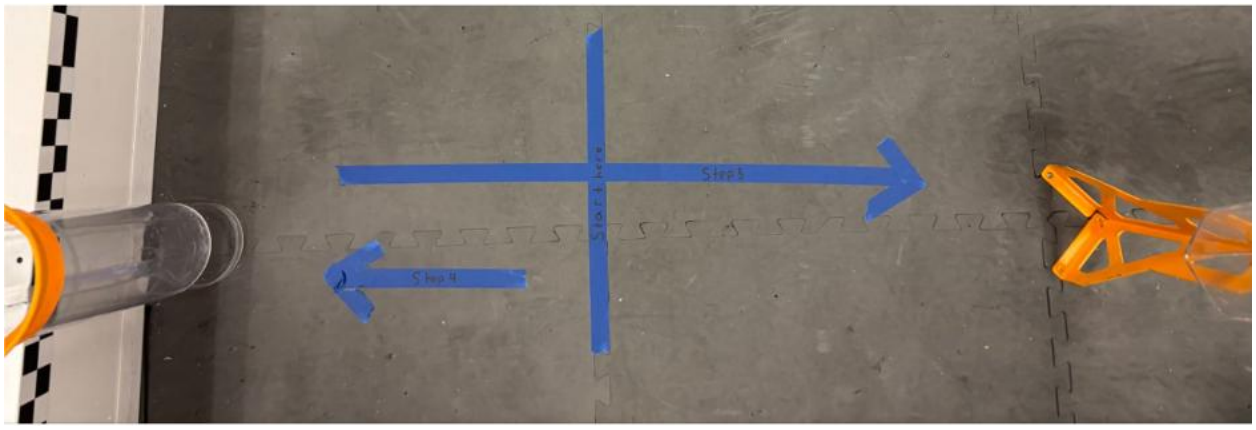
Meeting Goal: Does our matchloader perform the way we want it to?

Participants: Austin, David, Alex

Test 1 Procedure: Match imitation

1. Load matchtube
2. Line up robot one tile in front of the tube
3. Start timer
4. drive up to tube and clear
5. Move forward to long goal and score
6. Stop timer once all game pieces are scored

Diagram:

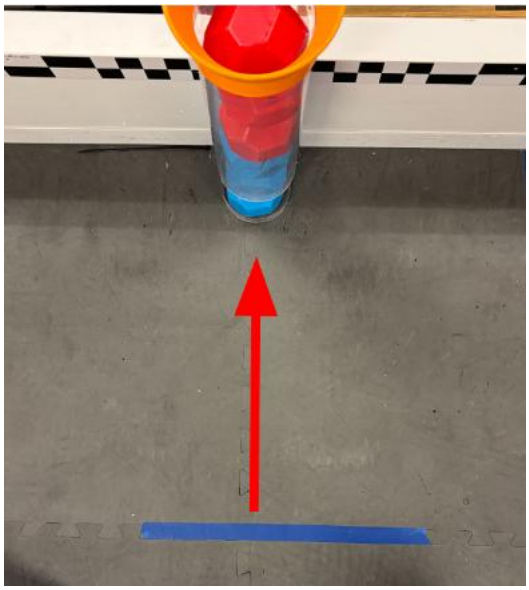


Trial 1	Trial 2	Trial 3	Trial 4	Trial 5	**Average**
6.09	8.09	5.22	7.07	8.35	**6.964 sec**

Test 2 Procedure: Clearing the tube

1. Fill up matchload tube
2. Line up robot one tile from tube
3. Lower match loader
4. Begin timer
5. Drive into the tube
6. Stop timer once tube is cleared

Diagram:



Trial 1	Trial 2	Trial 3	Trial 4	Trial 5	**Average**
1.56	3.01	1.98	2.34	1.70	**2.118 sec**

Conclusion:

We concluded our match loader is fast but inconsistent. Following these tests will be modifications to improve consistency. Our plans to do this are to adjust rubber band strength and distance stretched. This would allow us to test if the problem is the tension or the poly. Using the formula “$Y = (F/A) \div (\Delta L/L_0) = (F/A) \times (L_0/\Delta L)$” which is the Young’s modulus equation. This would allow us to properly determine where to put the band without having to cut or change the robot.

11/5/2025 - Matchloader: Tuning

Meeting Goal: Tune our robot to increase performance.

Participants: Austin, David, Alex

Observations:

Our matchloader, while able to deploy quickly, is inconsistent in performance. It ranges from unloading the matchloader effectively to taking a handful of jabs until it is able to facilitate blocks into the intake. The issue lies in the rubber band, which is either too weak or not tense enough to restrict the polycarbonate piece from unnecessary movement. The piece of poly carbonate also has some issues due to its size and shape.

Problem Status:

Medium

Solutions:

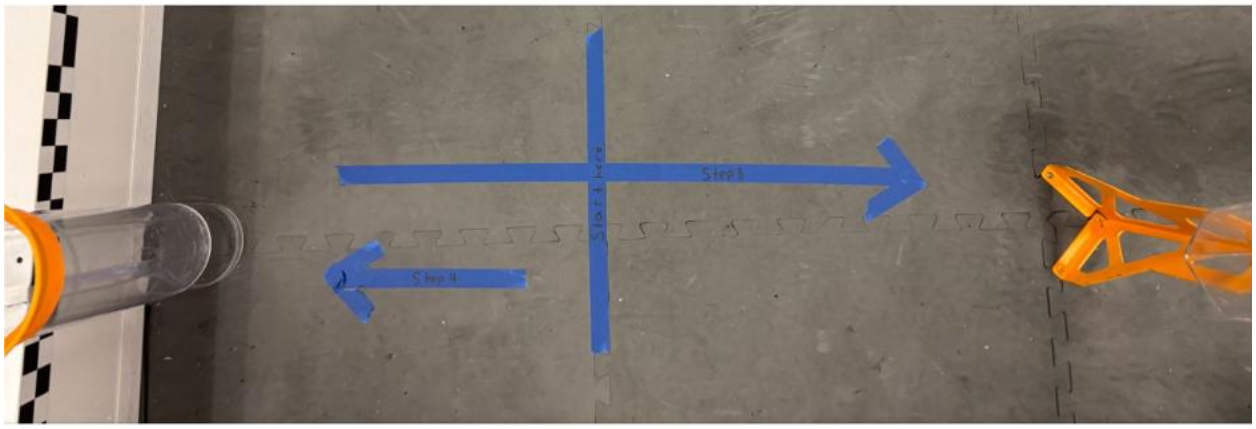
We will test different strengths of rubber bands, as well as different tension through stretching to see which is the most effective at remaining consistent. Along with this, we will test out different placements and utilizing the Young's modulus equation to further improve its consistency. We also will have to make a piece of polycarbonate that is more fit for the matchloader tubes.

Tests Re-Ran:

Test 1 Procedure: Match imitation

1. Load matchtube
2. Line up robot one tile in front of the tube
3. Start timer
4. drive up to tube and clear
5. Move forward to long goal and score
6. Stop timer once all game pieces are scored

Diagram:

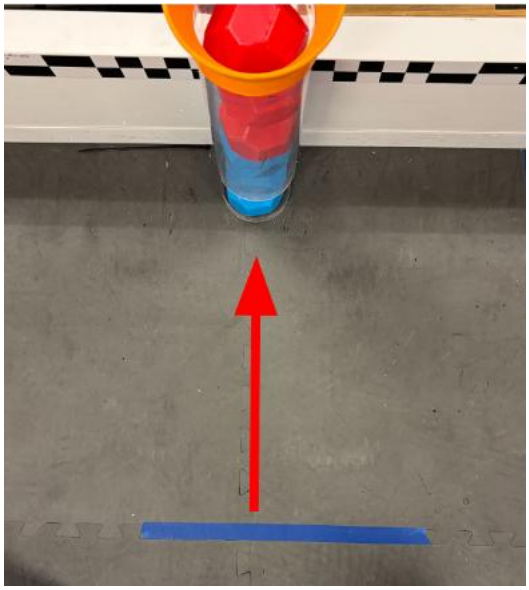


Trial 1	Trial 2	Trial 3	Trial 4	Trial 5	**Average**
6.30	7.82	6.51	5.22	7.04	**6.578 sec**

Test 2 Procedure: Clearing the tube

1. Fill up matchload tube
2. Line up robot one tile from tube
3. Lower match loader
4. Begin timer
5. Drive into the tube
6. Stop timer once tube is cleared

Diagram:



Trial 1	Trial 2	Trial 3	Trial 4	Trial 5	**Average**
1.61	2.51	1.74	2.18	1.63	**1.934 sec**

Conclusions:

The speed and consistency improved which will be very helpful for creating our autonomous routes and to improve the quality of life for our driver.

11/9/2025 - 1st Competition Recap

Meeting Goal: How did we do at our first competition?

Participants: Austin, David, Alex

Ranking:

Rank	Team	Driver Attempts	Driver Highscore	Programming Attempts	Programming Highscore	Total Highscore
1	5013X	3	90	3	52	142
2	22603B	3	80	3	54	134
3	22603A	3	71	3	35	106

We made top 3 at the competition with a 71 driver score and a 35 programming score for a combined 106. Our biggest area to gain in is programming, so we will focus on that, but still continue to practice driver skills. For the next competition, our goals are to score a 50 programming score and an 85 driver skills score.

Problems at the Comp:

1. Driver one matchloader had a failure in the ruberband rebounding
2. Driver 2 struggled to clear park fully and didn't end up parked
3. Driver 3 left hood on too long and missed park clear because of wasted time

Future Plans:

Before our next competition, we need to account for having opponents on the field as well. One method of defense is being able to **descore our opponents blocks from the long goals.** Thus, we have decided to add this to our robot to better be able to secure wins more easily. We will ideally need a simple effective mechanism that can be added onto the robot without requiring any adjustment of any other systems. Our next competition is on 12/13/2025, so we have about 1 month to design and build this system. We also need to plan and program our autonomous strategies for matches.



11/11/2025 - Descore Brainstorming

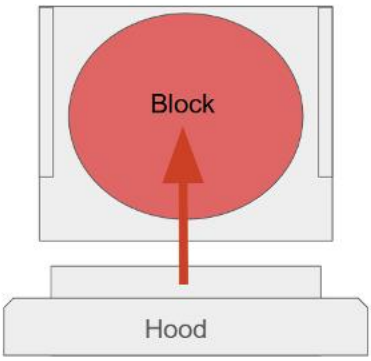
Meeting Goal: What options are available to us to descore the long goals?

Participants: Austin, Jenea

Descore Types:

Hood

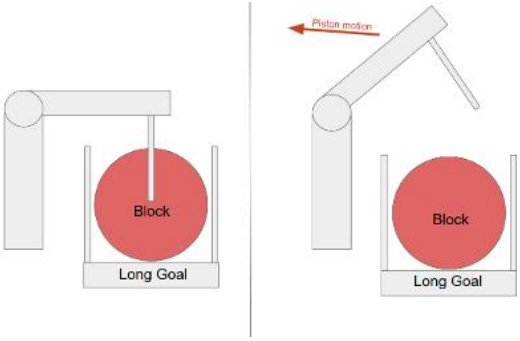
A curved guide placed above the intakr to control the objects trajectory. The hood is lined up with the open part of the long goal and would contact to transfer the blocks out. This would allow for a successful descore within the majority of the goal. Although swapping the control bonus would be hard because only the end goal would be accessible.



Pros: fast, easy, alignment,
Cons: size, use of limited poly

Catear

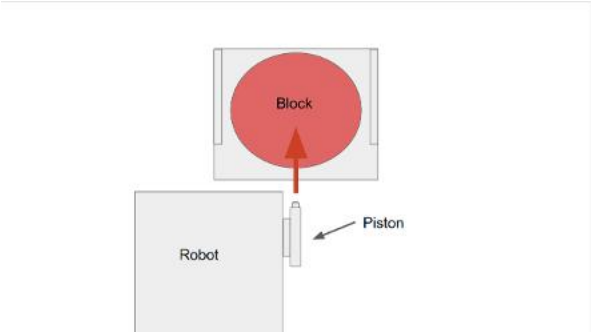
A chain and sprocket system used to transfer the power between shafts. By using a standoff or a screw the catear mechanism moves over the goal and lowers to allow efficient descoring. This also allows for the swap of control bonus unlike other options that is controlled by how much is descored.



Pros: easy, fast, alinement, more access to goal
Cons: uses pistons

Piston

A pneumatic actuator that provides a quick linear motion for opening, closing, and lifting mechanisms. By mounting a piston at the top the robot is able to desocre a full goal.Though it is effective is can make it harder to obtain the bonus.



Pros: Fast, easy
Cons: alinement, uses piston

11/13/2025 - Descore Selection

Meeting Goal: Of the brainstormed descore types above, what is the best to use?

Participants: Austin, David, Alex

Parameters:

Speed

- We want our descore to be able to quickly remove blocks from the long tubes so that we can get them all out before our opponents come to defend against us.
- **Weighting:** 0.28
- **Concept Scoring:**
  - ***Hood: 4/5***
  - ***Catear: 4/5***
  - ***Piston: 3/5***

Powering

- Since we intend to use 6 motors on the drive and 2 on the intake, we ideally want to be able to use only pneumatics to power this system.
- **Weighting:** 0.25
- **Concept Scoring:**
  - ***Hood: 5/5***
  - ***Catear: 4/5***
  - ***Piston: 4/5***

Complexity

- The simpler a mechanism is, the more easily we will be able to implement and adjust it and the less time we will need to spend overall on it so we want to prioritize a simple mechanism.
- **Weighting:** 0.23
- **Concept Scoring:**
  - ***Hood: 4/5***
  - ***Catear: 5/5***
  - ***Piston: 4/5***

Reliability

- We want our mechanism to be strong and reliable so it does not break in a match, since if it breaks we cannot descore our opponents blocks quickly.
- **Weighting:** 0.24
- **Concept Scoring:**
  - ***Hood: 4/5***
  - ***Catear: 5/5***
  - ***Piston: 3/5***

Decision Matrix:

	Weighting	Hood		Catear		Piston	
Speed	0.28	4	1.12	4	1.12	3	0.84
Powering	0.25	5	1.25	4	1.00	4	1.00
Complexity	0.23	4	0.92	5	1.15	4	0.92
Reliability	0.24	4	0.96	5	1.20	3	0.72
Totals:	1.00		**4.25**		**4.47**		**3.48**

Conclusion:

We will move forward with the cat-ear solution. This had the highest score in our decision matrix, at a 4.47/5, so we are confident that it will meet our needs well.

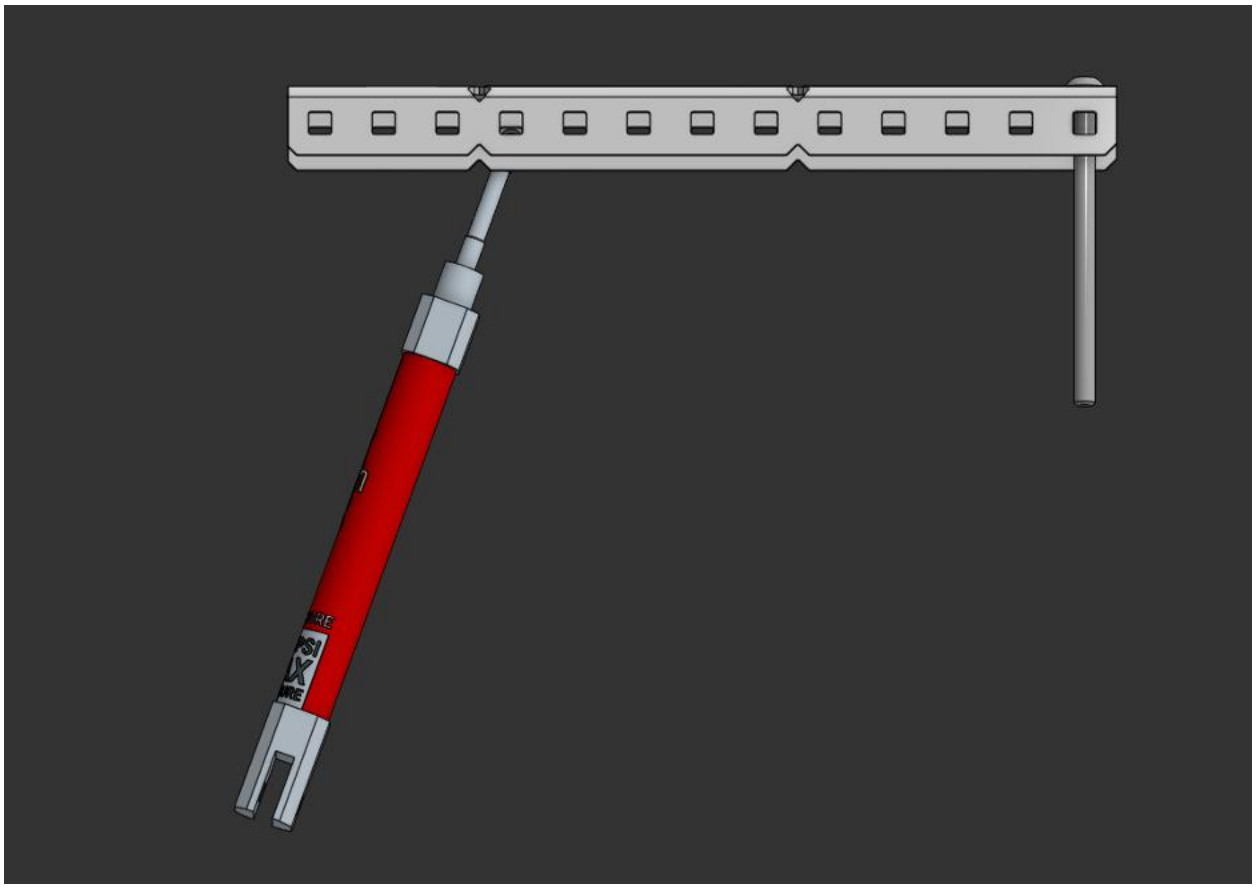
11/15/2025 - Descore Design Plan

Meeting Goal: Create a CAD model of the descore concept.

Participants: Austin, David, Alex

Design Specifications:

- **Pneumatic:** .50
- **Screw:** 2.5”



Additional info:

We plan to add green to our cat ear and to our controller. This is done to allow our co-drivers an easy call out for descore without giving it away to our opponent. By yelling “green left” or “green right” we are able to successfully confuse our opponents and allow our driver to easily get a descore without stressing the callout gave it away. We plan to have the screw seen above covered in a layer of green electrical tape allowing us to keep the desired width and still be able to slide across the goal. Our controller uses the claw grip where we can print a new green claw which gives an easy visual cue that you can see within your peripheral vision as it has great contrast to the blue grip.

11/16/2025 - Descore: Testing

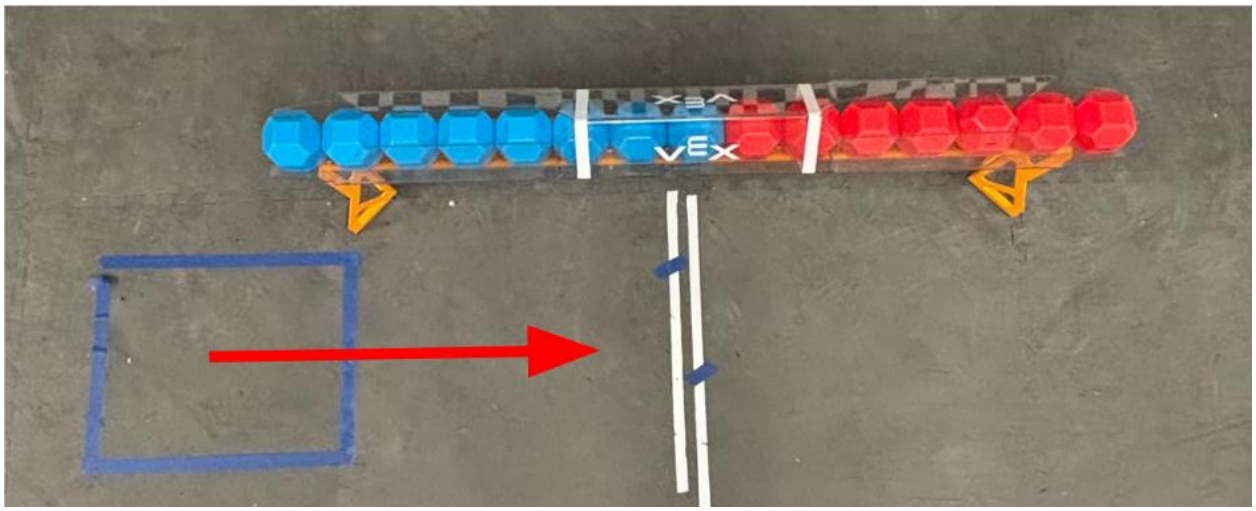
Meeting Goal: Does our descore perform the way we want it to?

Participants: Austin, David, Alex

Test 1 Procedure: Max descore

1. Fill long goal
2. Line up robot at end of goal
3. Lower descore
4. Ram robot into goal
5. Record how many got descored

Diagram:

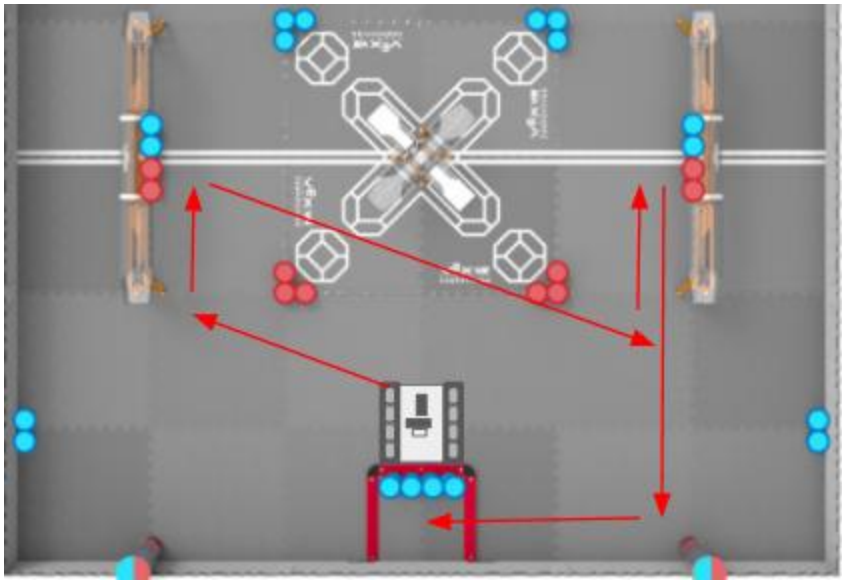


Trial 1	Trial 2	Trial 3	Trial 4	Trial 5	**Max**
11	9	10	8	11	**11**

Test 2 Procedure: Match imitation

1. Fill up both long goals
2. Line up robot at park zone
3. Start timer and drive towards goal 1
4. Descore and drive to other goal
5. Descore and move towards parking zone
6. Park and record time

Diagram:



Trial 1	Trial 2	Trial 3	Trial 4	Trial 5	**Average**
29.86	17.11	18.96	20.42	15.89	**20.448**

Conclusion:

Our descore tech is as effective as we hoped but problems are apparent. Though we can't fully descore it, we were able to get 11 out which is a majority of the goal. Once our team is more used to driving with cat ear, our times will get better. The true test is how effective it'll be in real competition and scrims. As a team we'll run more tests and make adjustments as needed but for now this will treat us nicely.

11/17/2025 - Descore: Tuning

Meeting Goal: Tune our robot to increase performance.

Participants: Austin, David, Alex

Observations:

The descore system is effective, however, it has difficulty descoring all blocks from a long goal. This can be for a multitude of reasons, such as its inability to reach the center of the long goal to just not being experienced in driving the bot and using this function.

Problem Status:

Minor

Solutions:

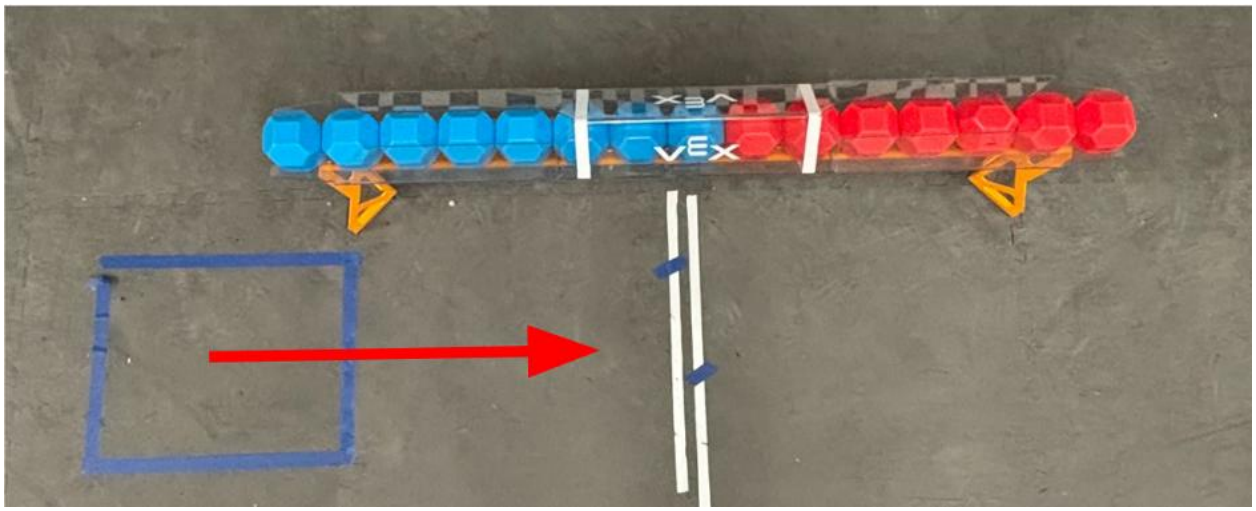
To fix this problem, we would most likely need a different kind of descoring system. Despite this, we will most likely leave the descore system alone because it is effective as is. Our driver just needs to practice with it to get used to it.

Tests Re-Ran:

Test 1 Procedure: Max descore

1. Fill long goal
2. Line up robot at end of goal
3. Lower descore
4. Ram robot into goal
5. Record how many got descored

Diagram:

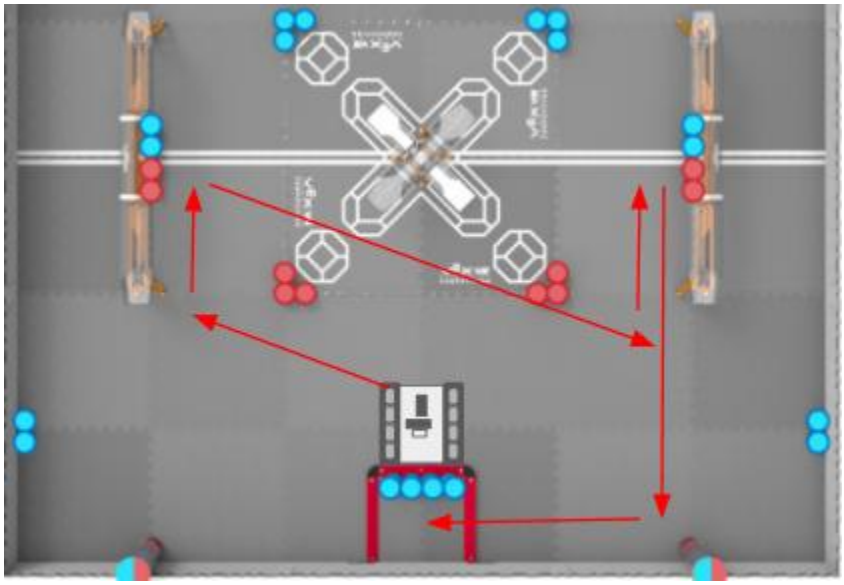


Trial 1	Trial 2	Trial 3	Trial 4	Trial 5	**Max**
11	9	10	8	11	**11**

Test 2 Procedure: Match imitation

1. Fill up both long goals
2. Line up robot at park zone
3. Start timer and drive towards goal 1
4. Descore and drive to other goal
5. Descore and move towards parking zone
6. Park and record time

Diagram:



Trial 1	Trial 2	Trial 3	Trial 4	Trial 5	**Average**
29.86	17.11	18.96	20.42	15.89	**20.448**

11/18/2025 - Autonomous Strategies

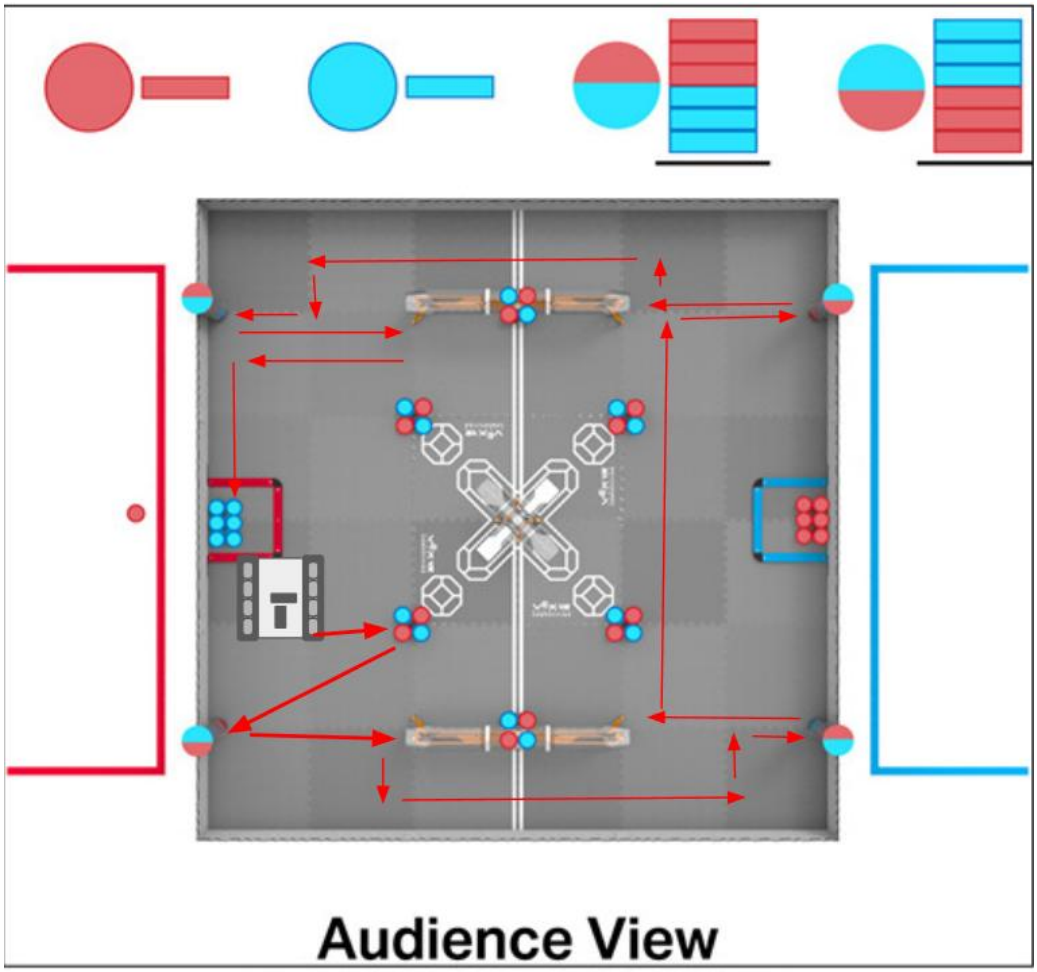
Meeting Goal: figure out what a good autonomous strategy to run at competitions and execute it.

Participants: Hamza, Andrew

2v2 auton, collect 3 middle, load 4 into long goal, use wing to screw over other team and get control

Solo win point, collect 3 middle, score into middle goal, score into long goal, go to other side, load 3 from matchloader, score into long goal

Skills, collect one middle, collect all from matchloader, score long goal, go to other side, load matchloader, score long goal, go to 3rd matchloader, load, score long goal, go to final matchloader, load, score long goal, park with clearance



12/1/2025 - December Plans

Meeting Goal: Figure out our goals for December

Participants: All

Team Decisions:

→ Our drive is satisfactory
→ Rebuild robot

Team Build Accomplishments:

→ Finished v2 bot
→ Added cat ear tech

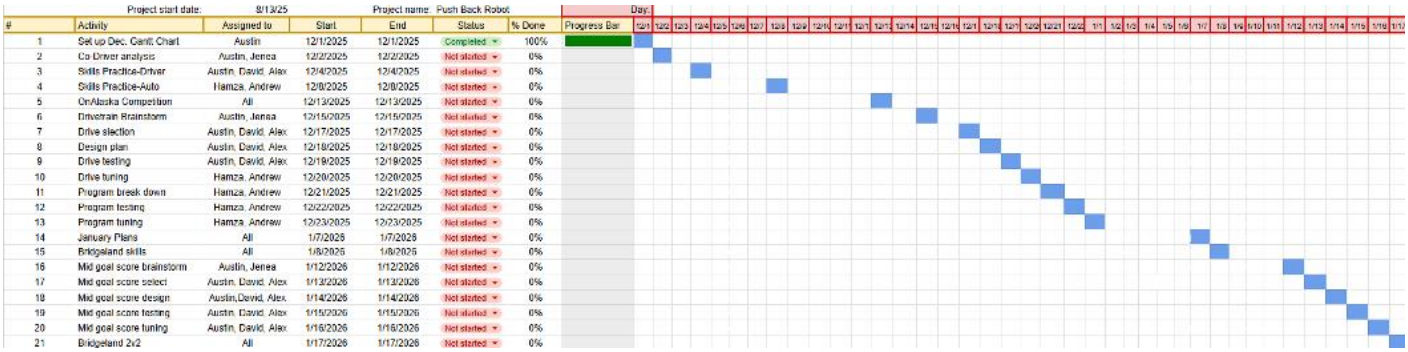
Team Programming Accomplishments:

→ We switched to odom
→ Added 2v2 code for both alliances

Team Goals:

→ **Prepare for our next competition**
→ Train new co-drivers in case our members can't make a comp.
→ Robot rebuild.
→ Break down the programming.

Updated gantt chart:



	Project start date:	8/13/25		Project name:	Push Back Robot		Day:
#	Activity	Assigned to	Start	End	Status	% Done	Progress Bar
1	Set up Dec. Gantt Chart	Austin	12/1/2025	12/1/2025	Completed	100%	
2	Co Driver analysis	Austin, Jenea	12/2/2025	12/2/2025	Not started	0%	
3	Skills Practice-Driver	Austin, David, Alex	12/4/2025	12/4/2025	Not started	0%	
4	Skills Practice-Auto	Hamza, Andrew	12/8/2025	12/8/2025	Not started	0%	
5	OnAlaska Competition	All	12/13/2025	12/13/2025	Not started	0%	
6	Drivetrain Brainstorm	Austin, Jenea	12/15/2025	12/15/2025	Not started	0%	
7	Drive slection	Austin, David, Alex	12/17/2025	12/17/2025	Not started	0%	
8	Design plan	Austin, David, Alex	12/18/2025	12/18/2025	Not started	0%	
9	Drive testing	Austin, David, Alex	12/19/2025	12/19/2025	Not started	0%	
10	Drive tuning	Hamza, Andrew	12/20/2025	12/20/2025	Not started	0%	
11	Program break down	Hamza, Andrew	12/21/2025	12/21/2025	Not started	0%	
12	Program testing	Hamza, Andrew	12/22/2025	12/22/2025	Not started	0%	
13	Program tuning	Hamza, Andrew	12/23/2025	12/23/2025	Not started	0%	
14	January Plans	All	1/7/2026	1/7/2026	Not started	0%	
15	Bridgeland skills	All	1/8/2026	1/8/2026	Not started	0%	
16	Mid goal score brainstorm	Austin, Jenea	1/12/2026	1/12/2026	Not started	0%	
17	Mid goal score select	Austin, David, Alex	1/13/2026	1/13/2026	Not started	0%	
18	Mid goal score design	Austin,David, Alex	1/14/2026	1/14/2026	Not started	0%	
19	Mid goal score testing	Austin, David, Alex	1/15/2026	1/15/2026	Not started	0%	
20	Mid goal score tuning	Austin, David, Alex	1/16/2026	1/16/2026	Not started	0%	
21	Bridgeland 2v2	All	1/17/2026	1/17/2026	Not started	0%	

Due to our district's calendar our progress is more spaced out and linear. Another update will be made after we get data from our next 3 competitions.

12/2/2025 - Co-Driving Analysis

Meeting Goal: Analyse what makes a good co-driver and how to effectively help the driver.

Participants: Jenea, Austin

Key notes:

→ Descoring is a huge thing to look out for both defense and offence
→ Matchloaders: don't load more than 6 at a time, load one at a time
→ Color coding: Our robot is color coded to allow simple and easy communication to our driver without giving away our intention.
→ Rules: Another job is to make sure your driver does not violate rules while driving as they can be accidental but preventable. In example
→ Time: Keeping time is a valuable part of co-driving as it takes away another thing your driver would have to keep track of. Valuable times to communicate before testing is: 1 min, 30 sec, 15sec, 10 sec, and a count down from 5 sec.

Valuable sources:

→ Game Manual: it provides rules to look out for and limitations that co-drivers have.
→ LukeDoesRobotics on YouTube: He has match commentaries from the current and past games which give insight on things to communicate to our driver.

Testing:

In testing Co-drivers will be referred to as CD for simplicity.
Color code: RED = Stressed ORANGE = moderate stress YELLOW = little stress Green = no stress

	Test 1	Test 2	Test 3	Test 4	Average
CD1 time+game CD2 Loading color callouts					
CD1 game+color callouts CD2 Time+Loading					
CD1 Game+loading CD2 Color callouts+time					

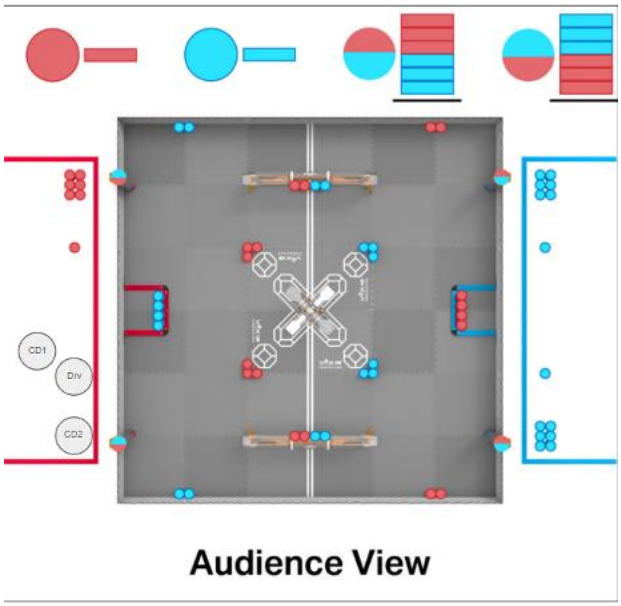
Conclusions:

After testing from our notes we have assigned tasks to our two co-drivers. CD1 would keep track of the game and the other robots looking for opportunities for our driver to capitalise in example descores and defence. CD1 would also keep track of our alliance partner and call to help them when needed. Also once the time has reached 10 seconds, CD1 will communicate to our driver to make his way to the parkzone. The responsibilities we placed on CD2 are timekeeping and loading the matchloaders. Having these two tasks allows CD1 to focus on the game at all times.

Callouts Developed:

→ Color based: Green is our descore, Red is our matchloader, Blue is our score
→ Time: 1 minute, 30 seconds, 15 seconds, 5 seconds counting down. CD1 will communicate to park at 10 seconds instead of CD2. Which was determined after our driver paid more attention to CD1 and we as a team saw the parking callout as a very important one.
→ Rules: This one is hard to communicate but the easiest way we found is just to say "don't do this" because we don't want to get disqualified which affects rankings/performance.

Location of Co-Drivers:



We would have CD1 positioned behind our driver to give callouts directly in his ear. CD1 is also positioned near the middle of the field which would provide them an easy viewing angle of the field. CD2 will take a position in front of the match tube with the number of blocks decided by our alliance.

Who should be Co-Drivers:

Our team consists of a lot of tall people so seeing the field is really easy. CD2 after testing will either be Alex or Hamza due to them not being super strong CD1's. CD1 will be either Austin or Andrew but due to availability it will likely be Austin. Austin and Andrew were able to show great game knowledge and leadership when testing which is why they were chosen.

12/4/2025 - Skills Practice - Driver

Meeting Goal: Prepare for our competition and see what our max driver score is.

Participants: Austin, David, Alex

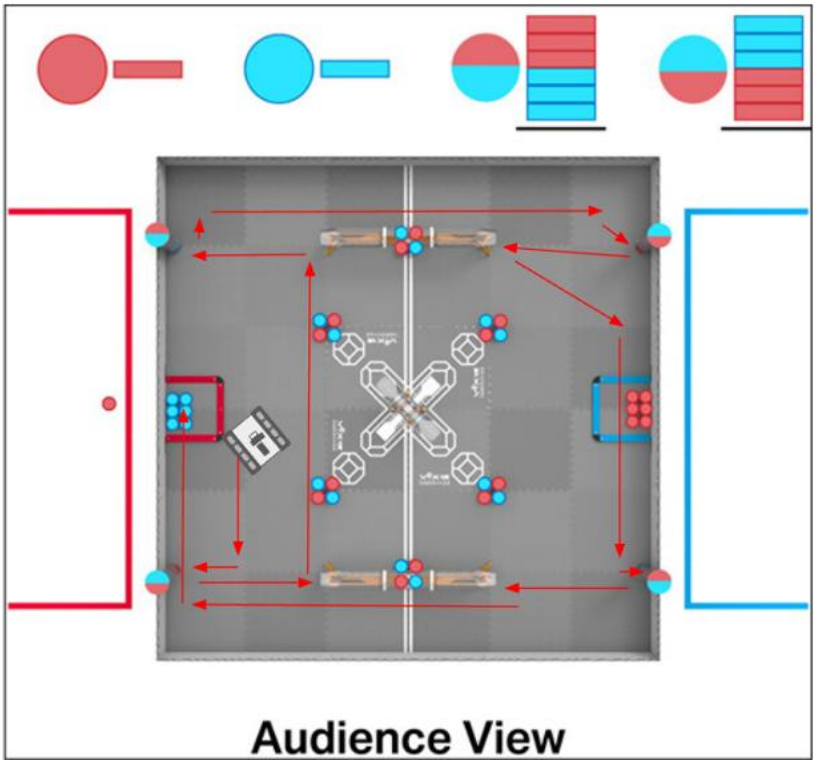
Competition Goals:

Our goal as a team is to hit 80 driver skills at our competition December 13th. Though time may be an issue due to the competition also being a 2v2 competition, that is a problem to tackle later. 80 Driver is highly achievable with the route we take. To achieve this we are going to refine our strategy and route and take notes on improvements we can make.

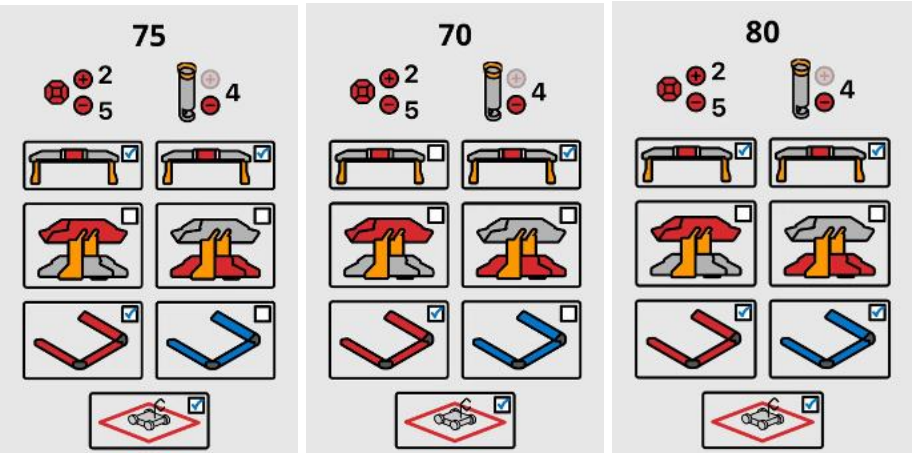
Strategy:

Our main strategy for skills is maximizing the big point bonuses as possible. So we look to clear all the loaders, clear all of the park zones and get both long goal controls.

Route:



Scores:



Room for improvement:

Our consistency is an issue with the route and the lack of the ability to score mid goal sets a cap on our score to 80. Even with max of 80 we dont alway hit it due to simple problems like failing to clear a park or not fully achieving long goal control. Fixing these would allow us to make an easy transition to mid goal once we develop one.

Conclusions:

As a team we need to improve consistency with the runs. Next month we plan to develop a way to score mid goal which will push our skills capabilities ten fold. With the competition this day to work on skills was very needed. Time is of the essence and our team is working to use every second.

12/8/2025 - Skills Practice - Autonomous

Meeting Goal: Access our autonomous portion of skills to understand what to improve before the 13th.

Participants: Hamza, Andrew

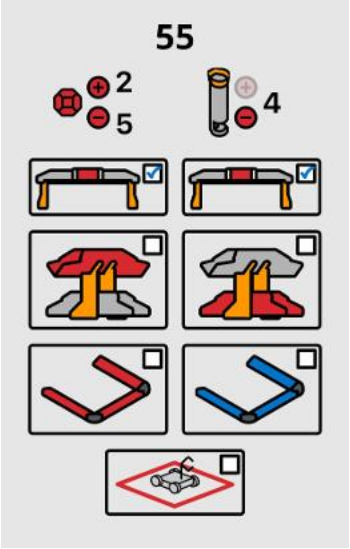
Competition Goals:

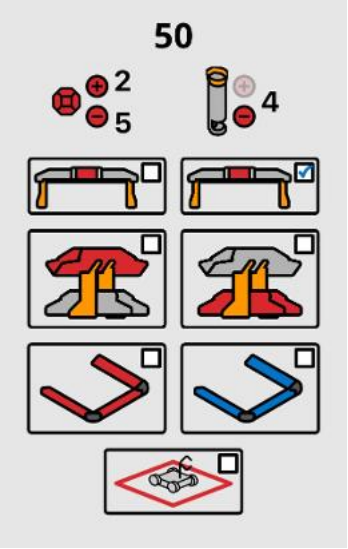
Our goal as a team is to hit 55 Autonomous skills at our competition December 13th. Even though our priority for code in preparation for this competition was 2v2, we still hope to achieve a high autonomous score and have divided time to work on that too. 55 Autonomous is highly achievable with the route we take. To achieve this we are going to refine the steps the robot takes, do test runs to fix potential issues, and utilize odometry for more consistent scores.

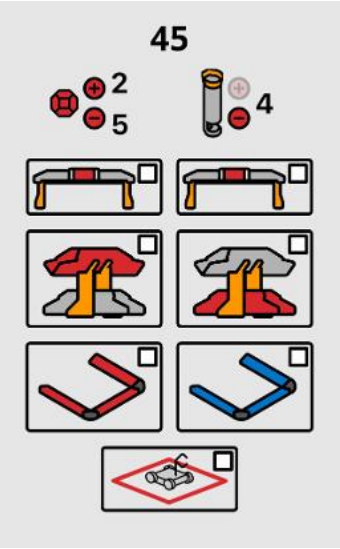
Strategy:

Our main strategy for skills is scoring as much as possible. So we look to clear all the loaders and get both long goal controls.

Scores:







Room for improvement:

Our consistency is sometimes an issue with the route and the lack of the ability to score mid goal or the lack of time to park sets a cap on our score to 55. Even with a max of 55 we don't always hit it due to simple problems like failing to matchload a tube or not fully achieving long goal control. Fixing these would allow us to hit higher scores more consistently.

Conclusions:

Overall, we need to improve consistency with the runs. Next month we plan to develop a faster code and robot that will allow us to make time for parking. With the upcoming competition, setting some time aside to achieve a good skills score was crucial. We are working on a limited time constraint, but we aim to maximize it to give us the best chance possible for success.

Code:

```
void auton_skills(){

//  Set co ords
chassis.set_coordinates(0, 0, 0);

//  Tube 1
chassis.drive_to_point(0, 38.5);

chassis.turn_to_angle(90);
matchloader.set(true); // Sets Matchloader Down

wait(0.5, seconds);

setVoltage(3.75); // Slow the robot down
driveWithShortExitTime(12, 1000);
setVoltage(8.5); // Reset

//  Load Tube
highintake.spin(forward, 100, percent);
wait(2, seconds);
highintake.stop();
//  Score other side
chassis.drive_distance(-20);
matchloader.set(false);

chassis.turn_to_point(0, -56);
chassis.drive_to_point(0, -56);

chassis.turn_to_angle(90);
setVoltage(6);
driveWithShortExitTime(-20, 1000);
chassis.drive_distance(1);
setVoltage(8.5);

//  Score Balls
highintake.spin(forward, 100, percent);
hood.spin(forward, 100, percent);

wait(3, seconds);
```

```
highintake.stop();
hood.stop();

//  Tube 2
chassis.turn_to_point(0, -55.5);
chassis.drive_to_point(0, -55.5);
chassis.turn_to_angle(90);

matchloader.set(true);
wait(0.5, seconds);

setVoltage(3);
driveWithShortExitTime(16, 1000);
chassis.drive_distance(-1);
setVoltage(8.5);

// Load Tube
highintake.spin(forward, 100, percent);
wait(2, seconds);
highintake.stop();


//  Score
setVoltage(6);
driveWithShortExitTime(-36, 1000);
chassis.drive_distance(1);
chassis.turn_to_angle(90);
setVoltage(8.5);

//  Score Balls
highintake.spin(forward, 100, percent);
hood.spin(forward, 100, percent);

wait(3, seconds);

highintake.stop();
hood.stop();
matchloader.set(false);

//  Tube 3

chassis.turn_to_point(-10, -56);
chassis.drive_to_point(-10, -56);

setVoltage(4);
chassis.right_swing_to_angle(0);
chassis.right_swing_to_angle(-90);
setVoltage(8.5);
```

```
wait(0.5, seconds);

chassis.drive_distance(90);

chassis.turn_to_point(-92.5, -56);
chassis.drive_to_point(-92.5, -56);

chassis.turn_to_angle(-90);
matchloader.set(true);

wait(0.5, seconds);

setVoltage(4);
driveWithShortExitTime(16, 1000);
chassis.drive_distance(-1);
setVoltage(8.5);

//  Load Tube
highintake.spin(forward, 100, percent);
wait(2, seconds);
highintake.stop();

chassis.turn_to_angle(-90);

// Score other side
chassis.drive_distance(-20);
matchloader.set(false);

chassis.turn_to_point(-95, 38.5);
chassis.drive_to_point(-95, 38.5);

chassis.turn_to_angle(-90);
setVoltage(6);
driveWithShortExitTime(-24, 1000);
chassis.drive_distance(1);
setVoltage(8.5);

//  Score Balls
highintake.spin(forward, 100, percent);
hood.spin(forward, 100, percent);

wait(3, seconds);

highintake.stop();
hood.stop();

//  Tube 4
chassis.turn_to_point(-95, 38.5);
chassis.drive_to_point(-95, 38.5);
chassis.turn_to_angle(-90);
```

```
matchloader.set(true);

wait(0.5, seconds);

setVoltage(3);
driveWithShortExitTime(13.5, 1000);
chassis.drive_distance(-1);
setVoltage(8.5);

//  Load Tube
highintake.spin(forward, 100, percent);
wait(2, seconds);
highintake.stop();

chassis.turn_to_angle(-90);

//  Score
setVoltage(6);
driveWithShortExitTime(-36, 1000);
chassis.drive_distance(1);
setVoltage(8.5);

//  Score Balls
highintake.spin(forward, 100, percent);
hood.spin(forward, 100, percent);

wait(3, seconds);

highintake.stop();
hood.stop();
matchloader.set(false);

}
```

12/13/2025 - OnAlaska Competition

Meeting Goal: How did we do at the competition?

Participants: All

Goals for competition per category:

Skills tournament:

Our goal for the skills aspect of the competition was to push for 80 driver and 112 programing. This goal seemed possible after our testing prior but it is always unknown what will happen during the competition.

2v2 tournament:

The goal is always to win but another goal is to make our team a notable team amongst the top teams at the competition. This will be our first 2v2 tournament which is a different flow of game compared to skills. Our robot is set up for success in 2v2 so this competition is an opportunity to put it to the test and make changes based on results.

What went wrong:

At the competition our team faced many challenges which we did not anticipate:

- Our robot was larger than we had expected, which failed inspection. This changing of size caused us to move parts around which lead to issues down the road.
- Due to a crack in the screen we had to swap to our spare brain. We accidentally mounted it upside down which led to our ports being wrong but was an easy fix.
- Our Odom was mounted backwards, causing the bot to cross the auton line every autonomous round. This was an easy fix that helped in the later rounds of competition.
- For our Skills run, our autonomous faced issues similar to the 2v2, but we unfortunately didn't have time to fix it. For driver, a rubber band left on the field got caught in our drive train and jammed the gears.

All of these struggles put our mental perseverance to the test but our team pushed through and did quite well in the competition and earned the Judges award.

Placement:

5th overall in the 2v2 tournament

Awards:

Judges award

Conclusions:

This competition was a learning experience for our team as we learned many things we would have to change moving forward. An upside was our call out “green” was a massive success as it allowed our driver to go for a descore before the opposing team noticed. Our driver has requested more speed so our team will go back to brain storming for the drivetrain. The 2v2 competition wasn't smooth sailing but we were able to persevere through it and achieve a respectable placement. We will be aiming higher in the future with the upcoming changes and practice.

Next steps and notes to improve:

Size of the robot:

- Creating a size checker similar to the ones used at competition
- Future robots cad being checked for size before building

Robot issues:

- Sometimes it's unavoidable but we will be double checking our robot that everything is in the right place and tightened before we start inspection.
- Better observe the condition of the field and robots as we had issues with parts getting stuck in our robot from others and another robot pulling our odom wheel off.
- After every round if time permits check the robot for loose or broken parts.

Teamwork is a work in progress as this is the first competition David, Austin, and Alex were in the driver box

Our next step is to follow our design process to redetermine our drivetrain configuration specifically what RPM’s we run at.

12/15/2025 - Drivetrain Brainstorm: Rpm

Meeting Goal: Access what rpm to have our drive train at.

Participants: Austin, Jenea

Different Rpm's a drive can run at:

450 rpm

450 Rpm drive train is a 25% speed reduction from a 600 rpm input using 36t gears into a 48t gear. Using this ratio allows for simplicity and is best for driver trains that require a balance between speed and torque.



Pros: Easy to build, high speed,
Cons: medium pushing force

480 rpm

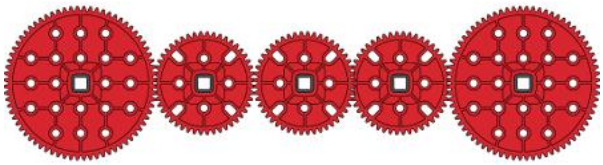
The 480 RPM drive train is a medium speed motor option. It offers a slightly higher speed than 450 RPM while still keeping reasonable torque. This prioritizes higher linear speed, which allows for the robot to quickly revel around the field and reposition during game play.



Pros: rapid field traversal, lower cycle times, improves maneuverability.
Cons: Reduced torque limits ,high speed makes precise alignment more difficult.

400 rpm

A 400 RPM drivetrain with 4" wheels is a high torque design. This drive train prioritize consistency, control, and mechanical reliability. Rather than maximum speed, whish makes it best for competitive match play and precision-based strategies.



Pros: Good balance between speed and torque, Strong acceleration and pushing power, easier driver control and precise positioning ,smoother handling
Cons: overheating, poor acceleration

Conclusions:

The 480 rpm design is best suited for our robot because it is light weight. We will have to develop a strategy for the alignment but that will come at a later point.

12/17/2025 - Drive Selection: RPM

Meeting Goal: Of the brainstormed drive types above, what is the best to use?

Participants: Austin, Alex, David

Parameters:

Speed

- We want our bot to be able to move across the field as quickly as possible so that we can increase our chances of scoring high. As well as this, it is important during 2v2 competitions to beat the opposing side when it comes to both getting to the goals first and blocking them from scoring.
- **Weighting:** This has the highest weight of all the parameters at 0.28 because it strongly affects our performance in both Skills and 2v2 competitions and our driver has requested our robot to be slightly faster.
- **Concept Scoring:**
  - ***450 RPM: 4/5 →***
  - ***480 RPM: 5/5 →***
  - ***400 RPM: 4/5 →***

Acceleration

- For our bot's top speed to matter, we want it to reach that top speed fairly quickly. Having a quicker acceleration helps the bot get to its top speed and move across the field in a short amount of time.
- **Weighting:** 0.26
- **Concept Scoring:**
  - ***450 RPM: 5/5 →***
  - ***480 RPM: 4/5 →***
  - ***400 RPM: 3/5 →***

Pushing Force

- We want to be able to program the drive fairly easily because it will leave more time for other tasks like driver practice and reduce the risk of issues arising. It will also give us more access to help if we select a commonly used drive type.
- **Weighting:** 0.24
- **Concept Scoring:**
  - ***450 RPM: 4/5 →***
  - ***480RPM: 5/5 →***
  - ***400 RPM: 4/5 →***

Complexity (building)

- We want a fairly simple drivetrain because this will take less time to build and leave more time for programming and driving. It also reduces the risk of building something wrong and simplifies repairs that happen at competitions.
- **Weighting:** 0.22

- **Concept Scoring:**
  - ***450 RPM: 5/5 →***
  - ***480 RPM: 5/5 →***
  - ***400 RPM: 5/5 →***

Decision Matrix:

	Weighting	450 RPM		480 RPM		400 RPM	
Speed	0.28	4	1.12	5	1.40	4	1.12
Acceleration	0.26	5	1.30	4	1.04	3	78
Pushing Force	0.24	4	0.96	4	0.96	4	0.96
Build Complexity	0.22	5	1.10	5	1.10	5	1.10
Totals:	1.00		**4.48**		**4.50**		**3.96**

Conclusion:

We will be moving forward with the 480Rpm drivetrain. This scored the highest weighted composite average due to its overall rounded out design, meaning it provides the most advantages for what we are looking for. The lowest score it received was a 4 in pushing force and acceleration, meaning that this parameter, while a little low, is still very satisfactory in its performance. The speed was the only one to score a 5, which is what our driver requested.

Next Steps:

Test and prepare for our next competition

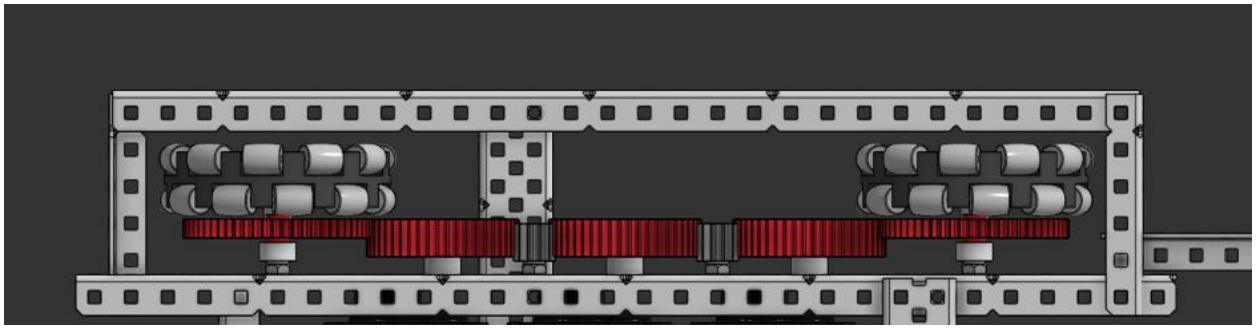
12/18/2025 - Drive RPM: Design plan

Meeting Goal: Create a CAD model of the drive concept.

Participants: Austin, David, Alex

Design Specifications:

- **RPM:** 480
- **Wheel configuration:** 2 omni
- **Wheel size:** 3.25 inch
- **Gear ratio:** 48:60
- **Motor configuration:** 6 11W, 3 per drive side



Additional info:

This drive train requires 12 teeth idlers to allow us to have the same spacing as our previous robots which worked quite well for us. The reason our drive train has become wider is because our team lacks the capability to half cut 48 tooth gears which would be required to have a 3 hole internal gap instead of 4.

12/19/2025 - Drive RPM: Testing

Meeting Goal: Does our drive perform the way we want it to?

Participants: Austin, David, Alex

Test 1 Procedure: Speed

1. Line robot up with 72 inches of space in front
2. Start stopwatch and push joysticks forward
3. Stop stopwatch when robot hits 72 inch mark
4. Divide 72 inches by time in seconds and record
5. Repeat steps 1-4 5 times.

Diagram:

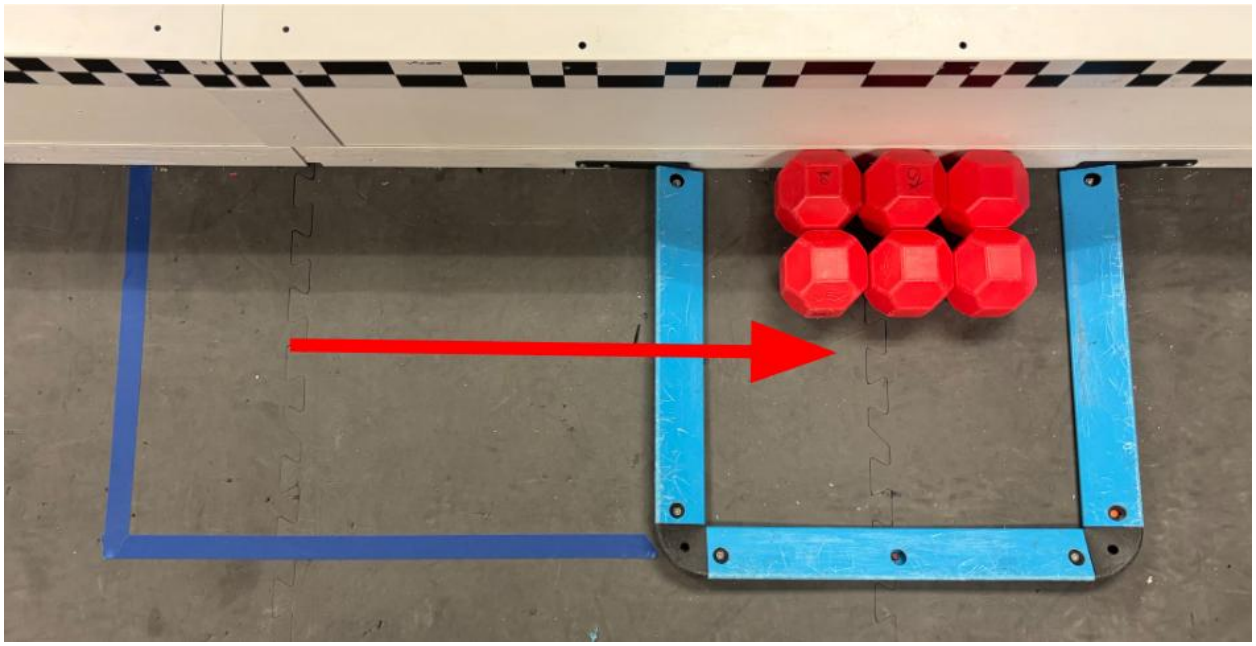


Trial 1	Trial 2	Trial 3	Trial 4	Trial 5	**Average**
105	91	102	93	90	**96.2 ms^-1**

Test 2 Procedure: Parking

1. Set our robot parallel with the parking zone
2. Start time and drive into the parking zone
3. Stop time when all blocks are cleared and the robot is parked.
4. Repeat 1-4 5 times

Diagram:



Trial 1	Trial 2	Trial 3	Trial 4	Trial 5	**Average**
1.2	.98	1.1	1.5	.83	**1.122 sec**

Conclusion:

Our new drivetrain is to our drivers satisfaction and is faster by 6.7%. The parking did seem harder without sleds but for now is to our satisfaction.

Next steps:

Tune based on our data then move into breaking down our program and test it for flaws moving into January.

12/20/2025 - Drivetrain: Tuning

Meeting Goal: Tune our robot to increase performance.

Participants: Austin, David, Alex

Observations:

The drivetrain we selected performed as expected. It met our standards for how it should perform in speed and acceleration. It does seem to struggle with getting our bot into the parking zone, however.

Problem Status:

Minor to none

Solutions:

Despite having a bit of trouble getting into the parking zone, it is not a probable reason from deconstructing our drivetrain and creating a new one. This problem should be fairly easy to fix by adding poly sleds to our bot.

Tests Re-Ran:

Since we will be adding polly sleds at a later point the tests will be done once we have brainstormed and selected a design.

12/21/2025 - Program: break down

Meeting Goal: Break down and explain our program.

Participants: Hamza, Andrew

Part 1: Setup

In C++, setting up the code is one of the most crucial parts of the program. Without defining every motor, every sensor, or every function, the robot will not do what it's told. In the beginning of our code, we utilize this by first identifying all the pieces connected to the brain, then inserting those values into calculations that allow the robot/chassis to function smoothly and as one.

Example: (Defines all external motors, sensors, and pneumatics)

```
using namespace vex;

extern brain Brain;

// VEXcode devices
extern motor Left1;
extern motor Left2;
extern motor Left3;
extern motor Right1;
extern motor Right2;
extern motor Right3;
extern motor highintake;
extern motor hood;
extern controller Controller1;
extern digital_out matchloader;
extern inertial Inertial21;
extern rotation RotationForward;
extern rotation RotationSideway;
extern digital_out wing;

/**
* Used to initialize code/tasks/devices added using tools in VEXcode Pro.
*
* This should be called at the start of your int main function.
*/
void  vexcodeInit( void );
```

Part 2: PID

PID (Proportional, Integral, Derivative) is a series of calculations that uses feedback to calculate the error in speed in regards to its setpoint. This is crucial for a successful and consistent autonomous, as it allows the robot to drive and turn smoothly, precisely, and quickly. This also prevents overshooting and oscillation. In our code, we integrate this into our code by stating the constants when the code starts, allowing the robot to use it throughout its run.

Example:

```
// Each constant set is in the form of (maxVoltage, kP, kI, kD, startI).
void auton_constant_testing(){

    chassis.set_turn_constants(12, 0.3, 0.02, 2.5, 15);
    chassis.set_drive_constants(8.5, 0.525, 0.02, 2.5, 0);
    chassis.set_heading_constants(11, .25, 0, 0, 0);

    chassis.set_swing_constants(12, .3, .001, 2, 15);
```

Part 3: Odometry

Odometry uses two rotational sensors (one forwards and one sideways) in order to accurately track its position on the field at all times. This makes it so that instead of making the robot run for a certain amount of time or rotations (both inconsistent over multiple runs), you can just tell the robot to turn and drive to a point. This makes coding both easier and more consistent, and streamlines the process of coding the program. (This can be seen throughout the code in the next part).

Part 4: The Code

Once everything is set up, PID is tuned, and odometry is functioning, only then are we able to start telling the robot what to do. Over the course of a few days, we are able to use trial and error to result in a code that works consistently and scores well.

Example: (2v2 Code for Onalaska)

```
void auton_right(){

 //  Set co ords, Wing Set Up
 chassis.set_coordinates(0, 0, 0);
 wing.set(false);

 //  Drive and collect middle 3
 chassis.drive_distance(5);

 setVoltage(4);
 chassis.turn_to_point(16, 24);
 highintake.spin(forward, 100, percent);
 chassis.drive_to_point(16, 24);
 setVoltage(8.5);
```

```
  wait(0.5, seconds);

//  line up long goal
chassis.turn_to_point(42.5, 0);
highintake.stop();
chassis.drive_to_point(42.5, 0);

chassis.turn_to_angle(180);

//  Score long goal
setVoltage(5);
driveWithShortExitTime(-25, 1000);
chassis.drive_distance(1);
setVoltage(8.5);

highintake.spin(reverse, 100, percent);
hood.spin(reverse, 100, percent);

  wait(0.25, seconds);

highintake.stop();
hood.stop();

wait(0.1, seconds);

highintake.spin(forward, 100, percent);
hood.spin(forward, 100, percent);

  wait(2, seconds);

highintake.stop();
hood.stop();
}
```

12/22/2025 - Program: Testing

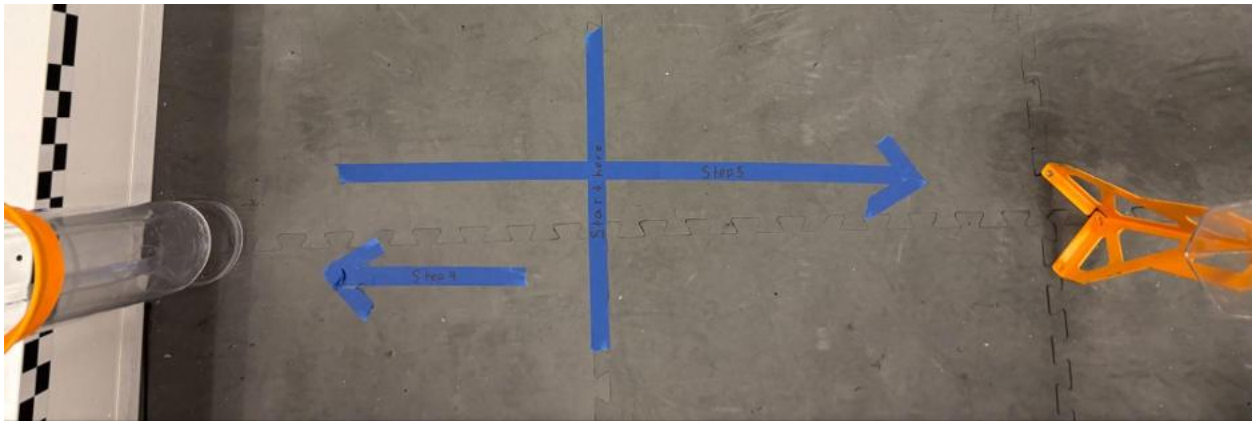
Meeting Goal: Does our program function the way we want it to?

Participants: Hamza, Andrew

Test 1 Procedure: Match imitation

1. Load matchtube
2. Line up robot one tile in front of the tube
3. Start timer
4. drive up to tube and clear
5. Move forward to long goal and score
6. Stop timer once all game pieces are scored

Diagram:

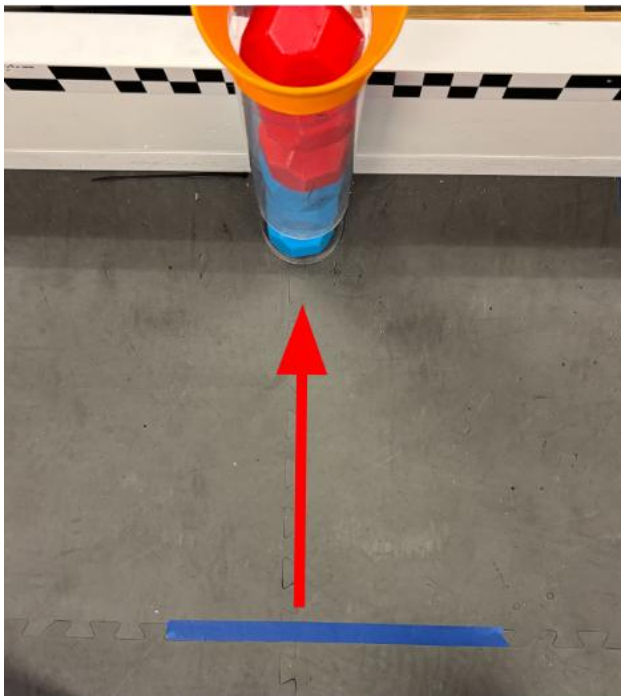


Trial 1	Trial 2	Trial 3	Trial 4	Trial 5	**Average**
10s	9.3s	9.5s	8.4s	10.2s	**9.48s**

Test 2 Procedure: Clearing the tube

1. Fill up matchload tube
2. Line up robot one tile from tube
3. Lower match loader
4. Begin timer
5. Drive into the tube
6. Stop timer once tube is cleared

Diagram:



Trial 1	Trial 2	Trial 3	Trial 4	Trial 5	**Average**
2.1s	3s	2.6s	2.7s	2.2s	**2.52s**

Conclusion:

Our program is able to successfully and consistently matchload and score with enough speed. While there is still room for minor improvements with the occasional jam in the intake, the code itself works to the best of its ability.

12/23/2025 - Programing: Tuning

Meeting Goal: Tune our program to increase performance.

Participants: Austin, David, Alex

Observations:

Our program has some minor issues when it comes to the intake. It occasionally gets jammed when it takes up the blocks. This will cost us points, but the rest of the program is able to perform consistently with only slight changes in position.

Problem Status:

Medium Low

Solutions:

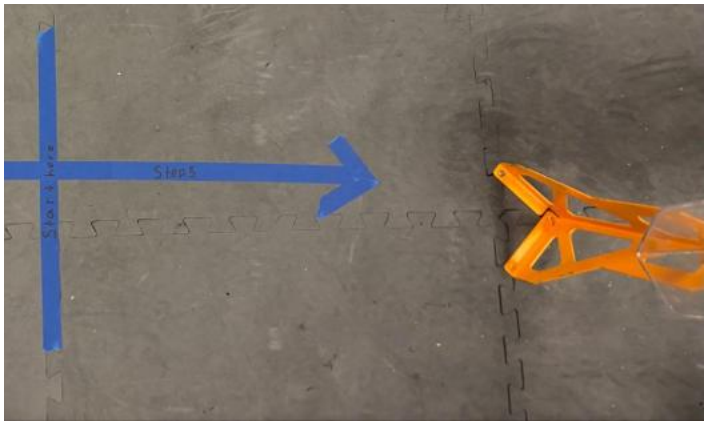
Since we would like to get as many points as possible, we will most likely find a way to prevent further jams. The simplest and most effective solution is to make the intake reverse for a few moments during intake. We will further observe when it typically jams in order to determine when we should program the reverse.

Tests:

Test 1 Procedure: Scoring Without Anti-Jam

1. Load 7 balls into robot
2. Line up robot one tile in front of the long goal
3. Start timer
4. Drive up to goal and score
5. Stop timer once all game pieces are scored (fail if it jams and doesn't score)

Diagram:



Trial 1	Trial 2	Trial 3	Trial 4	Trial 5	**Average**
2s	Fail	2.3s	3.1s	Fail	**2.5s (when successful)**

Test 2 Procedure: Scoring With Anti-Jam

1. Load 7 balls into robot
2. Line up robot one tile in front of the long goal
3. Start timer
4. Drive up to goal and score
5. Stop timer once all game pieces are scored

Trial 1	Trial 2	Trial 3	Trial 4	Trial 5	**Average**
2.2s	3s	2.6s	2.7s	3.1s	**2.72s**

Conclusion:

With the addition of the anti-jam, it is able to score all the blocks much more consistently. While the reverse does slow down the code slightly as demonstrated by the tests, the trade-off is worth it and should allow us to score every time.

1/07/2026 - January Plans

Meeting Goal: Plan what to do for the upcoming month.

Participants: All

Team Decisions:

→ Develop mid goal scoring
→ Attend 3 competitions

Team Build Accomplishments:

→ Switched to 480 rpm drive
→ lifted horizontal odom

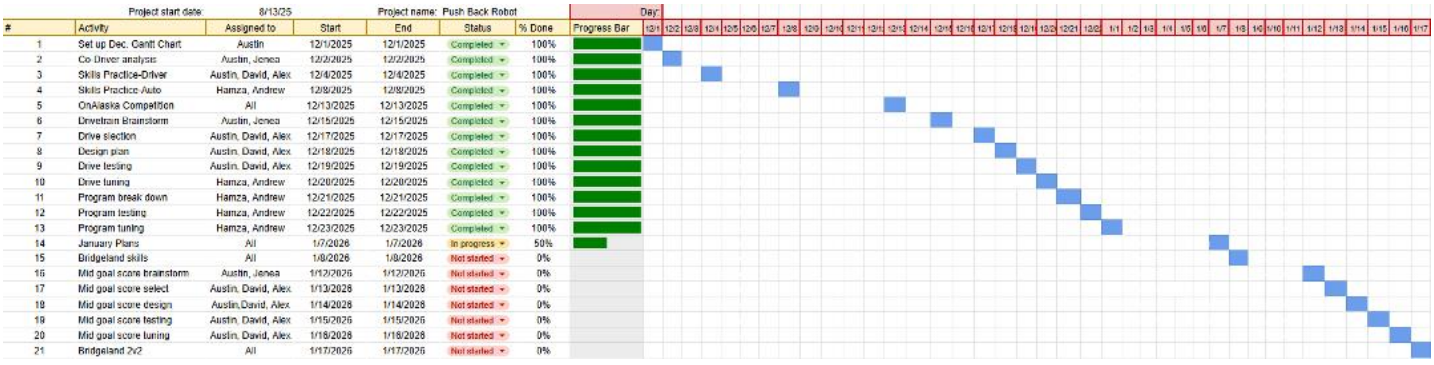
Team Programming Accomplishments:

→ Successful implementation of odom
→ Successful and consistent 2v2 code
→ Broke down the code

Team Goals:

→ Add mid goal scoring
→ Add a visual sensors for auton
→ Get 70 auton for skills

Gantt chart progress:



	Project start date:	8/13/25		Project name:	Push Back Robot		Day:
#	Activity	Assigned to	Start	End	Status	% Done	Progress Bar
1	Set up Dec. Gantt Chart	Austin	12/1/2025	12/1/2025	Completed	100%	
2	Co-Driver analysis	Austin, Jenea	12/2/2025	12/2/2025	Completed	100%	
3	Skills Practice-Driver	Austin, David, Alex	12/4/2025	12/4/2025	Completed	100%	
4	Skills Practice-Auto	Hamza, Andrew	12/8/2025	12/8/2025	Completed	100%	
5	OnAlaska Competition	All	12/13/2025	12/13/2025	Completed	100%	
6	Drivetrain Brainstorm	Austin, Jenea	12/15/2025	12/15/2025	Completed	100%	
7	Drive slection	Austin, David, Alex	12/17/2025	12/17/2025	Completed	100%	
8	Design plan	Austin, David, Alex	12/18/2025	12/18/2025	Completed	100%	
9	Drive testing	Austin, David, Alex	12/19/2025	12/19/2025	Completed	100%	
10	Drive tuning	Hamza, Andrew	12/20/2025	12/20/2025	Completed	100%	
11	Program break down	Hamza, Andrew	12/21/2025	12/21/2025	Completed	100%	
12	Program testing	Hamza, Andrew	12/22/2025	12/22/2025	Completed	100%	
13	Program tuning	Hamza, Andrew	12/23/2025	12/23/2025	Completed	100%	
14	January Plans	All	1/7/2026	1/7/2026	In progress	50%	
15	Bridgeland skills	All	1/8/2026	1/8/2026	Not started	0%	
16	Mid goal score brainstorm	Austin, Jenea	1/12/2026	1/12/2026	Not started	0%	
17	Mid goal score select	Austin, David, Alex	1/13/2026	1/13/2026	Not started	0%	
18	Mid goal score design	Austin,David, Alex	1/14/2026	1/14/2026	Not started	0%	
19	Mid goal score testing	Austin, David, Alex	1/15/2026	1/15/2026	Not started	0%	
20	Mid goal score tuning	Austin, David, Alex	1/16/2026	1/16/2026	Not started	0%	
21	Bridgeland 2v2	All	1/17/2026	1/17/2026	Not started	0%	

Great progress has been made and we've been working hard to stay on schedule and take advantage every day we have to work.

1/8/2026 - Bridgeland Skills

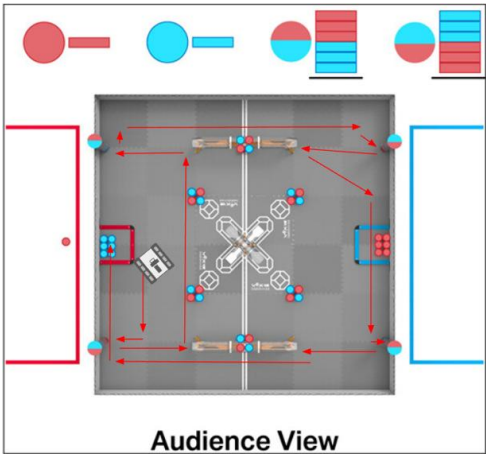
Meeting Goal: How did we do at the competition?

Participants: All

Competition goals:

Our goal for the competition is to win the competition as a whole and achieve a total score of 117. The goal driver score to hit is 80 while autonomous is 37. If our route hits 80 we will try a new route for 85 driver.

Routes for driver:



Placement:

Rank	Team	Driver Attempts	Driver Highscore	Programming Attempts	Programming Highscore	Total Highscore
1	22603B	3	80	3	51	131
2	22603A	3	58	3	8	66
3	77786S	3	34	3	12	46

Problems:

Intake lacked consistency especially when match loading. Funnels weren't consistent causing us to lose time. The lack of midgoal scoring caused our score to be capped at 80. Parking and clearing both park zones was a struggle due to our lack of parking sleds.

Future plans:

Add a mid goal scoring mechanism. Continually tune intake to where it can work consistently on matchloader and on field game elements. Cad out and laser cut parking sleds out of poly carbonate for a smoother parking cross and clear.

1/12/2026 - Mid goal score: Brainstorm

Meeting Goal: Access what options for mid goal scoring we can do.
Participants: Austin, Jenea

Types of mid goal scoring:

Starting information:

Our robot is already developed so brainstorming mid goal scoring for other types of robots would go against our agenda as a team. This brainstorming is a way for our robot to have mid goal scoring capabilities without changing the robot too drastically.

String

A flexible material used to transfer motion or assist in lifting. With a strong moddle goal scoring mechanism it allows for it to be passive without the use of pistons. It pushes into the goal alone and the bend in the strong pull down on the rubber band allows the block to move into the middle goal.

Pros: Cheap,simple
Cons: slow, easily defendable, extremely difficult to tune

Rubber band

Elastic material used for passive tension, returning mechanisms, or assisting movement.
With the use of a rubber band middle goal scoring mechanism it allows for active scoring with the use of a position. The rubber band attached to the piston pulls down in an elevator motion. This allows for a fast way to transfer the blocks through the robot at a lowering point for middle goal scoring.

Pros: Cheap, rapid scoring, can use a smaller piston, easy to tune
Cons: requires air pressure, band can stretch / break easily

Ziptie

A plastic faster for holding components. By using a zip tie based middle goal scoring mechanism it allows for active scoring. This method requires a zip tie to be connected to the piston and move in an elevator motion. This then allows for the band to move down and score blocks.

Pros: Cheap, durable, simple, easy to tune, fast, strong pushing force
Cons: requires air pressure

Conclusions: The methods available to us are very cheap and simple to set up.

1/13/2026 - Mid goal score: Selection

Meeting Goal: Of the brainstormed mid goal scoring types above, what is the best to use?
Participants: Austin, David, Alex

Parameters:

Speed

- We want our scoring to be able to quickly score without the chance of defending. This includes fast activation.
- **Weighting:** This is scored as our second highest at 0.26, as our robot is very speed based and our mid goal is not different. Being able to activate and score quickly can be the difference in a match win and lose.
- **Concept Scoring:**
  - ***String: 2/5*** → *The slowest of the options as it required pressure of driving into the goal to open. This would make it really easy to defend against.*
  - ***Rubberband: 3/5*** → *Almost as fast as a zip tie but due to the factor of the stretch it could take longer to activate despite the piston.*
  - ***Zip Tie: 4/5*** → *The fastest of the solutions as the zip tie is solid and wouldn't be able to stretch like the other two.*

Complexity (building)

- We want a fairly simple mid goal score because this will take less time to build and leave more time for programming and testing. It also reduces the risk of building something wrong and simplifies repairs that happen at competitions.
- **Weighting:** This is our 3rd highest priority at 0.24, because we want a mid goal score that is simple to build, since as long as we have enough time we can figure out how to build any type of solution.
- **Concept Scoring:**
  - ***String: 3/5*** → *This solution would require our team to design a way to make the required pressure to move the string.*
  - ***Rubberband: 4/5*** → *We would use a piston located near our odom to activate but the amount of rubberband tension would make it a bit more complicated to the zip tie idea.*
  - ***Zip Tie: 5/5*** → *This solution will also use a piston attached near our odom but is simpler to attach as tension wouldn't need to be considered.*

Tuning

- Due to us adding this to our current robot tuning will be needed to allow it to get the desirable result without the need of rebuilding.
- **Weighting:** This is our lowest priority, at 0.22, because yes tuning will be needed but isn't our top priority at our current position.

- **Concept Scoring:**
  - ***String: 2/5*** → *Would be very hard to tune.*
  - ***Rubberband: 4/5*** → *Tension would have to be factored in but otherwise easy to tune.*
  - ***Zip Tie: 5/5*** → *Easy to tune as only the location of the piston would be needed to be changed.*

Durability

- We want a solution that will hold up in competition as mid goal is starting to become a valuable part of our strategy moving forward.
- **Weighting:** This has the highest weight of all the parameters, at 0.28, because it affects the performance at competition. If the mid goal scoring mechanism breaks at competition and we don't have a replacement or in the middle of a match it can cause issues especially on the programming side of things.
- **Concept Scoring:**
  - ***String: 4/5*** → *Pretty durable but the string can start to break after repetitive use.*
  - ***Rubberband: 2/5*** → *Prone to breaking which can cause many issues at competition.*
  - ***Zip Tie: 5/5*** → *Very durable and wouldn't need to worry about it for a while.*

Decision Matrix:

	Weighting	String		Rubberband		Zip tie	
Speed	0.26	2	0.52	3	0.78	4	1.04
Build Complexity	0.24	3	0.72	4	0.96	5	1.20
Tuning	0.22	2	0.44	4	0.88	5	1.10
Durability	0.28	4	1.12	2	0.56	5	1.40
Totals:	1.00		**2.8**		**3.18**		**4.74**

Conclusion:

We will move forward with the zip tie solution as it offered the highest score in all categories and would hold up at competition. Though these solutions are fairly similar the material makes a huge difference in results as seen when we weighed them against each other.

Next Steps:

Create a design for our concept. Then move into testing and competition prep.

1/14/2026 - Mid goal score: Design plan

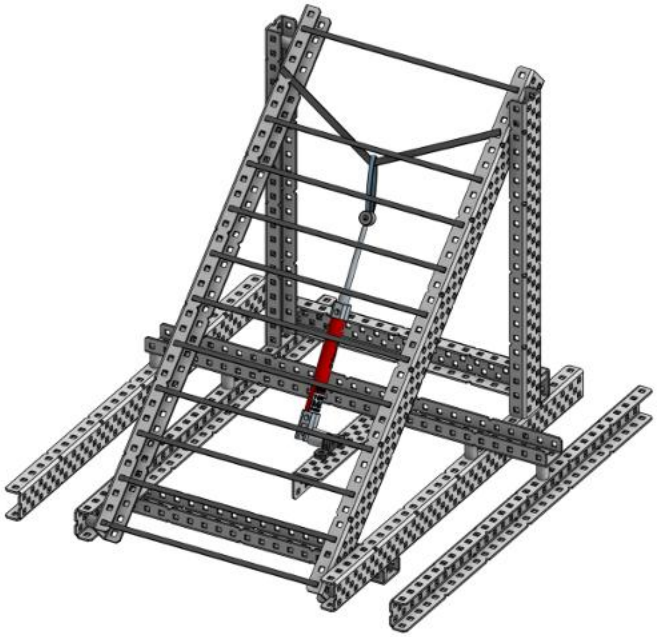
Meeting Goal: Create a cad model of the mid goal scoring concept.

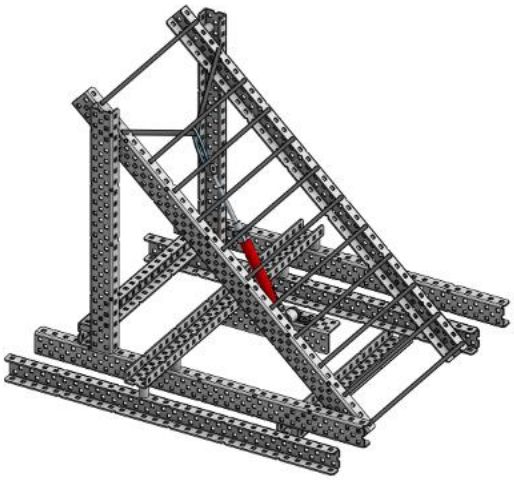
Participants: Austin, David, Alex

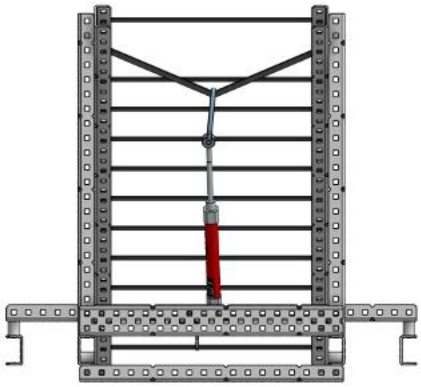
Design Specifications:

- 66mm piston
- Small ziptie
- Shaft Collar

Cad:







Additional info:

For our middle goal scoring the piston is located on top of our linear odom metal mount. Which allows us to save weight and have simplicity. It runs under our elevator which makes it all easily accessible for maintenance and tuning. The 60mm piston is mounted on a screw joint which allows it to pivot if needed and is also extremely simple.

1/15/2026 - Mid goal score: Testing

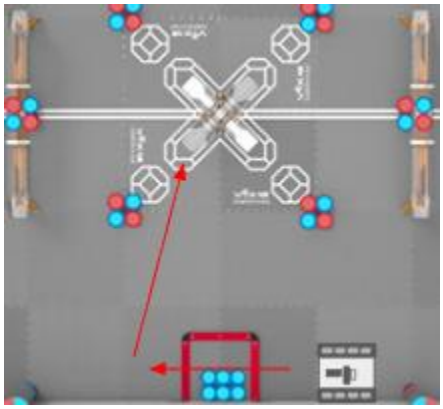
Meeting Goal: Does our mid goal scoring perform the way we want it to?

Participants: Austin, David, Alex

Test 1 Procedure: Skills imitation

1. Set up skills park zone blocks
2. Give robot a preload
3. Line up robot one tile to the left of the parkzone
4. Start timer
5. drive over to parkzone
6. Move forward and start intake
7. Once all blocks are intaked move towards mid goal
8. Score into mid goal
9. Stop timer once all 7 are scored

Diagram:



Trial 1	Trial 2	Trial 3	Trial 4	Trial 5	**Average**
10.47	9.10	10.93	8.89	9.48	**9.774 sec**

Test 2 Procedure: Speed of outtake

1. Fill up robot with game pieces
2. Activate mid goal score
3. Begin timer
4. Start outtake
5. Stop timer once robot is cleared

Trial 1	Trial 2	Trial 3	Trial 4	Trial 5	**Average**
3.34	3.22	3.58	3.39	3.28	**3.362 sec**

Test 3 Procedure: Capacity

6. Prepare a group of game pieces
7. Start intaking until robot is full
8. Activate mid goal
9. Outtake and check for jamming
10. Repeat until max amount of blocks is found

Using abbreviation DNO for did not outtake and SO for successful outtake.

Trial 1	Trial 2	Trial 3	Trial 4	Trial 5	**Max**
14 DNO	12 DNO	10 DNO	8 DNO	7 SO	**7 blocks**

Conclusion:

Paired with our new drivetrain our robot was able to finish the test at a very fast speed. This speed gives hope for the future of both skills and 2v2 as a good mid goal can make a solo win point for 2v2 and drastically improve a skills score. Our max capacity is halved but enough for mid goal control in skills. We will put it all to the test at our next competition, the bridgeland 2v2, skills.

1/16/2026 - Mid goal: Tuning

Meeting Goal: Tune our robot to increase performance.

Participants: Austin, David, Alex

Observations:

During its performance, it was able to carry out its purpose effectively and was quick to deploy. The only issue that was caused by its installment is that it has limited our bots max capacity when in use.

Problem Status:

Minor

Solutions:

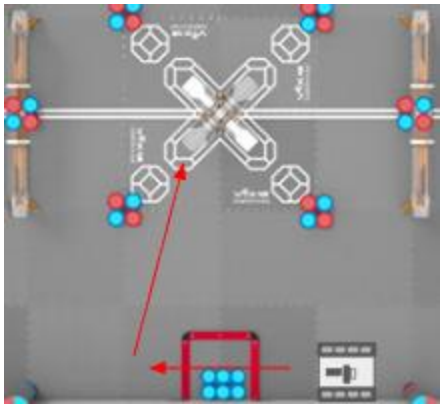
Due to the fact that the midscore must be in the middle of our bot, halving its capacity for this build is inevitable and unavoidable. To attempt to fix this problem we adjusted our funneling within the robot. In the future, we will have a midscore more integrated into our build to hopefully remedy this.

Tests Re-Ran:

Test 1 Procedure: Skills imitation

1. Set up skills park zone blocks
2. Give robot a preload
3. Line up robot one tile to the left of the parkzone
4. Start timer
5. drive over to parkzone
6. Move forward and start intake
7. Once all blocks are intaked move towards mid goal
8. Score into mid goal
9. Stop timer once all 7 are scored

Diagram:



Trial 1	Trial 2	Trial 3	Trial 4	Trial 5	**Average**
9.73	9.10	10.06	8.93	8.79	**9.322 sec**

Test 2 Procedure: Speed of outtake

1. Fill up robot with game pieces
2. Activate mid goal score
3. Begin timer
4. Start outtake
5. Stop timer once robot is cleared

Trial 1	Trial 2	Trial 3	Trial 4	Trial 5	**Average**
3.17	3.25	3.46	3.29	3.51	**3.336 sec**

Conclusions:

With the improvements in our funneling our outtaking speed has improved. The capacity did not improve but the improvements in speed makes up for the lack of capacity.

1/17/2026 - Bridgeland 2v2, Skills

Meeting Goal: How did we do at the competition?

Participants: All

Goals for competition per category:

Skills tournament:

Our goal for the skills aspect of the competition was to push for 80 driver and try to get a solid auton score despite the time restraint set on our programmers.

2v2 tournament:

The goal is always to win but another goal is to test out our new drivetrain in a competitive environment. Another goal is to run consistent 2v2 autos. With the swap of drivetrains caused for an overall code overhaul with the numbers used.

What went wrong:

At the competition our team faced some challenges:

- With our team swapping to 480 drive we had to update our code for skills which with the short notice of the competition made it hard to make a viable code.
- Due to the lack of notice of the competition we did not have enough time to swap to fresh wheels so crossing over park zones were difficult without sleds. Our wheels also started to break which led to the struggle.
- Minor issue with setting our odom the right way in one round that led to an auto bonus tie.
- During the quarterfinals another robot rammed into our robot and hit our battery cord causing our robot to white screen.

Overall the competition didn't have many setbacks. Any issues are either minor or an easy fix.

What went well:

At the competition our team faced many positives

- The color corresponding commands were a success because it gave quick and short instructions for the driver to accomplish. This limited our opponents ability to be able to hear our call outs.
- The 2v2 aspect of our competition was better than we expected because we had very good and coordinated teamwork amongst our team. Since we were a coordinated team, our chances of getting forced error was very low. Everyone had their role which allowed the competition to be less of a stressful day.
- Inside the box during the 2v2, we adapted to our teammates very efficiently. First, we asked for how their robot performs, and then correspondingly adapted to the best

strategy to win the round. This allowed us for a positive win streak and allowed us to end the day with a 6-0in the qualification rounds.

Placement:

8th overall in the 2v2 tournament

Awards:

Design award

Conclusions:

This competition went alot better than the onalaska comp as in quals we went 6-0 and only got knocked out in quarter finals due to a white screen error on our brain due to a batter issue. We are quite proud of this competition as we saw competitive success and got awarded the design award.

Next steps and notes to improve:

Autonomous:

- Work on the matchloader for better consistency during prog.
- Spend more time on the program

Robot issues:

- We plan to start designing parking sleds to help get into the park zones easier and protect our gears.
- With the white screen being a scary possibility that can happen we will be moving where we have our battery mounted to not risk this happening again.
- Our intake had some jamming issues so we are going to try new solutions.

Teamwork has massively improved, with David, Austin, and Andrew in the driver box. The color callouts massively helped with getting an advantage.

1/20/2026 - Poly sleds: brainstorming

Meeting Goal: Access what options for parking sleds we can make.

Participants: Austin, Jenea

Different Shapes that can be used:

Iteration 1

This poly sled design utilizes a rectangular shape with one beveled corner and is a base for the other iterations. With this design, the height of it makes the bottom half exposed and not attached to the drive directly, unlike the top half. Its beveled corner is relatively short in length, allowing the bot to ramp up into the park zone. However, it might not be enough of a boost for the bot to lift itself into the park zone.



Pros: smaller slope to avoid sliding back, broadness of height could help the bot move into the park zone by giving it an extra ledge to push itself up with
Cons: Is more susceptible to being caught on things, higher possibility of cracks and breaks on bottom

Iteration 2

w

Iteration 2 is an average between Iteration 1 and Iteration 3 by having more structural integrity in its shorter design like Iteration 3 and by having a less jarring slope like iteration1 that will be more effective in allowing the bot to lift itself into the park zone. However, compared to Iteration 1, the slope is slightly longer, allowing the bot to have a more steady incline to rely on. As well as this, the sloped edge will be more structurally sound than that of Iteration 3 because it has more support from the c-channel of the drive that it is attached to.

Pros: Overall positive average of the other iterations, has a medium length slope on a steady incline the bot can rely on, structurally sound
Cons: Does not ease the bot into the park zone, possibly too small for any significant impact

Iteration 3



Iteration 3 is the simplest of the three designs with only four sides. It uses up less polycarbonate than the other two due to its slimmer shape and long slope. This simplicity has its perks by decreasing the amount of damage it might take on its bottom side and by having less obstruction. However, the sudden sharp point that it comes to can be easily broken off when trying to park, especially since it will most likely be sticking out without any supports attached to it. As well as this, the steeper slope that it has might be a vice. While it will help the bot begin to lift up into the park zone, it has room for error and can slide backwards due to the slop being so long.

Pros: Uses less polycarbonate to make, has a slimmer base for structural integrity
Cons: longer slope might cause slide back, pointed edge will be susceptible to cracks or breaking

Conclusions:

All iterations are fairly simple to accomplish and only very slightly in functionality.

1/21/2026 - Poly Sleds: Selection

Meeting Goal: Of the brainstormed poly sled types above, what is the best to use?

Participants: Austin, David, Alex

Parameters:

Efficiency

- We want our sleds to have a good angle for getting in and over the barrier.
- **Weighting:** This is scored at 0.26 because we want our solution to work. It doesn't do us any good if it doesn't work.
- **Concept Scoring:**
  - ***Iteration 1: 3/5*** → *This solution has the smallest angle. It does not extend as far as the others, though it does still cover the 1in barrier.*
  - ***Iteration 2: 4/5*** → *This solution is a little larger than option 1, but still smaller than option 3.*
  - ***Iteration 3: 5/5*** → *This option has the largest angle span.*

Cutting Complexity

- The fewer cuts that have to be made in the plastic, the easier it is to make. This also accounts for how much usable plastic is left in the sheet.
- **Weighting:** This is rated at 0.22, because it is fairly unimportant as we have a lot of available plastic and these pieces are not super large.
- **Concept Scoring:**
  - ***Iteration 1: 4/5*** → *This option uses the most plastic area and has an additional set of connection holes that need to be cut out. It also has a bevel, meaning more changes in direction are needed since there are more sides.*
  - ***Iteration 2: 5/5*** → *This option has the bevel like option 1, but doesn't have the extra holes.*
  - ***Iteration 3: 5/5*** → *This option only has one set of holes and no bevel.*

Tuning

- Due to us adding this to our current robot tuning will be needed to allow it to get the desirable result without the need of rebuilding.
- **Weighting:** This is ranked at 0.23 because we want to have flexibility in where we connect the plastic piece in case it doesn't work as intended.
- **Concept Scoring:**
  - ***Iteration 1: 5/5*** → *This option has 2 mounting holes, which means we have more options for height in terms of where we connect the sleds to the drive.*
  - ***Iteration 2: 3/5*** → *This option does not have much room for flexibility.*
  - ***Iteration 3: 4/5*** → *The angle this option gives us allows for flexibility because we can move it up or down and still have contact on the barrier because of the extension of the angle.*

Design Strength

- We want a solution that will hold up in competition because the sleds will be exposed and won't do us any good if they break during a match.
- **Weighting:** This has the highest weight of all the parameters, at 0.28, because it affects the performance at competition. If the sleds break at competition and we don't have a replacement or in the middle of a match it can cause issues.
- **Concept Scoring:**
  - ***Iteration 1: 5/5*** → *Double mounting holes provide more security holding them, and the bevel helps prevent breaking because there are no sharp points.*
  - ***Iteration 2: 3/5*** → *This option only has 1 mounting hole set, but still has the bevel.*
  - ***Iteration 3: 2/5*** → *This option has a sharp point as well as only 1 mounting hole set.*

Decision Matrix:

	Weighting	Option 1		Option 2		Option 3	
Efficiency	0.26	3	0.78	4	1.04	5	1.30
Cutting Complexity	0.22	4	0.88	5	1.10	5	1.10
Tuning	0.24	5	1.20	3	0.72	4	0.96
Design Strength	0.28	5	1.40	3	0.84	2	0.56
Totals:	1.00		**4.26**		**3.70**		**3.92**

Conclusion:

We will move forward with the first option because it provides the best compromise between all of our parameters. Its lowest score is a 3/5, which indicates no high cause for concern. We will cut the sled shape out of poly and attach it to our robot. We do not need to plan anything further because we already have the shape from the brainstorming above.

1/22/2026 - Poly Sleds: Testing

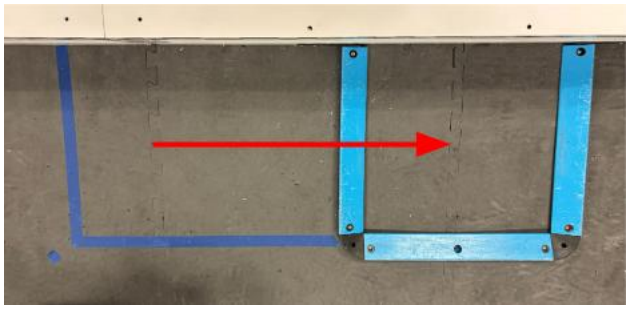
Meeting Goal: Do our poly sleds perform the way we want it to?

Participants: Austin, David, Alex

Test 1 Procedure: Parking

1. Line up robot one tile from parking zone
2. Start timer
3. drive up to park zone and park
4. Stop timer once robot is parked

Diagram:

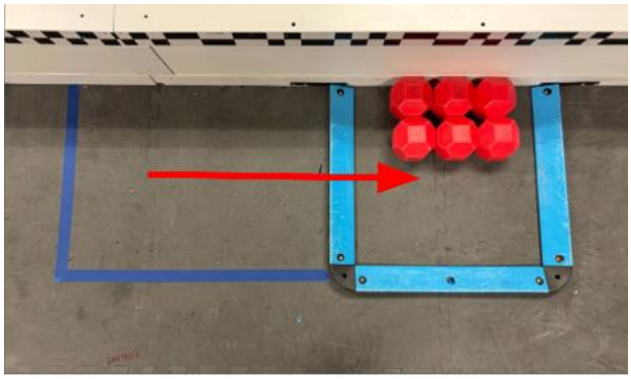


Trial 1	Trial 2	Trial 3	Trial 4	Trial 5	**Average**
1.19s	1.81s	0.73s	0.66s	1.24s	**1.13s**

Test 2 Procedure: Clearing the park zone

1. Set up park zone for skills
2. Line up robot one tile from park zone
3. Begin timer
4. Drive into the park zone
5. Stop timer once park zone is cleared

Diagram:

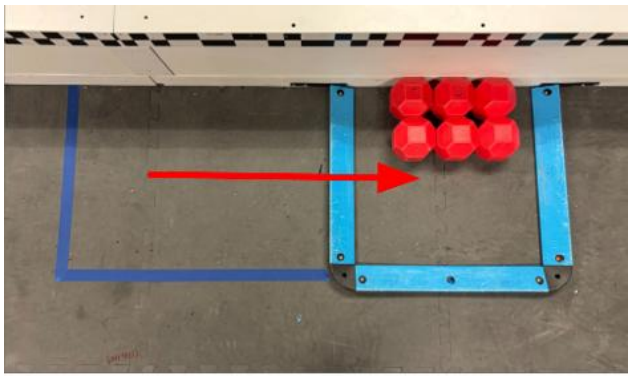


Trial 1	Trial 2	Trial 3	Trial 4	Trial 5	**Average**
4.56s	1.19s	2.03s	1.74s	6.35s	**3.17s**

Test 3 Procedure: Clearing the park zone with ML

1. Set up park zone for skills
2. Line up robot one tile from park zone
3. Lower match loader
4. Begin timer
5. Drive into the park zone
6. Stop timer once park zone is cleared

Diagram:

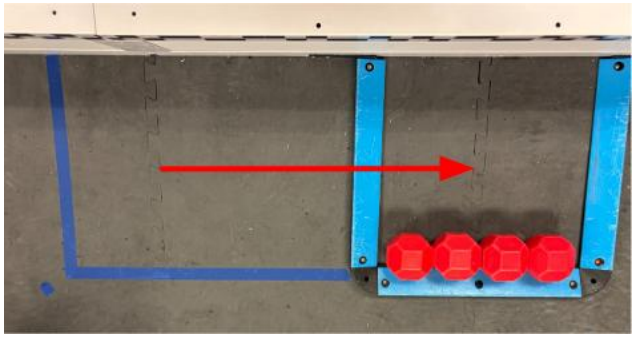


Trial 1	Trial 2	Trial 3	Trial 4	Trial 5	**Average**
1.19s	2.44s	2.66s	2.62s	2.72s	**2.33s**

Test 4 Procedure: Clearing the park zone (2v2)

1. Set up park zone for 2v2
2. Line up robot one tile from park zone
3. Begin timer
4. Drive into the park zone
5. Stop timer once park zone is cleared

Diagram:

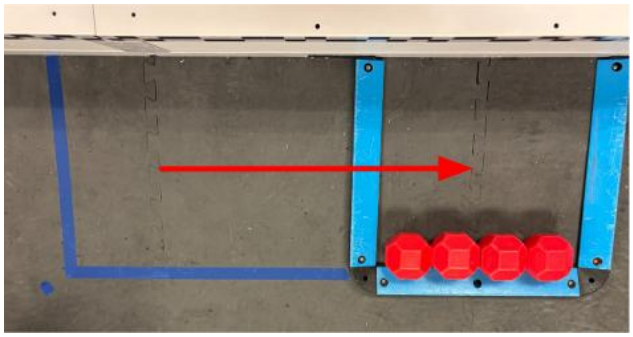


Trial 1	Trial 2	Trial 3	Trial 4	Trial 5	**Average**
5.14s	1.05s	3.32s	1.94s	1.83s	**2.656s**

Test 5 Procedure: Clearing the park zone with ML (2v2)

1. Set up park zone for 2v2
2. Line up robot one tile from park zone
3. Lower match loader
4. Begin timer
5. Drive into the park zone
6. Stop timer once park zone is cleared

Diagram:



Trial 1	Trial 2	Trial 3	Trial 4	Trial 5	**Average**
2.17s	2.27s	2.46s	1.19s	2.53s	**2.124s**

Conclusion:

We found that our parking sleds are satisfactory when testing. We noticed that when the matchloader is down clearing the park zone became a lot easier as it was able to push the blocks and not get caught underneath our robot. Now that we can clear the park zones effectively we are going to shift our focus on fixing our autonomous skills program.

Next steps:

Testing and fixing the issues with our autonomous program for both 2v2 and skills. Our starting focus will be the skills program.

1/22/2026 - Poly Sleds: Tuning

Meeting Goal: Tune our poly sleds to increase performance.

Participants: Austin, David, Alex

Observations:

The poly sleds do their job at helping the bot ramp itself up and into the parking zones that is satisfactory for our time. It does sometimes, however, take a couple of tries before the bot is in the parking zone depending on how the bot approaches the park zone and what angle it's at.

Problem Status:

Minor

Solutions:

We believe that this is the best it will be for now and our current version of our bot. In the future, we may look into some more varied designs that might possibly be more effective.

Tests Re-Ran:

Due to not changing the design at this time there were no re-ran tests.

1/24/2026 - Autonomous Skills: Testing

Meeting Goal: Find out why our autonomous hasn't hit for skills at competition.

Participants: Hamza, Andrew

Observations:

The autonomous period in skills has failed in the last few competitions. The alignments miss, it's unable to clear matchloaders, and unable to park. The code would do something different every run making any form of consistency impossible.

Problem Status:

Major

Solutions:

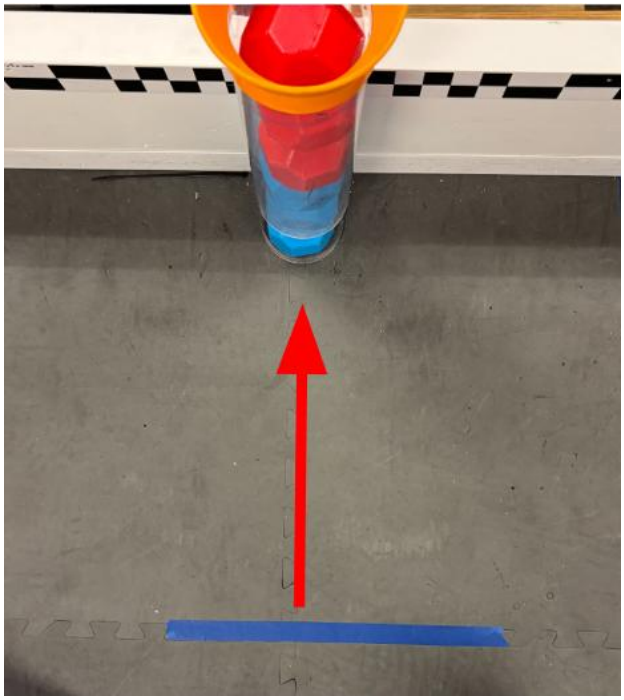
We believe that this is due to an IMU and Odom drift, in which the reading for turning degrees gradually shifts throughout the code and the rotation sensors don't accurately track the position of the robot. Some solutions include replacing the sensors, adding a distance sensor to reset the sensors throughout a run, and further tuning the robot for better runs.

Tests:

Test 1 Procedure: Clearing the tube

1. Fill up matchload tube
2. Line up robot one tile from tube
3. Lower match loader
4. Begin timer
5. Drive into the tube
6. Stop timer once tube is cleared

Diagram:



Trial 1	Trial 2	Trial 3	Trial 4	Trial 5	**Average**
2.2s	2.9s	2.6s	2.6s	2.3s	**2.52s**

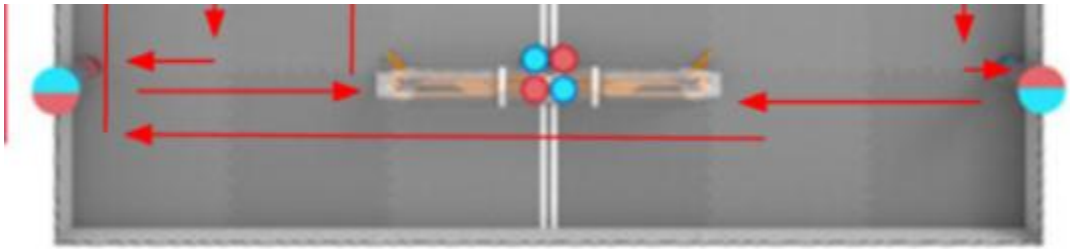
Conclusion:

Our program is able to successfully and consistently matchload with enough speed. This means that the code was likely failing in alignment instead of matchloading capability, as on its own it worked consistently. This supports one of the likely problems highlighted earlier.

Test 2 Procedure: Alignment. Skills Imitation

1. Clear first tube
2. Drive to opposite side of Long Goal
3. Score
4. Drive to second tube
5. Clear
6. Drive back to Long Goal
7. Score

Diagram:



Trial 1	Trial 2	Trial 3	Trial 4	Trial 5	**Average**
Fail	Fail	Fail	Fail	Fail	**Fail**

Problem:

This section alone was completely unable to hit. However, the test did reveal the problem in the IMU and odometry. The IMU was faulty, drifting as the code ran, changing the start reference angle of 0 making its turns inaccurate. This could be seen with the heading reading displayed on the brain while the code is running. In addition, the wheels attached to the rotation sensors were skipping, meaning there wasn't enough tension pushing the wheel down and not allowing for the sensor to track the robot's movement.

Test 2 Rerun: After fixing issues

Trial 1	Trial 2	Trial 3	Trial 4	Trial 5	**Result**
12 Scored, 2 Cleared Matchloaders	11 Scored, 1 Cleared Matchloaders	10 Scored, 1 Cleared Matchloaders	12 Scored, 2 Cleared Matchloaders	12 Scored, 2 Cleared Matchloaders	**Success**

Conclusion:

After replacing the IMU and fixing the odometry, the code was able to run much more smoothly with alignment hitting every time. This is a major development for code as it allows us to hit a higher skills score and gives a consistent platform to code with.

1/27/2026 - Autonomous 2v2: Improvements

Meeting Goal: Tune our autonomous program to be faster.

Participants: Austin, David, Alex

Observations:

The autonomous period in 2v2, while hitting extremely consistently, is slower than optimal as our strategy relies on scoring 4 and using our descore as quickly as possible in order to push our blocks and guarantee control.

Problem Status:

Minor

Solutions:

Some ways to increase the speed of our code is to make the robot itself faster, tune pid, and further optimize the route and bot functions.

Solution 1: Speed/PID

The most obvious way to speed up a code is to speed up the robot itself (run a higher voltage and tune the pid). This will save at least a second and won't require any change in route or code, making it the easiest time save possible. Tuning PID is also important as it also controls the timeouts before it can move to the next function (how long it pauses after a turn or straight before it does the next command), so reducing the pause consequently reduces the overall time.

Solution 2: Matchloader

One minor issue we faced in our previous code was a rolling block, which could cost us. Putting the matchloader as it approaches the 3 middle blocks both prevents this and allows the bot to head towards them at full speed, making it the ideal solution.

```
void waitmatchloader(){
   wait(0.5, seconds);
   matchloader.set(true);
   wait(1.25, seconds);
   matchloader.set(false);
```

(Function that puts down matchloader)

```
highintake.spin(forward, 100, percent);
vex::thread t1(waitmatchloader);

chassis.drive_distance(32);

chassis.turn_to_angle(130);
```

(Thread allows matchloader function to run simultaneously, putting it down at the perfect time)

Solution 3:

After scoring, it would take 3 moves to line up the descore. This is both highly inefficient and takes up precious time. Instead, we can use swings so that only two quick successive movements are needed to line up. This saves a minimum of two seconds while also leaving less margin for error.

```
chassis.right_swing_to_angle(100);
chassis.left_swing_to_angle(180);
```

Conclusion:

After these adjustments, we have been able to cut down over 3s of our code to make the total runtime 9-10s. While there is room for further improvement, this is a massive time save overall and ensures we have a competitive and quick 2v2 code that scores 4 blocks with control while limiting the opposing team (An effective push code).

1/28/2026 - Win Point Break Down

Meeting Goal: Look into how to earn the win point

Participants: Austin, Jenea

What is a win point:

During the autonomous period of 2v2 if an alliance meets the requirements placed by the game manual your alliance will gain a win point that can affect both the match and your ranking.

Types of win points:

Our team has identified 3 types of win points that are possible.

-Local autonomous win point

- At least seven (7) blocks of the alliance's color are scored.
- At least three (3) different goals include at least one (1) scored block of the alliance's color.
- At least three (3) blocks of the alliance's color have been removed from loaders adjacent to the alliance's alliance station.
- Neither robot is contacting the park barrier.

-Signature autonomous win point

- At least ten (10) blocks of the alliance's color are scored.
- At least three (3) different goals include at least two (2) scored blocks of the alliance's color.
- At least three (3) blocks of the alliance's color have been removed from loaders adjacent to the alliance's alliance station.
- Neither robot is contacting the park barrier.

-Solo autonomous win point:

- Follows the same rules as the others. Just one robot performs the required actions to get the win point.

Our teams goal:

As a team we plan to start progression on a solo autonomous winpoint. Having a solo win point can be a great tool if your alliance doesn't have a solid program as it can provide your alliance with the win point. The win point can determine the outcome of very close games so it's important to have if needed.

1/30/2026 - Mechanically InKleined

Meeting Goal: How did we do at the competition?
Participants: All

Goals for competition per category:

Skills tournament:
Our Goal for the skills portion of this competition is to improve our previous best skills score of 106 combined.

2v2 tournament:
Our goal for the 2v2 segment of the competition is to place at minimum in the top 3 seeds so we can pick a strong alliance and have the best chance at winning the tournament.

What went wrong:
At the competition our team faced some challenges:

- During a qualifications match another robot rammed into our robot and hit our battery cord causing our robot to white screen.
- Early on in the competition our matchloader broke in half completely.
-

Overall the competition didn't have many setbacks. Any issues are either minor or an easy fix mainly because we brought extra poly carbonate.

What went well:
At the competition our team faced many positives

- The color corresponding commands are a continued success because it gave quick and short instructions for the driver to accomplish. This limited our opponents ability to be able to hear our call outs.
- The 2v2 aspect of our competition we almost were flawless in qualification going 7-1.
- Inside the box during the 2v2, we adapted to our teammates very efficiently. First, we asked for how their robot performs, and then correspondingly adapted to the best strategy to win the round. This allowed us to go 7-1 in qualifying matches and showed continued excellence.

Placement:
1st overall in the 2v2 tournament

Awards:

Design award
Tournament champions

Conclusions:

The competition went really well and felt we had our best interview to date. As a team we are getting really comfortable with each other so teamwork is massively improving. With some minor issues with our program that we will have to work on fixing in the future. A few mechanical issues will need to be handled once we come back Monday.

Next steps and notes to improve:

Autonomous:

- Work on the matchloader for better consistency during prog.
- Spend more time on the program

Robot issues:

- We plan to start designing parking sleds to help get into the park zones easier and protect our gears.
- With the white screen being a scary possibility that can happen we will be moving where we have our battery mounted to not risk this happening again.
- Our intake had some jamming issues so we are going to try new solutions.

Teamwork has massively improved, with David, Austin, and Andrew in the driver box. The color callouts massively helped with getting an advantage.

2/1/2026 - February Plans

Meeting Goal: Plan what to do for the upcoming month.

Participants: All

Team Decisions:

→ We had an incredibly strong performance.
→ Work on tuning and improving subsystems.
→ Redo how we code.

Team Build Accomplishments:

→ Our robot only had minor issues that were extremely easy to resolve.
→ Improved use of polly

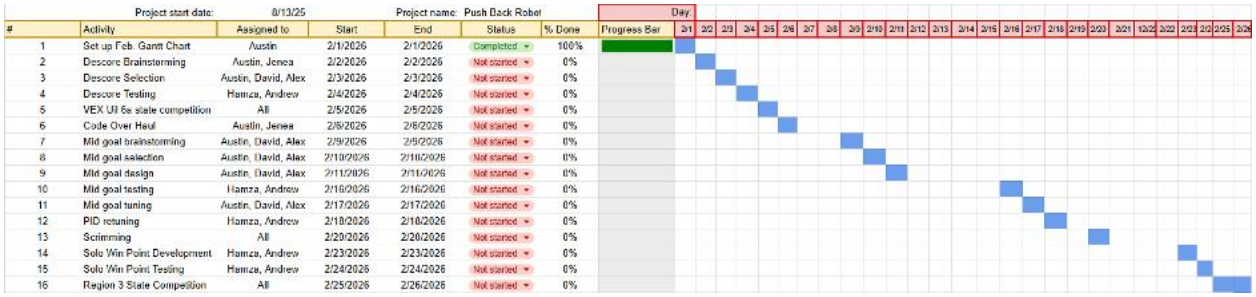
Team Programming Accomplishments:

→ Our 4 block push autonomous is continually improving in speed and consistency.
→ Tuning

Team Goals:

→ Solo win point
→ Code over haul
→ Tune our robot for states
→ Worlds qual

Updated gantt chart:



	Project start date:	8/13/25		Project name:	Push Back Robot	
#	Activity	Assigned to	Start	End	Status	% Done
1	Set up Feb. Gantt Chart	Austin	2/1/2026	2/1/2026	Completed	100%
2	Descore Brainstorming	Austin, Jenea	2/2/2026	2/2/2026	Not started	0%
3	Descore Selection	Austin, David, Alex	2/3/2026	2/3/2026	Not started	0%
4	Descore Testing	Hamza, Andrew	2/4/2026	2/4/2026	Not started	0%
5	VEX UI 6s state competition	All	2/5/2026	2/5/2026	Not started	0%
6	Code Over Haul	Austin, Jenea	2/6/2026	2/6/2026	Not started	0%
7	Mid goal brainstorming	Austin, David, Alex	2/9/2026	2/9/2026	Not started	0%
8	Mid goal selection	Austin, David, Alex	2/10/2026	2/10/2026	Not started	0%
9	Mid goal design	Austin, David, Alex	2/11/2026	2/11/2026	Not started	0%
10	Mid goal testing	Hamza, Andrew	2/16/2026	2/16/2026	Not started	0%
11	Mid goal tuning	Austin, David, Alex	2/17/2026	2/17/2026	Not started	0%
12	PID retuning	Hamza, Andrew	2/18/2026	2/18/2026	Not started	0%
13	Scrimming	All	2/20/2026	2/20/2026	Not started	0%
14	Solo Win Point Development	Hamza, Andrew	2/23/2026	2/23/2026	Not started	0%
15	Solo Win Point Testing	Hamza, Andrew	2/24/2026	2/24/2026	Not started	0%
16	Region 3 State Competition	All	2/25/2026	2/26/2026	Not started	0%

With our school's calendar we try our best to work as much as we can with the time we are given.

2/2/2026 - Descore Improvements: Brainstorming

Meeting Goal: Plan out improvements that can be made to our descore arm.

Participants: Austin, Jenea

Option 1: High Strength Shaft

A high strength shaft descore uses a shaft as the main structure for the arm with screws going through it.

Pros: Simple, low profile
Cons: Heavy, Prone to breaking,

Option 2: Boxxed Angle arm

A boxed Angle arm descore uses 2 long pieces of angle structure put together with half inch standoffs in between.

Pros: low profile, light, simple
Cons: prone to bend

Option 3: Simple c channel

A simple c channel descore arm uses a 1x1x2 c channel as the main structure for the arm.

Pros: simple, hard to bend, easily maintained.
Cons: blocks sight, asymmetric

2/3/2026 - Descore Improvements: Selection

Meeting Goal: Of the brainstormed descore improvements above, what is the best to use?
Participants: Austin, David, Alex

Parameters:

Efficiency

- We want our descore arm to be as efficient and fast as possible
- **Weighting:** This is scored at 0.26 because we want our solution to work. It doesn't do us any good if it doesn't work.
- **Concept Scoring:**
  - ***Option 1: 3/5***
  - ***Option 2: 5/5***
  - ***Option 3: 4/5***

Complexity

- How complex is the design to make and would we be able to easily replace it at comp if it breaks.
- **Weighting:** This is rated at 0.22, because it is fairly unimportant as all the designs are fairly simple.
- **Concept Scoring:**
  - ***Option 1: 3/5***
  - ***Option 2: 4/5***
  - ***Option 3: 5/5***

Tuning

- Due to us adding this to our current robot tuning will be needed to allow it to get the desirable result without the need of rebuilding.
- **Weighting:** This is ranked at 0.23 because we want to have flexibility in how its made so we can make adjustments based on driver preference.
- **Concept Scoring:**
  - ***Option 1: 2/5***
  - ***Option 2: 5/5***
  - ***Option 3: 5/5***

Design Strength

- We want a solution that will hold up in competition because the descore arm will be exposed and used a lot and won't do us any good if it breaks during a match.
- **Weighting:** This has the highest weight of all the parameters, at 0.28, because it affects the performance at competition. If the descore breaks at competition and we don't have a replacement or in the middle of a match it can cause issues.
- **Concept Scoring:**

- ***Iteration 1: 2/5***
- ***Iteration 2: 4/5***
- ***Iteration 3: 3/5***

Decision Matrix:

	Weighting	Option 1		Option 2		Option 3	
Efficiency	0.26	3	0.78	5	1.3	4	1.04
Complexity	0.22	3	0.66	4	0.88	5	1.10
Tuning	0.24	2	0.48	5	1.20	5	1.20
Design Strength	0.28	2	0.56	4	1.12	3	0.84
Totals:	1.00		**2.48**		**4.5**		**4.18**

Conclusion:

We will move forward with the second option as it was the highest scoring of the options and offers the driver with a greater margin of error to work with. During a high stress match an easier to work with descore arm can be the difference between a win and loss.

2/4/2026 - Descore Improvements: Testing

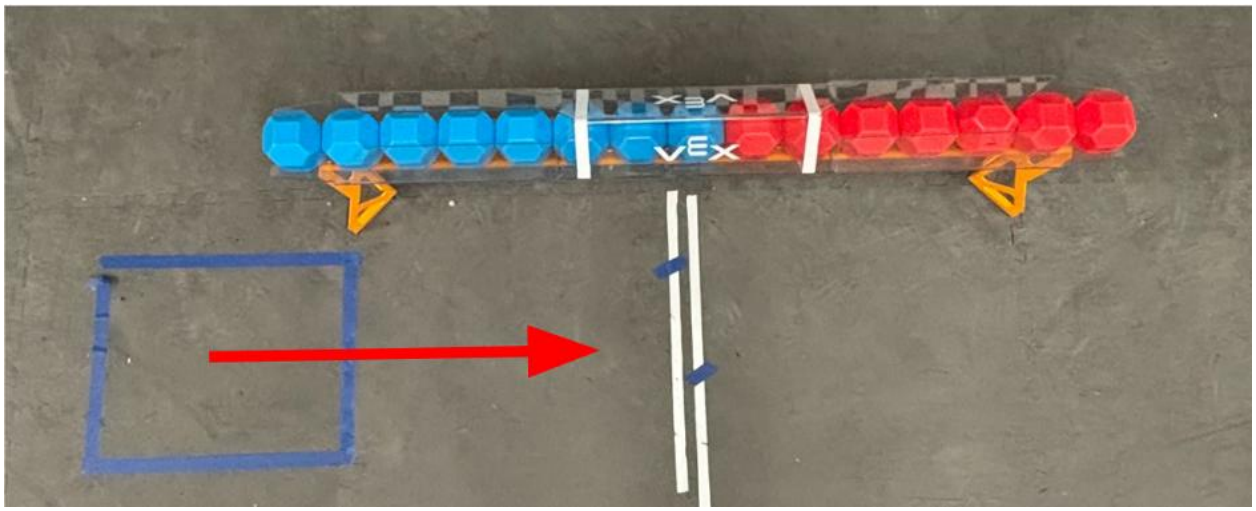
Meeting Goal: Does our new descore arm perform the way we want it to?

Participants: Austin, David, Alex

Test 1 Procedure: Max descore

6. Fill long goal
7. Line up robot at end of goal
8. Lower descore
9. Ram robot into goal
10. Record how many got descored

Diagram:

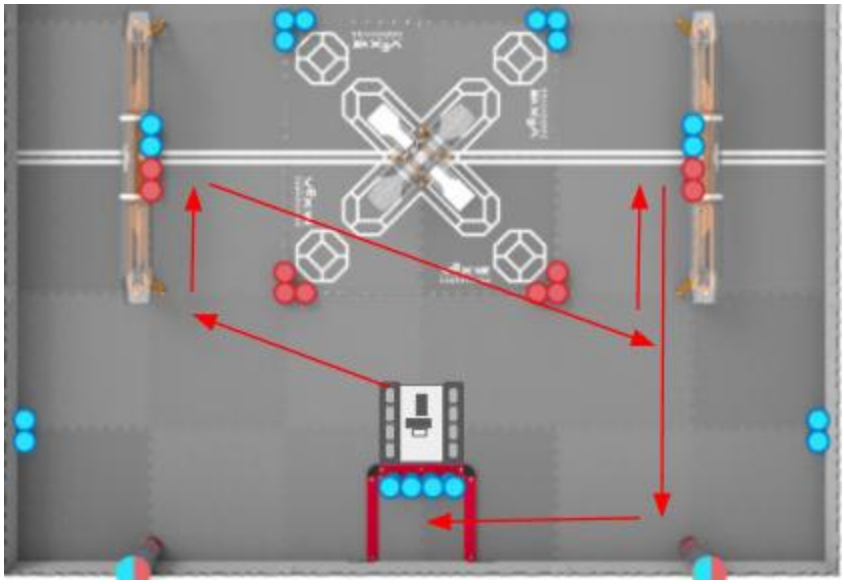


Trial 1	Trial 2	Trial 3	Trial 4	Trial 5	**Max**
10	11	10	9	11	**11**

Test 2 Procedure: Match imitation

7. Fill up both long goals
8. Line up robot at park zone
9. Start timer and drive towards goal 1
10. Descore and drive to other goal
11. Descore and move towards parking zone
12. Park and record time

Diagram:



Trial 1	Trial 2	Trial 3	Trial 4	Trial 5	**Average**
21.97	19.63	17.59	18.10	14.72	**18.402 sec**

Conclusion:

Our descore tech is as effective as we hoped but problems are apparent. Though we can't fully descore it, we were able to get 11 out which is a majority of the goal. The margin of error was greatly increased with this design which improved consistency and our speed.

2/5/2026 - VEX UIL Texas State Championship 6A

Meeting Goal: How did we do at the competition?

Participants: All

Goals for competition per category:

Skills tournament:

Our goal was to get new drivers experience at competition since the pressure and fields are different from the ones at home. The goal is to get practice for Andrew, Hamza, and Austin. Our goal is to hit a 70 driver and a 15-30 prog.

2v2 tournament:

Our goal in the 2v2 section of this competition is to gain experience and see what goes wrong with our robot before the Texas r3 championships.

What went wrong:

At the competition our team faced some challenges:

- During quarter finals against the 1st seeded alliance our main elevator chain snapped preventing our scoring.

Overall the competition didn't have many setbacks. Any issues are either minor or an easy fix.

- During the skills competition we ran into issues with our matchloaders consistency which really affected our ability to properly do skills.

What went well:

At the competition our team faced many positives

- We were able to make connections with other teams
- Let new drivers get skills practice on a competition field
- Color callouts were working as well as we hoped
- Our game strategy was put to the test and was able to pull off a 1v2

Placement:

9th overall in the 2v2 tournament

Awards:

Quarter finalist

Conclusions:

Throughout the competition we kept up with the better teams which gives our team a lot of hope for the region 3 competition. Almost upsetting in the quarter finals round we unfortunately had our main elevator chain snap but we still put up a fight despite being unable to score.

Next steps and notes to improve:

Autonomous:

- Work on the matchloader for better consistency during prog.
- Spend more time on the program

Robot issues:

- We plan to start designing parking sleds to help get into the park zones easier and protect our gears.
- With the white screen being a scary possibility that can happen we will be moving where we have our battery mounted to not risk this happening again.
- Our intake had some jamming issues so we are going to try new solutions.

Teamwork has massively improved, with David, Austin, and Andrew in the driver box. The color callouts massively helped with getting an advantage.

2/6/2026 - Code: Overhaul and Explanation

Meeting Goal: Overhaul code to allow for better autonomous runs.

Participants: Hamza, Andrew

Problems:

While our current platform of code did the job, it lacked consistency and speed. It was more inconsistent, and didn't give any room for growth, limiting our max potential and keeping us losing to better teams.

Solution:

After UIL, and before Region 3, we would have two weeks to adjust and improve our team. My plan is to completely overhaul the code by switching to the use of PROS and using Lemlib as a base. This would put us on level ground with the competition and with good enough coding from the team with the added features this provides we should be able to have competitive autonomous runs, as the autonomous bonuses and win points matter a lot in this game.

What this provides:

Lemlib allows for easy integration of code while providing several features. Similar features include PID and Odometry, but it also allows us to utilize new features such as new movement options and Motion Chaining.

PID:

PID stands for Proportional-Integral-Derivative control, and is used for smoother and more accurate movements. This is done by calculating an error, then returning an output that can be applied to reduce that error. A simple version of this function can be visualized with:

PID(error)= error * total error * change in error

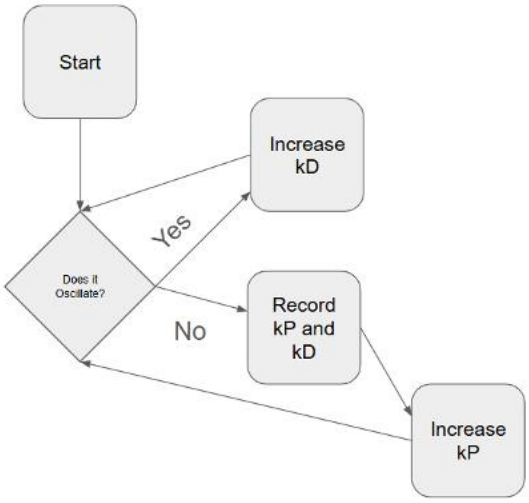
The three components are demonstrated in this. The proportional value increases the force if the error is greater, or reduces if the error is smaller. The integral value calculates the total error, and if it's constant for some time, it will build up force accordingly. The derivative value adjusts the force based on the change in error. If the error increases rapidly (it's reaching its setpoint too fast), it will slow down the force to prevent overshoot.

Advantages:

While this is the same system we used in our own library, using Lemlib allows us to utilize a more advanced version of this. This means on the field we can move more accurately and more efficiently.

Tuning:

Tuning the PID is the same as our own created library, but it is still a time-taking procedure. Using these numbers in combination is necessary for a functional robot using this type of control. The process for this starts with the kP and kD terms, which are the two terms most responsible for consistent movement. We will continually increase the numbers until there is no oscillation, following the process demonstrated through the chart below:



Note: kI (Integral) while useful is not needed for most movements and can sometimes cause an accumulation of error that messes up the code if done wrong, so to simplify we opted out of using it.

After this process, we are able to tune the numbers and by the time you are finished and have also added your other tuning settings such as timeouts and error margins, we ended up with this:

```
// lateral motion controller
lemlib::ControllerSettings linearController(3.5, // proportional gain (kP)
                                            0, // integral gain (kI)
                                            10, // derivative gain (kD)
                                            3, // anti windup
                                            1, // small error range, in inches
                                            100, // small error range timeout, in
milliseconds
                                            3, // large error range, in inches
                                            500, // large error range timeout, in
milliseconds
                                            35 // maximum acceleration (slew)
);
```

```
// angular motion controller
lemlib::ControllerSettings angularController(2.5, // proportional gain (kP)
                                             0, // integral gain (kI)
                                             12, // derivative gain (kD)
                                            3, // anti windup
                                             1, // small error range, in degrees
                                             100, // small error range timeout,
in milliseconds
                                             3, // large error range, in degrees
                                             500, // large error range timeout,
in milliseconds
                                             0 // maximum acceleration (slew)
);
```

Odometry:

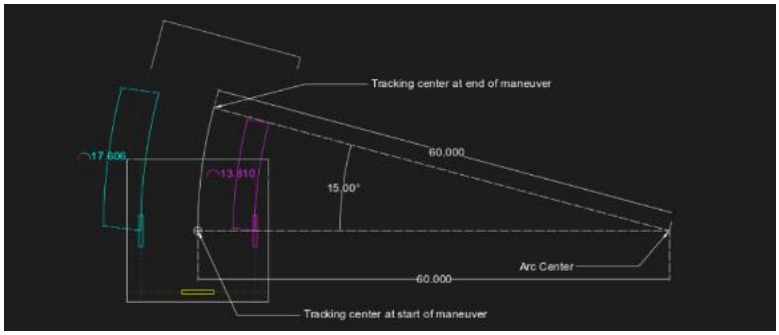
Odometry is the use of primarily rotational sensors to plot an effective coordinate grid on the field. By knowing the width and tracing centre of the robot, it creates an effective bubble so that the robot knows exactly where it is on the field. This means that at any point, you can tell the robot to drive or point to any point on the field.

Advantages:

While this is the same system we used in our own library, using Lemlib allows us to utilize a more advanced version of this. This means on the field we can move more accurately and more efficiently.

How this works:

Position tracking works by modelling the robot's motion (the motion of the tracking center) as an arc over an infinitely short time interval. By using the track width, it's able to move both the left and right wheels concentric to the tracing centre's arc.



This diagram also represents how the arc angle is the same as the change in orientation of the robot. This angle can be represented (in radians) by the formula:

$$\Delta\theta = \frac{\Delta L - \Delta R}{s_L + s_R}$$

This is an example of a "state function"; if any two of the following are known (starting orientation, ending orientation, and distance between left and right wheel travel), it is possible to find the other one. Since any translation without rotation does not change the difference between these numbers (it changes both by the same amount), this formula applies regardless of where the robot is on the field, or how it got there. This means that orientation can be calculated as an absolute quantity, rather than a relative change from the last arc approximation.

Also based on this arc, the translation (change in position) can be calculated. To do this, the radius of the tracking center's arc needs to be known. This can be calculated as the radius of the right side arc plus the offset of the right tracking wheel from the tracking center:

$$r_R = \frac{\Delta R}{\Delta\theta} + s_R$$

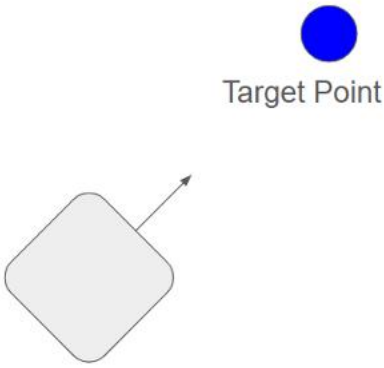
There are two types of motion that the robot can undergo: translational and rotational. It is trivial to show that the position of the wheel does not matter for translational motion, as the velocity is the same at all points on the robot. So let’s go back to the basics, how exactly do the odometry and PID work together?

How Move to Point Works:

There are multiple levels to how odometry works in unison with error calculations, so to simplify I will be using diagrams and example code from an older library of mine, while breaking it down into levels and the process of adding code (this is what we did for our first library).

Level 0:

Let's assume we are already facing the target point:

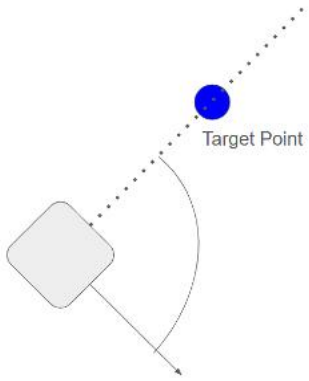


We want to drive straight forward until we reach the target point. In other words, we want to minimize the distance between the robot and the target point. If we call this distance the "error," it becomes clear that a PID controller is applicable here. To calculate the error, we can use the distance formula.

```
// constants:
// maxError: the maximum error you allow for the distance
fn moveToPoint(targetPoint) {
    while error > maxError {
        let error = distance(robot, targetPoint);
        let output = computePID(error);
        driveForward(output);
    }
}
```

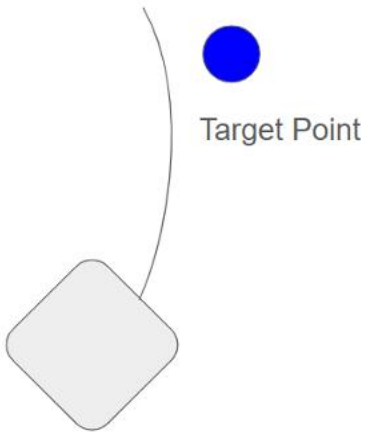
Level 1:

However, this isn't applicable in most cases as if you're even off by one degree you will drive right past the point and never settle. This is when we introduce our lateral and angular pid from earlier. (Diagram with angular error annotated)



So to fix this, we can add code so that it adjusts itself and hopefully gets at least close to the target point. However, if the motors are overheating or there is some obstacle, it can still have a near miss and again never settle.

```
// we now have two PID functions: computeLateralPID and
computeAngularPID
// constants:
// maxLateralError: the maximum error you allow for the distance
// maxAngularError: the maximum error you allow for the angle
before we start driving to the target
fn moveToPoint(targetPoint) {
    while abs(angularError) > maxAngularError {
        // assume angularError is from -180° - 180°
        let angularError = angleFrom(robot, targetPoint);
        let output = computeAngularPID(angularError);
        driveTurn(output);
    }
    while error > maxLateralError {
        let error = distance(robot, targetPoint);
        let output = computeLateralPID(error);
        driveForward(output);
    }
}
```

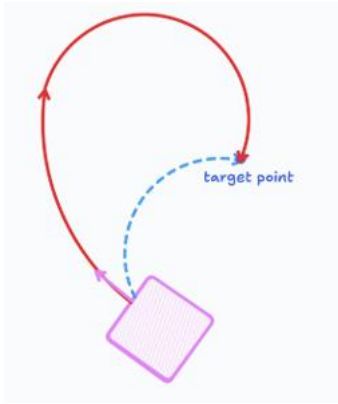


Level 2:

To fix this, you have to be able to detect when you are starting to deviate from the path. Starting and stopping is too slow, so you have to calculate these terms simultaneously. For this, we can use the arcade drive within the code to calculate the lateral and angular error. This would look something like:

```
fn driveArcade(straight, turn) {
    moveLeftSide(straight + turn);
    moveRightSide(straight - turn);
}
fn moveToPoint(targetPoint) {
    while lateralError > maxLateralError {
        let lateralError = distance(robot, targetPoint);
        let lateralOutput = computeLateralPID(lateralError);
        let angularError = angleFrom(robot, targetPoint);
        let angularOutput = computeAngularPID(angularError);
        driveArcade(lateralOutput, angularOutput);
    }
}
```

However, that's not all. Let's assume the robot is facing away significantly from the target point. The move to point function would move forwards not knowing any better resulting in a much longer turn.



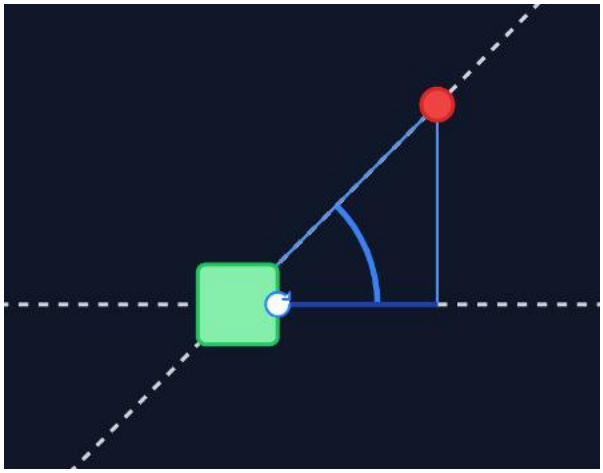
Level 3:

Ideally, there'd be a way to reduce the lateral pid so that it doesn't want to move forwards as much. To do this, you can multiply the raw distance (the only thing we were using before) by the cosine of the angular error. This works as when the angular error is near 0, cosine is close to 1, so it moves at a higher speed. However, when the angular error is bigger, cosine gets smaller, slowing down the robot. One thing to note is that when the cosine is negative, Lemlib forces the robot to drive forward so you have to set the drive condition to false to make it move backwards.

```
fn moveToPoint(targetPoint) {
    while lateralError > maxLateralError {
        let angularError = angleFrom(robot, targetPoint);
        let angularOutput = computeAngularPID(angularError);

        let lateralError = distance(robot, targetPoint) *
cos(angularError);
        if abs(angularError) > 90° {
            lateralError = 0;
        }
        let lateralOutput = computeLateralPID(lateralError);

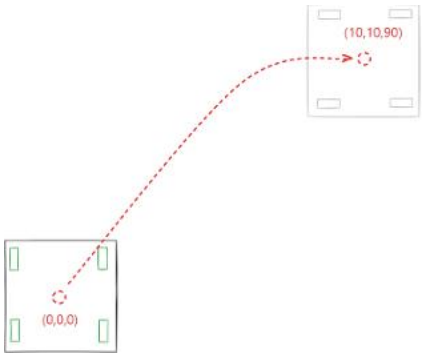
        driveArcade(lateralOutput, angularOutput);
    }
}
```



And that's how we coded to move to point in our previous library and the basics of how a lot of libraries like Lemlib also function. While the code I demonstrated in these steps would still need editing (as it's an older project of mine), I believe they showcase the principle of how movements work within our current code and how they will work in Lemlib.

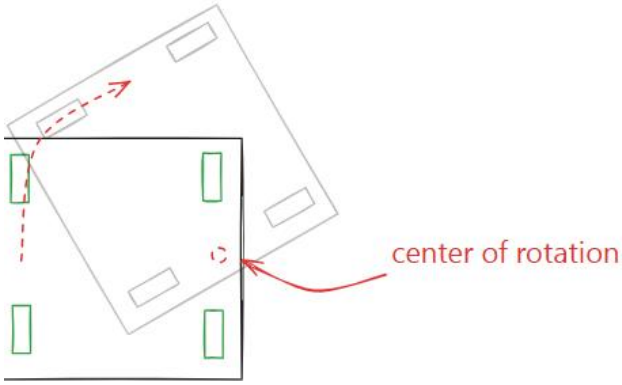
Advanced Movements:

Move to Point, while effective, obviously isn't the only movement we use. For lateral movement, Lemlib allows us to use Move to Pose, which achieves the same thing with the added layer of ending with the bot facing in a desired angle. The turn for this is adjusted using the lead parameter, which controls the width of the turn. (example of this below)




```
chassis.moveToPose(
    0, // x = 0
    0, // y = 0
    90, // theta = 90
    4000, // timeout of 4000ms
    {.lead = 0.5, .horizontalDrift = 2}
);
```

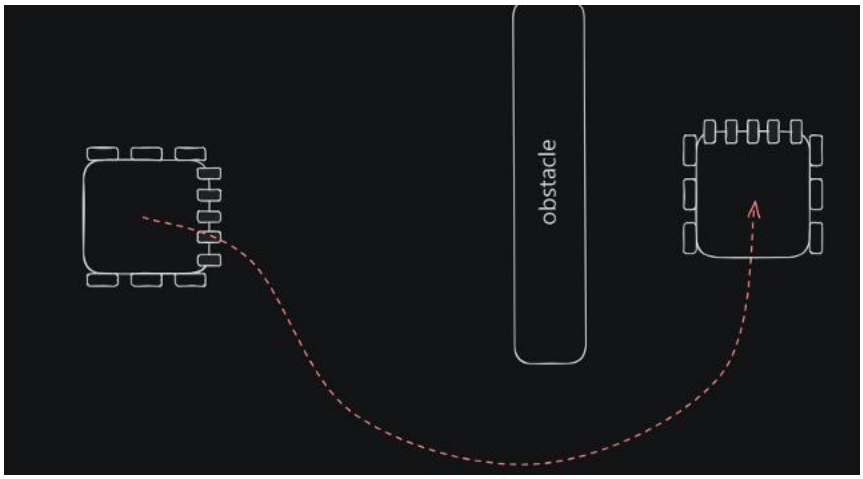
For angular movements, there's regular turns and swings. Turns use both sides of the drivetrain to turn and stay within the same spot, whereas swings turn using only one side of the drivetrain, making them rotate along a centre of rotation on the edge of the robot.



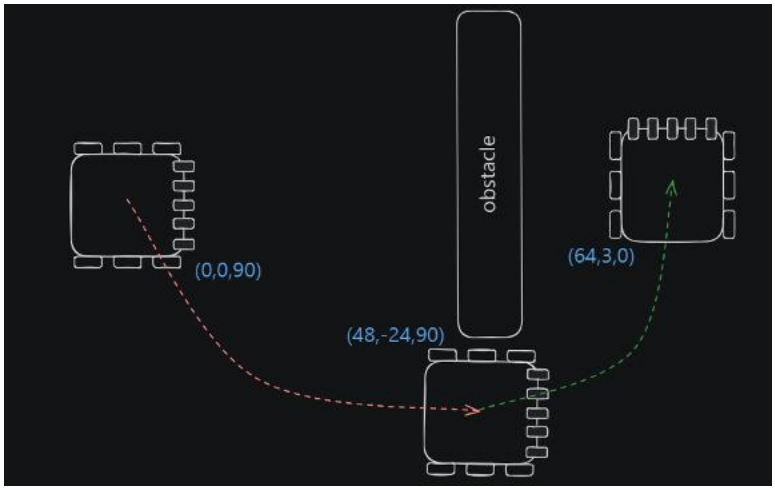
This can be especially useful in pushback when turning out of a long goal or to avoid obstacles, and can usually save crucial time if used correctly.

Motion Chaining:

When there is a path that can't be described in one motion, but we don't want to stop in the middle, we can use motion chaining. An example of this is moving around an obstacle. While Move to Pose could work, it would require the robot to stop in the middle wasting time. However, not stopping the robot means it can never settle, so how do you implement this?



First, to not make the robot slow down, you can implement a minimum speed. This minimum speed overrides everything from PID, allowing the robot to stay in motion. However, when you do this, the robot can never settle at the midpoint. That's where Early Exit Range is implemented. When the error is below this stated parameter, it will automatically move on to the next function. (example on next page)




```
void goAroundObstacle() {
    chassis.setPose(0, 0, 90);

    // go around obstacle to prevent hitting it
    chassis.moveToPose(
        48,
        -24,
        90,
        2000,
        {.minSpeed=72, .earlyExitRange=8}
        // a minSpeed of 72 means that the chassis will slow down as
        // it approaches the target point, but it won't come to a full stop

        // an earlyExitRange of 8 means the movement will exit 8" away from
        // the target point
    );

    // go to target position
    chassis.moveToPose(64, 3, 0, 2000);
}
```

Configuration:

Now that we know everything works, we actually have to set it up. Every motor has to have its cartridge stated, its direction (negative if reversed), and port. Assume these rules as well as general C++ rules when configuring.

Drivetrain:

We run a 6 motor drivetrain with three on each side. Each cartridge is a blue motor, and some ports are reversed (negative). We have to state all of this when defining the motor group. For now, we just have to state the motor group for each side. We then also have to state the other features of our drivetrain such as track width, outside rpm, wheel size, etc.

```
// motor groups
pros::MotorGroup leftMotors({-4, -2, -3}, pros::MotorGearset::blue); // left
motor group
pros::MotorGroup rightMotors({1, 5, 6}, pros::MotorGearset::blue); // right motor
group
```

```
// drivetrain settings
lemlib::Drivetrain drivetrain(&leftMotors, // left motor group
                              &rightMotors, // right motor group
                              11.875, //track width
                              lemlib::Omniwheel::NEW_325,
                              480, // drivetrain rpm
                              2 // horizontal drift is 2. If we had traction
wheels, it would have been 8
```

Motors and Pneumatics:

Next, we have set up our intake and hood motors, as well as our matchloader, descore, and trapdoor for midgoal pistons. Since these are all individual, we won't be putting these into motor groups.

```
pros::Motor intake(-10, pros::MotorGearset::blue); // intake
pros::Motor hood(7, pros::MotorGearset::blue); // hood

pros::ADIDigitalOut wing('B');  // Wing
pros::ADIDigitalOut load('A');  // Matchloader
pros::ADIDigitalOut mid('C');  // Trapdoor
```

Sensors (Odometry):

Now we set up our sensors. The IMU just needs to be stated. However for odometry to work, we will also need to state the distances from the tracking centre and the size of the wheels attached to them. Then, we have to state later what exactly we are choosing to use for odometry.

```
// Inertial Sensor
pros::Imu imu(20);

// tracking wheels
// horizontal tracking wheel encoder. Rotation sensor
pros::Rotation horizontalEnc(13);
// vertical tracking wheel encoder. Rotation sensor
pros::Rotation verticalEnc(11);
// horizontal tracking wheel.
lemlib::TrackingWheel horizontal(&horizontalEnc, lemlib::Omniwheel::NEW_2,
-3.25);
// vertical tracking wheel.
lemlib::TrackingWheel vertical(&verticalEnc, lemlib::Omniwheel::NEW_2, 1.5);
```

```
// sensors for odometry
lemlib::OdomSensors sensors(&vertical, // vertical tracking wheel
                            nullptr, // vertical tracking wheel 2, set to nullptr
as we don't have a second one
                            &horizontal, // horizontal tracking wheel
                            nullptr, // horizontal tracking wheel 2, set to
nullptr as we don't have a second one
                            &imu // inertial sensor
);
```

PID:

We also have to state our tuned PID constants. Within this, we can also add extra improvements such as slew (acceleration) and error times.

```
// lateral motion controller
lemlib::ControllerSettings linearController(3.5, // proportional gain (kP)
                                            0, // integral gain (kI)
                                            10, // derivative gain (kD)
                                            3, // anti windup
                                            1, // small error range, in inches
                                            100, // small error range timeout, in
milliseconds
                                            3, // large error range, in inches
                                            500, // large error range timeout, in
milliseconds
                                            35 // maximum acceleration (slew)
);

// angular motion controller
lemlib::ControllerSettings angularController(2.5, // proportional gain (kP)
                                             0, // integral gain (kI)
                                             12, // derivative gain (kD)
                                            3, // anti windup
                                             1, // small error range, in degrees
                                             100, // small error range timeout,
in milliseconds
                                             3, // large error range, in degrees
                                             500, // large error range timeout,
in milliseconds
                                             0 // maximum acceleration (slew)
);
```

Driver Control:

Within our chassis, we also need to state our controller settings if we want to add throttle and steer curves to help with sensitivity and deadzones.

```
// input curve for throttle input during driver control
lemlib::ExpoDriveCurve throttleCurve(3, // joystick deadband out of 127
                                     10, // minimum output where drivetrain will
move out of 127
                                     1.05 // expo curve gain
);

// input curve for steer input during driver control
lemlib::ExpoDriveCurve steerCurve(3, // joystick deadband out of 127
                                  10, // minimum output where drivetrain will
move out of 127
                                  1.05 // expo curve gain
);
```

Chassis:

Finally, you have to combine all the chassis related terms so that they all work in unison. This includes the drivetrain, pid, and the curves.

```
// create the chassis
lemlib::Chassis chassis(drivetrain, linearController, angularController, sensors,
&throttleCurve, &steerCurve);
```

2/9/2026 - Mid Goal Score v2: Brainstorming

Meeting Goal:

Participants: Austin, Jenea

Trap door

A trap door middle scoring mechanism involves 2 angles separated by a 3 inch standoff which allows us to pivot an entire section of our ramp for the blocks to roll out of the robot at a lower height.

Pros: Easy to build, high speed,
Cons: medium pushing force

Counter roller

A counter roller uses gears and chain to add another roller underneath the hood to push blocks through. By using the same motor as the hood this adds friction but also adds pushing force against the blocks in the middle goal.

Pros: increased scoring speed in mid goal and long goal.
Cons: Difficult to build, and tune

Band puller

The band pull down is simply a piston attached to the bands that span across our ramp and to score in the middle goal we activate the piston the bands lower and the blocks can exit at a lower height.

Pros: Extremely simple
Cons: overall poor performance

2/10/2026 - Mid Goal Scoring V2: Selection

Meeting Goal: Tune our awp route and code to increase performance.

Participants: Austin, David, Alex

Parameters:

Efficiency

- We want our sleds to have a good angle for getting in and over the barrier.
- **Weighting:** This is scored at 0.26 because we want our solution to work. It doesn't do us any good if it doesn't work.
- **Concept Scoring:**
  - ***Trap door: 5/5***
  - ***Counter roller: 4/5***
  - ***Band puller: 3/5***

Complexity

- The fewer cuts that have to be made in the plastic, the easier it is to make. This also accounts for how much usable plastic is left in the sheet.
- **Weighting:** This is rated at 0.22, because it is fairly unimportant as we have a lot of available plastic and these pieces are not super large.
- **Concept Scoring:**
  - ***Trap door: 4/5***
  - ***Counter roller: 2/5***
  - ***Band puller: 5/5***

Tuning

- Due to us adding this to our current robot tuning will be needed to allow it to get the desirable result without the need of rebuilding.
- **Weighting:** This is ranked at 0.23 because we want to have flexibility in where we connect the plastic piece in case it doesn't work as intended.
- **Concept Scoring:**
  - ***Trap door: 5/5***
  - ***Counter roller: 3/5***
  - ***Band puller: 4/5***

Design Strength

- We want a solution that will hold up in competition because the sleds will be exposed and won't do us any good if they break during a match.
- **Weighting:** This has the highest weight of all the parameters, at 0.28, because it affects the performance at competition. If the sleds break at competition and we don't have a replacement or in the middle of a match it can cause issues.
- **Concept Scoring:**

- ***Trap door: 4/5***
- ***Counter roller: 5/5***
- ***Band puller: 3/5***

Decision Matrix:

	Weighting	Option 1		Option 2		Option 3	
Efficiency	0.26	5	1.30	4	1.04	3	0.78
Complexity	0.22	4	0.88	3	0.66	5	1.10
Tuning	0.24	5	1.20	2	0.48	4	1.12
Design Strength	0.28	4	1.12	5	1.40	3	0.84
Totals:	1.00		**4.5**		**3.58**		**3.84**

Conclusion:

We will move forward with the first option as it scored the highest and is the most efficient and wouldn't affect the structure of our robot that much. The trap door can also help with the issue of blocks slipping through the bands as it will replace the top few bands. This option has a lot of room to tune that will be needed.

2/11/2026 - Mid Goal Scoring V2: Design Plan

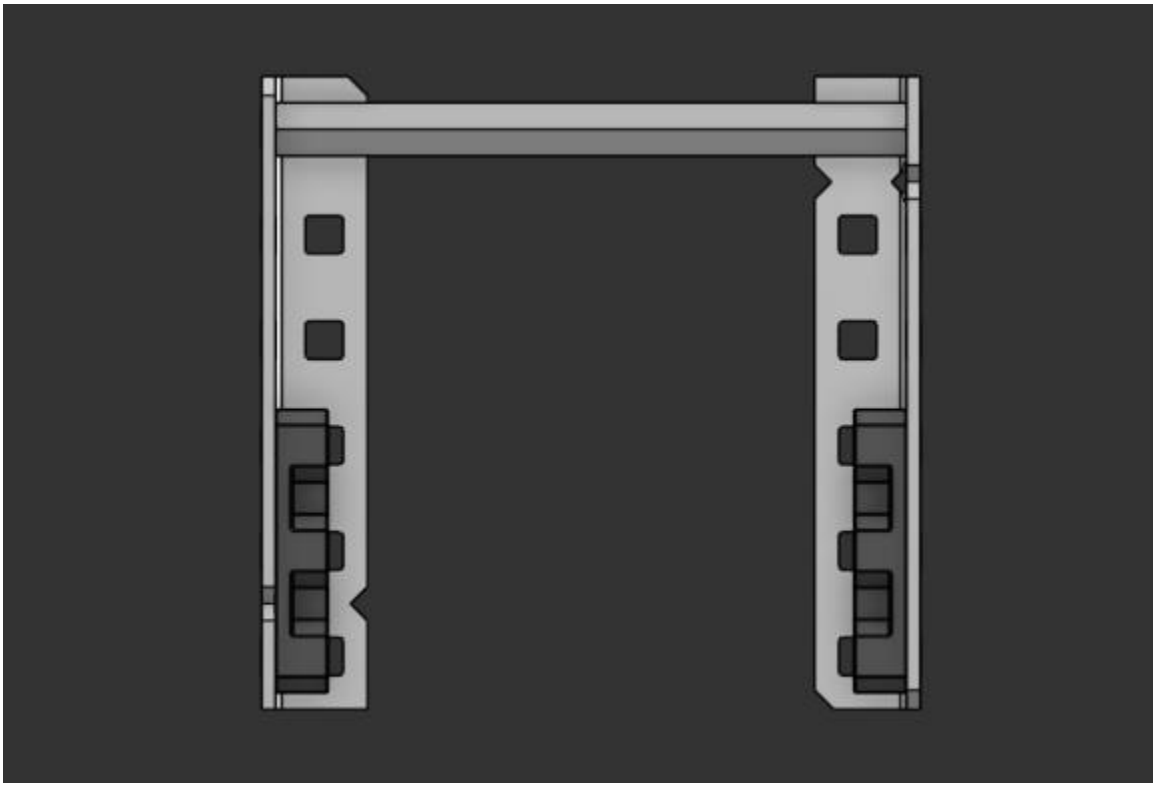
Meeting Goal: Tune our awp route and code to increase performance.

Participants: Austin, David, Alex

Design Specifications:

- **Powering:** Piston
- **Arm type:** 2-bar

CAD:



Bill of Materials:

1- 1 3” standoff
2- 2 low profile bearings
3- 1 .25 piston
4- 2 6 hole half cut c-channel
5- 2 .5” screws

2/16/2026 - Mid Goal Scoring V2: Testing

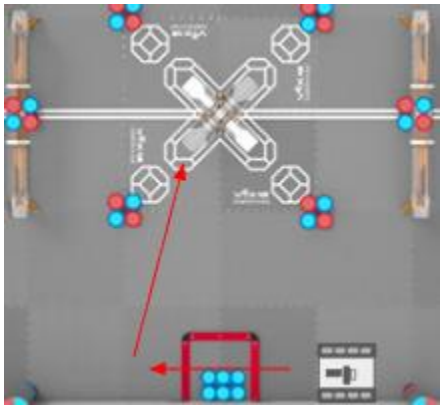
Meeting Goal: Tune our awp route and code to increase performance.

Participants: Hamza, Andrew

Test 1 Procedure: Skills imitation

10. Set up skills park zone blocks
11. Give robot a preload
12. Line up robot one tile to the left of the parkzone
13. Start timer
14. drive over to parkzone
15. Move forward and start intake
16. Once all blocks are intaked move towards mid goal
17. Score into mid goal
18. Stop timer once all 7 are scored

Diagram:



Trial 1	Trial 2	Trial 3	Trial 4	Trial 5	**Average**
8.61	9.37	9.89	8.75	9.04	**9.132sec**

Test 2 Procedure: Speed of outtake

11. Fill up robot with game pieces
12. Activate mid goal score
13. Begin timer
14. Start outtake
15. Stop timer once robot is cleared

Trial 1	Trial 2	Trial 3	Trial 4	Trial 5	**Average**
2.83	2.59	3.10	2.69	2.55	**2.752sec**

Test 3 Procedure: Capacity

16. Prepare a group of game pieces
17. Start intaking until robot is full
18. Activate mid goal
19. Outtake and check for jamming
20. Repeat until max amount of blocks is found

Using abbreviation DNO for did not outtake and SO for successful outtake.

Trial 1	Trial 2	Trial 3	Trial 4	Trial 5	**Max**
14 DNO	12 DNO	10 DNO	8 DNO	7 SO	**7 Blocks**

Conclusion:

This design did not see an improvement in the total capacity but it did improve the rate in which blocks outtake. This allowed for our times to improve by almost a second which will come in handy when working on autonomous skills and SWP. The consistency of scoring also improved as the other design was prone to jams and was overall slower.

2/17/2026 - Mid goal: Tuning

Meeting Goal: Tune our robot to increase performance.

Participants: Austin, David, Alex

Observations:

During testing and driving the bot with the new subsystem we noticed an issue when we want to score on long goals. Blocks could fall out due to the gap created by the trapdoor.

Problem Status:

Minor to none

Solutions:

By adding rubber bands to the open part of the trapdoor we are able to fix this problem and not affect the performance of the subsystem.

Tests Re-Ran:

Due to no major problems being found while testing, no tests will need to be re-ran.

2/18/2026 - PID: Tuning

Meeting Goal: Tune our robot to increase performance.

Participants: Hamza, Andrew

Observations:

While our current turns and acceleration is good, there is still some oscillation and inconsistency within the motion due to the acceleration being too harsh or the slow down being too strong.

Problem Status:

Minor to none

Solution:

By going back to tuning pid, and slightly adjusting the numbers based off of these observations, we can fix these issues pretty quickly.

Procedure:

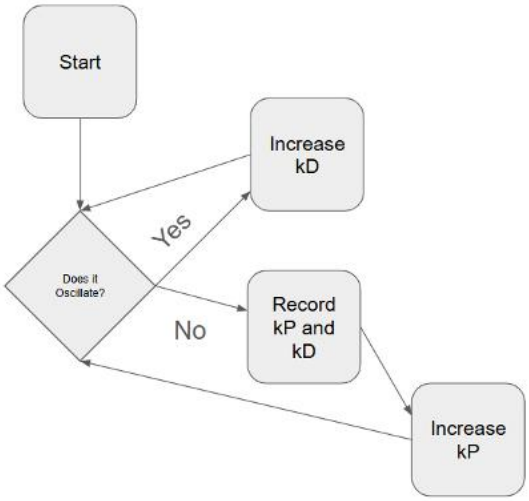
To tune the PID, you first have to separate the pid into lateral (forwards) and angular (turning) PID. We can change the autonomous part in the code to test these two individually.

```
void autonomous() {
     // set position to x:0, y:0, heading:0
    chassis.setPose(0, 0, 0);

    // forward 42in with long timeout to test oscillation
    chassis.moveToPoint(0, 42, 10000);
}
void autonomous() {
     // set position to x:0, y:0, heading:0
    chassis.setPose(0, 0, 0);

    // turn 90 degrees with long timeout to test oscillation
    chassis.turnToHeading(90, 10000);
}
```

After running the code, we follow this line of logic to tune pid for the highest speed and consistency possible.



Before Tuning:

Prior to this tuning, one other issue we also faced was that the turning in driver control was really unresponsive and didn't want to drive at the same time. As well as this, the acceleration would sometimes cause the robot to even tip slightly as it went from a stop to moving again when the max speed wasn't restricted.

```
// lateral motion controller
lemlib::ControllerSettings linearController(3.5, // proportional gain (kP)
                                            0, // integral gain (kI)
                                            10, // derivative gain (kD)
                                            3, // anti windup
                                            1, // small error range, in inches
                                            100, // small error range timeout, in
milliseconds
                                            3, // large error range, in inches
                                            500, // large error range timeout, in
milliseconds
                                            35 // maximum acceleration (slew)
);

// angular motion controller
lemlib::ControllerSettings angularController(2.5, // proportional gain (kP)
                                             0, // integral gain (kI)
                                             12, // derivative gain (kD)
                                            3, // anti windup
                                             1, // small error range, in degrees
```

```
                                            100, // small error range timeout,
in milliseconds
                                            3, // large error range, in degrees
                                            500, // large error range timeout,
in milliseconds
                                            0 // maximum acceleration (slew)
);
```

After Tuning:

After going through this process again, we were able to determine that the movements were previously sluggish due to a really high kD, and harsh acceleration due to no restriction on it. We fixed this by mainly reducing the Derivative and introducing more slew.

```
// lateral motion controller
lemlib::ControllerSettings linearController(3, // proportional gain (kP)
                                            0, // integral gain (kI)
                                            8, // derivative gain (kD)
                                            3, // anti windup
                                            1, // small error range, in inches
                                            100, // small error range timeout, in
milliseconds
                                            3, // large error range, in inches
                                            500, // large error range timeout, in
milliseconds
                                            45 // maximum acceleration (slew)
);

// angular motion controller
lemlib::ControllerSettings angularController(2.5, // proportional gain (kP)
                                            0, // integral gain (kI)
                                            6, // derivative gain (kD)
                                            3, // anti windup
                                            1, // small error range, in degrees
                                            100, // small error range timeout,
in milliseconds
                                            3, // large error range, in degrees
                                            500, // large error range timeout,
in milliseconds
                                            40 // maximum acceleration (slew));
```

Notebook Appendix

Participants: Austin, Jenea

Citations:

All drawings unless sited were created by the 22603A team during the 2025-2026 Vex v5 season.

Title of resource: Holonomic Drive - BLRS Wiki
Author(s): Purduesigbots
Date of release: 2023
Location: wiki.purduesigbots.com/hardware/vex-drivetrains/holonomic-drive

Title of resource: CS-STEM Network
Author(s): Unknown
Date of release: Unknown
Location: www.cs2n.org/u/mp/badge_pages/104

Title of resource: Vex Classic
Author(s): Xenon27
Date of release: Sept. 2022
Location: https://www.vexforum.com/t/creating-a-basic-drive/106873

Title of resource: Push Back Game Manual
Author(s): The Game Design Committee
Date of release: June 26, 2025
Version: 1.0
Location: https://content.vexrobotics.com/docs/25-26/v5rc-push-back/docs/Push-Back-1.0.pdf

Full Code:

```
#include "main.h"
#include "lemlib/api.hpp" // IWYU pragma: keep
#include "lemlib/chassis/chassis.hpp"
#include "pros/misc.h"

// controller
pros::Controller controller(pros::E_CONTROLLER_MASTER);

// motor groups
```

```
pros::MotorGroup leftMotors({-4, -2, -3}, pros::MotorGearset::blue); // left
motor group
pros::MotorGroup rightMotors({1, 5, 6}, pros::MotorGearset::blue); // right motor
group

pros::Motor intake(-10, pros::MotorGearset::blue); // intake
pros::Motor hood(7, pros::MotorGearset::blue); // hood

pros::ADIDigitalOut wing('B');  // Wing
pros::ADIDigitalOut load('A');  // Matchloader
pros::ADIDigitalOut mid('C');  // Trapdoor

// states for pistons for toggling
bool loadState = false;
bool wingState = false;
bool midState = false;

// Inertial Sensor
pros::Imu imu(20);

// tracking wheels
// horizontal tracking wheel encoder. Rotation sensor
pros::Rotation horizontalEnc(13);
// vertical tracking wheel encoder. Rotation sensor
pros::Rotation verticalEnc(11);
// horizontal tracking wheel.
lemlib::TrackingWheel horizontal(&horizontalEnc, lemlib::Omniwheel::NEW_2,
-3.25);
// vertical tracking wheel.
lemlib::TrackingWheel vertical(&verticalEnc, lemlib::Omniwheel::NEW_2, 0.5);

// drivetrain settings
lemlib::Drivetrain drivetrain(&leftMotors, // left motor group
                              &rightMotors, // right motor group
                              12, //track width
                              lemlib::Omniwheel::NEW_325,
                              480, // drivetrain rpm
```

```
                            2 // horizontal drift is 2. If we had traction
wheels, it would have been 8
);

// lateral motion controller
lemlib::ControllerSettings linearController(3, // proportional gain (kP)
                                            0, // integral gain (kI)
                                            8, // derivative gain (kD)
                                            3, // anti windup
                                            1, // small error range, in inches
                                            100, // small error range timeout, in
milliseconds
                                            3, // large error range, in inches
                                            500, // large error range timeout, in
milliseconds
                                            45 // maximum acceleration (slew)
);

// angular motion controller
lemlib::ControllerSettings angularController(2.5, // proportional gain (kP)
                                             0, // integral gain (kI)
                                             6, // derivative gain (kD)
                                            3, // anti windup
                                             1, // small error range, in degrees
                                             100, // small error range timeout,
in milliseconds
                                             3, // large error range, in degrees
                                             500, // large error range timeout,
in milliseconds
                                             40 // maximum acceleration (slew)
);

// sensors for odometry
lemlib::OdomSensors sensors(&vertical, // vertical tracking wheel
                            nullptr, // vertical tracking wheel 2, set to nullptr
as we don't have a second one
                            &horizontal, // horizontal tracking wheel
```

```
                            nullptr, // horizontal tracking wheel 2, set to
nullptr as we don't have a second one
                            &imu // inertial sensor
);

// input curve for throttle input during driver control
lemlib::ExpoDriveCurve throttleCurve(3, // joystick deadband out of 127
                                     10, // minimum output where drivetrain will
move out of 127
                                     1.019 // expo curve gain
);

// input curve for steer input during driver control
lemlib::ExpoDriveCurve steerCurve(3, // joystick deadband out of 127
                                  10, // minimum output where drivetrain will
move out of 127
                                  1.019 // expo curve gain
);

// create the chassis
lemlib::Chassis chassis(drivetrain, linearController, angularController, sensors,
&throttleCurve, &steerCurve);



/**
 * Runs initialization code. This occurs as soon as the program is started.
 *
 * All other competition modes are blocked by initialize until it finishes.
 */
void initialize() {
    pros::lcd::initialize(); // initialize brain screen
    chassis.calibrate(); // calibrate sensors
}

/**
 * Runs while the robot is disabled
 */
```

```
void disabled() {}

/**
 * runs after initialize if the robot is connected to field control
 */
void competition_initialize() {}


void rightAutonomous() {

    // set position to x:0, y:0, heading:0
    chassis.setPose(0, 0, 0);


    // Collect 3 middle balls
    intake.move(127);
    chassis.moveToPoint(10, 36, 1500);
    pros::delay(500);
    load.set_value(true);


    // Line up with the goal and shoot
    chassis.turnToPoint(43, 10, 750);
    load.set_value(false);
    chassis.moveToPoint(43, 10, 1500);
    chassis.turnToHeading(180, 750);
    pros::delay(500);


    intake.move(-20);
    chassis.moveToPoint(43, 1000, 750, {.forwards = false});
    pros::delay(500);
    hood.move(127);
    intake.move(127);
    pros::delay(2000);
    intake.move(0);
    hood.move(0);
```

```
    // Line up for push and descore
    chassis.turnToPoint(50, 5, 750);
    chassis.moveToPoint(50, 5, 1500);
    chassis.turnToHeading(180, 750);


    chassis.moveToPoint(50, 40, 2000, {.forwards = false});
    leftMotors.set_brake_mode(pros::E_MOTOR_BRAKE_HOLD);
    rightMotors.set_brake_mode(pros::E_MOTOR_BRAKE_HOLD);
}

void SWPautonomous() {

     // set position to x:0, y:0, heading:0
    chassis.setPose(0, 0, 0);
    wing.set_value(true);
    intake.move(127);
      // Get Matchload
    chassis.moveToPoint(0, -38, 1000,  {.forwards=false, .earlyExitRange=15});
    chassis.turnToHeading(-90, 1000, {.minSpeed=5, .earlyExitRange=5});
    load.set_value(true);
    chassis.moveToPoint(-27, -38, 1000, {.maxSpeed=60});
    pros::delay(1400);
// Score
    chassis.moveToPoint(20, -40.5, 1000, {.forwards=false,.maxSpeed=100,
.earlyExitRange=15});
    hood.move(-20);
    intake.move(-40);
    pros::delay(1100);
    hood.move(127);
    intake.move(127);
    pros::delay(1300);
   // Collect 3 middle
    chassis.swingToHeading(40, DriveSide::RIGHT, 3000, {.minSpeed=10});
    hood.move(0);
    load.set_value(false);
    intake.move(127);
```

```
    chassis.moveToPoint(31, -18, 3000, {.minSpeed=15, .earlyExitRange=5});
    pros::delay(800);
    load.set_value(true);
// Collect 3 middle
    chassis.moveToPoint(30, 41, 1250, {.earlyExitRange=3});
    pros::delay(50);
    load.set_value(false);
    pros::delay(900);
    load.set_value(true);
// Score midgoal
    chassis.turnToHeading(-44, 1000);
    chassis.moveToPoint(40,11,750,  {.forwards=false});
    intake.move(-40);
    hood.move(-40);
    pros::delay(750);
    intake.move(127);
    hood.move(75);
    pros::delay(500);
    load.set_value(false);
// Line up Long Goal
     chassis.moveToPoint(0, 57, 1500);
     hood.move(0);
// Score
     chassis.turnToHeading(-90,1000, {.minSpeed=10, .earlyExitRange=1});
     chassis.moveToPoint(42.5, 56, 750, {.forwards=false,.maxSpeed=100,});
    pros::delay(750);
    hood.move(127);
}

void SkillsAutonomous() {

     // set position to x:0, y:0, heading:0
    chassis.setPose(0, 0, 0);
    wing.set_value(true);
    intake.move(127);
    // Get Matchload
    chassis.moveToPoint(0, -38, 1000,  {.forwards=false, .earlyExitRange=15});
    chassis.turnToHeading(-90, 1000, {.minSpeed=5, .earlyExitRange=5});
```

```
    load.set_value(true);
    chassis.moveToPoint(-27, -38, 1000, {.maxSpeed=60});
    pros::delay(3000);
// Score
    chassis.moveToPoint(21, -40.5, 1200, {.forwards=false,.maxSpeed=100,
.earlyExitRange=15});
    hood.move(-20);
    intake.move(-40);
    pros::delay(1100);
    hood.move(127);
    intake.move(127);
    pros::delay(3000);
   // Collect 3 middle
    chassis.swingToHeading(40, DriveSide::RIGHT, 3000, {.minSpeed=10});
    hood.move(0);
    load.set_value(false);
    intake.move(127);
    chassis.moveToPoint(31, -18, 3000);
    pros::delay(800);
    load.set_value(true);
// Collect 3 middle
    chassis.moveToPoint(30, 41, 1250, {.earlyExitRange=3});
    pros::delay(50);
    load.set_value(false);
    pros::delay(900);
    load.set_value(true);
// Score midgoal
    chassis.turnToHeading(-44, 1000);
    chassis.moveToPoint(40,11,750,  {.forwards=false});
    intake.move(-40);
    hood.move(-40);
    pros::delay(750);
    intake.move(127);
    hood.move(75);
    pros::delay(500);
    load.set_value(false);
// Line up Long Goal
    chassis.turnToHeading(-55, 500, {.minSpeed=15, .earlyExitRange=5});
```

```
    chassis.moveToPoint(0, 57, 1500);
    hood.move(0);
// Score
    chassis.turnToHeading(-90,1000, {.minSpeed=10, .earlyExitRange=1});
    chassis.moveToPoint(42.5, 56, 750, {.forwards=false,.maxSpeed=100,});
    pros::delay(750);
    hood.move(127);
    pros::delay(1500);
    hood.move(0);



    chassis.turnToHeading(-90,1000, {.minSpeed=10, .earlyExitRange=1});
    load.set_value(true);
    chassis.moveToPoint(-40, 55, 1200, {.maxSpeed=70});
    chassis.turnToHeading(-90,1000);
    pros::delay(3000);

    chassis.moveToPoint(-10, 56, 1200, {.forwards=false});

    chassis.moveToPoint(0, -50, 3000,  {.forwards=false, .earlyExitRange=15});
    chassis.turnToHeading(-90, 1000);

    chassis.moveToPoint(20, -40.5, 1000, {.forwards=false,.maxSpeed=100,
.earlyExitRange=15});
    hood.move(-20);
    intake.move(-40);
    pros::delay(1100);
    hood.move(127);
    intake.move(127);
    pros::delay(3000);
   // Line up Park
    chassis.swingToHeading(0, DriveSide::RIGHT, 3000, {.minSpeed=10});
    load.set_value(false);
    chassis.moveToPoint(5, 7, 3000, {.minSpeed=15, .earlyExitRange=5});
    chassis.turnToHeading(-90, 1000);

    chassis.moveToPoint(-100, 7, 5000);
```

```
}



/**
 * Runs in driver control
 */
void opcontrol() {
    // loop to continuously update motors
    while (true) {
        // get joystick positions
        int leftY = controller.get_analog(pros::E_CONTROLLER_ANALOG_LEFT_Y);
        int rightX = controller.get_analog(pros::E_CONTROLLER_ANALOG_RIGHT_X);
        // move the chassis with arcade drive
        chassis.arcade(leftY, rightX);

        if (controller.get_digital(pros::E_CONTROLLER_DIGITAL_L1)) {
            intake.move(127);
        } else if (controller.get_digital(pros::E_CONTROLLER_DIGITAL_L2)) {
            intake.move(-127);
        } else {
            intake.move(0);
        }

        if (controller.get_digital(pros::E_CONTROLLER_DIGITAL_R1)) {
            hood.move(127);
        } else if (controller.get_digital(pros::E_CONTROLLER_DIGITAL_R2)) {
            hood.move(-127);
        } else {
            hood.move(0);
        }

       if (controller.get_digital_new_press(pros::E_CONTROLLER_DIGITAL_DOWN)) {
            loadState = !loadState;         // flip the state
            load.set_value(loadState);      // applies to piston
        }

        if (controller.get_digital_new_press(pros::E_CONTROLLER_DIGITAL_RIGHT)) {
```

```
            wingState = !wingState;        // flip the state
            wing.set_value(wingState);     // applies to piston
        }

        if (controller.get_digital_new_press(pros::E_CONTROLLER_DIGITAL_B)) {
            midState = !midState;        // flip the state
            mid.set_value(midState);     // applies to piston
        }

        // delay to save resources
        pros::delay(10);
    }
}
```